

STAJ ÇALIŞMA KAYNAĞI · BİRLEŞİK DERLEME

# STM32 ile Üç Fazlı BLDC Motor Kontrolü

Gömülü C temellerinden sensörsüz altı adımlı (six-step) komütasyon simülasyonuna kadar — staj boyunca hazırlanan yedi çalışma belgesi ve uygulama ekran görüntüleri, temelden en son aşamaya tek kaynakta.

HAZIRLAYAN

Yusuf Nalbant

TARİH

Eylül 2026

KAPSAM

7 belge · 12 ekran görüntüsü

YAPI

4 kısım · 7 bölüm

KARTLAR: STM32F401CCU6 · NUCLEO-G491RE

GÖMÜLÜ C

CUBEMX / CUBEIDE

SIX-STEP

UART TELEMETRİ

# Yol Haritası

Belgeler, staj boyunca hazırlandıkları gibi rastgele değil; bir öğrenme sırasına göre dizildi. Aşağıdaki dört kısım, hiç gömülü C yazmamış bir noktadan başlayıp gerçek karta yüklenen sensörsüz komütasyon simülasyonuna ve kendi izleme arayüzüne kadar ilerler. Soldan sağa okuyun; her kısım bir öncekinin üzerine kurulur.

## KISIM 1

### Temel Bilgi

Gömülü C, bellek, pin, timer, UART ve saat frekansı. Kod ve donanımın ortak dili.

## KISIM 2

### İlk STM32 Uygulaması

LED PWM, buton kesmesi ve state machine ile ilk uçtan uca çalışan proje.

## KISIM 3

### BLDC & Geri Besleme Teorisi

Altı adımlı komütasyon mantığı ve rotor konumunu okuyan encoder / Hall sensörleri.

## KISIM 4

### Simülasyon & Uygulama

Önce F401 ile donanımsız, sonra Nucleo-G491 ile telemetri ve kendi izleme arayüzü.

#### EKRAN GÖRÜNTÜLERİ HAKKINDA

Kısım 4'teki iki proje belgesinin ardından, o aşamada bilgisayarda alınan uygulama ekran görüntüleri açıklamalarıyla birlikte eklendi: CubeMX yapılandırması, CubeIDE derlemesi, karta yazma, CoolTerm telemetrisi ve staj sırasında yazılan kendi izleme uygulaması.

**Belge** — özgün çalışma dokümanı, orijinal biçiminde

**Uygulama görüntüsü** — ilgili aşamada alınan ekran görüntüsü

# İçindekiler

## KISIM 1 · TEMEL BİLGİ

- 01** Gömülü C'ye Sıfırdan  
STM32/ESP ile BLDC motor kontrolüne giden yol
- 02** Pin, Timer, UART ve Hertz  
STM32'de bir pinin, timer'ın ve seri hattın çalışması

## KISIM 2 · İLK STM32 UYGULAMASI

- 03** LED PWM Kontrolü ve Buton ile State Machine  
TIM2 PWM, EXTI/NVIC kesme ve 4 durumlu state machine

## KISIM 3 · BLDC VE GERİ BESLEME TEORİSİ

- 04** Üç Fazlı BLDC Motoru Six-Step Sürme  
Görsel anlatım: her kavram sıralı çizim kareleriyle
- 05** Encoder ve Hall Sensörleri  
Pozisyon, hız ve komütasyon için rotor konumu

## KISIM 4 · SİMÜLASYON VE UYGULAMA

- 06** Sensörsüz BLDC Komütasyon Simülasyonu  
STM32F401 · donanımsız · state machine + UART  
F401 uygulama görüntüleri [3 GÖRÜNTÜ](#)  
CubeMX pinout · saat · karta yazma · CoolTerm
- 07** NUCLEO-G491RE Üzerinde BLDC Komütasyonu  
CubeMX → CubeIDE → ST-LINK → CoolTerm zinciri  
Nucleo uygulama görüntüleri [9 GÖRÜNTÜ](#)  
Yapılandırma · telemetri · kendi izleme arayüzü

# 1

KISIM BİR

## Temel Bilgi

Gömülü yazılım, kavramların birbirine bağlandığı bir alandır. Bu kısım en baştan başlar: C dilinin bellekle ilişkisi, bir mikrodenetleyici pininin ne olduğu, timer'ın nasıl saydığı, iki cihazın UART ile nasıl konuştuğu ve hepsini birbirine bağlayan saat frekansı.

---

### BU KISIMDA

- 01** Gömülü C'ye Sıfırdan  
STM32/ESP ile BLDC motor kontrolüne giden yol
- 02** Pin, Timer, UART ve Hertz  
STM32 üzerinde temel çevre birimleri ve Hertz

# Gömülü C'ye Sıfırdan

STM32/ESP ile BLDC motor kontrolüne giden yol

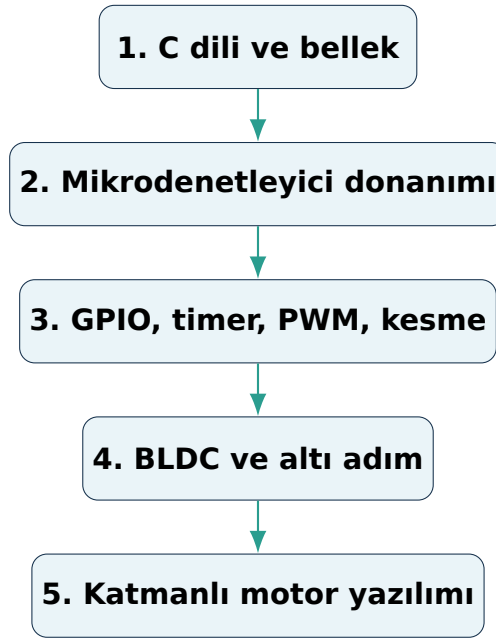
## Temelden projeye, sözdiziminden donanıma

Bu kaynak daha önce gömülü C kodu yazmamış bir bilgisayar mühendisliği öğrencisi için hazırlanmıştır. Her kavramda yalnızca “nasıl” değil, “neden” sorusu da cevaplanır.

**Hedef proje:** Üç fazlı BLDC motoru bir motor sürücü üzerinden altı adımlı komütasyonla güvenli biçimde harekete geçirmek; 60 elektriksel derecelik sektörleri anlamak ve kontrol yazılımını katmanlı kurmak.

# Bu kaynak nasıl kullanılmalı?

Bu kitabı baştan sona bir defada ezberlemeye çalışma. Gömülü yazılım, kavramların birbirine bağlandığı bir alandır. Önce küçük bir kodu okuyabilir hale gelir, sonra bir çevre birimini kontrol eder, en sonunda zamanlama ve güvenlik gerektiren motor kontrolüne geçersin.



Her bölümde şu dört kutu türünü göreceksin:

- **Sözdizimi:** Yazım biçimini parçalara ayırır.
- **Neden?:** Bu yapının projede hangi sorunu çözdüğünü açıklar.
- **Dikkat:** Donanım hasarı veya sessiz yazılım hatası oluşturabilecek noktaları belirtir.
- **Bölüm özeti:** Öğrenmiş olman gerekenleri kontrol eder.

## Elektriksel güvenlik sınırı

Motorun U, V ve W faz kabloları STM32 veya ESP pinlerine bağlanmaz. Mikrodenetleyici yalnızca sürücü kartının lojik girişlerini kontrol eder. Motor akımı; DC güç kaynağı, inverter MOSFET'leri ve motor fazları üzerinden akar. Gerçek motora enerji vermeden önce sürücünün veri sayfası, besleme gerilimi, akım limiti, dead-time ve hata girişleri doğrulanmalıdır.

---

# İçindekiler

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Bu kaynak nasıl kullanılmalı?</b>                                    | <b>i</b>  |
| <b>I C Dilinin Temeli</b>                                               | <b>1</b>  |
| <b>1 Gömülü sistem nedir?</b>                                           | <b>2</b>  |
| 1.1 Bilgisayardaki program ile gömülü program arasındaki fark . . . . . | 2         |
| 1.2 Bu projedeki fiziksel zincir . . . . .                              | 2         |
| 1.3 Bir mikrodenetleyici programının yaşamı . . . . .                   | 3         |
| <b>2 Derleme süreci ve ilk C programı</b>                               | <b>4</b>  |
| 2.1 Kaynak kod makine koduna nasıl dönüşür? . . . . .                   | 4         |
| 2.2 İlk programı parçalayalım . . . . .                                 | 4         |
| 2.3 Bildirim, ilk değer verme ve atama farklıdır . . . . .              | 5         |
| <b>3 Tipler, değişkenler ve operatörler</b>                             | <b>6</b>  |
| 3.1 Neden sabit genişlikli tipler? . . . . .                            | 6         |
| 3.2 İşaretli ve işaretli sayıların tuzağı . . . . .                     | 6         |
| 3.3 Temel operatörler . . . . .                                         | 7         |
| 3.4 Tür dönüşümü ve taşıma . . . . .                                    | 7         |
| <b>4 Karar yapıları ve döngüler</b>                                     | <b>8</b>  |
| 4.1 if, else if ve else . . . . .                                       | 8         |
| 4.2 switch ile durum seçmek . . . . .                                   | 8         |
| 4.3 for ve while . . . . .                                              | 9         |
| <b>5 Fonksiyonlar ve programı parçalara ayırmak</b>                     | <b>10</b> |
| 5.1 Fonksiyon neden vardır? . . . . .                                   | 10        |
| 5.2 Değer ile geçirme . . . . .                                         | 10        |
| 5.3 Tek sorumluluk ilkesi . . . . .                                     | 11        |
| <b>6 Bellek ve işaretçiler</b>                                          | <b>12</b> |
| 6.1 Değişken aslında nedir? . . . . .                                   | 12        |
| 6.2 Adres alma: & operatörü . . . . .                                   | 12        |
| 6.3 Adresteki değere erişme: * operatörü . . . . .                      | 12        |
| 6.4 NULL işaretçi . . . . .                                             | 13        |
| 6.5 const ile işaretçinin dört hali . . . . .                           | 13        |
| <b>7 Diziler, tablolar ve sınır güvenliği</b>                           | <b>15</b> |
| 7.1 Dizi nedir? . . . . .                                               | 15        |
| 7.2 Dizi adı ve işaretçi ilişkisi . . . . .                             | 15        |
| 7.3 Tablo tabanlı kontrol . . . . .                                     | 15        |
| 7.4 sizeof ile eleman sayısı . . . . .                                  | 16        |

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>8 struct, enum ve typedef</b>                         | <b>17</b> |
| 8.1 enum: sayılara anlamlı ad vermek . . . . .           | 17        |
| 8.2 struct: ilişkili verileri bir arada tutmak . . . . . | 17        |
| 8.3 typedef ne yapar? . . . . .                          | 17        |
| <b>9 static, const, volatile ve kapsam</b>               | <b>19</b> |
| 9.1 Kapsam ve yaşam süresi farklıdır . . . . .           | 19        |
| 9.2 Dosya seviyesinde static . . . . .                   | 19        |
| 9.3 const: değişmez niyet . . . . .                      | 19        |
| 9.4 volatile: program dışı değişiklik . . . . .          | 19        |
| 9.5 extern: başka derleme birimindeki sembol . . . . .   | 20        |
| <b>10 Header dosyaları ve katmanlı mimari</b>            | <b>21</b> |
| 10.1 Bildirim ve tanım . . . . .                         | 21        |
| 10.2 Include guard neden vardır? . . . . .               | 21        |
| 10.3 Açılı ve tırnaklı include farkı . . . . .           | 21        |
| 10.4 Bu proje için önerilen katmanlar . . . . .          | 22        |
| <b>II Mikrodenetleyici ve Donanımla Konuşmak</b>         | <b>23</b> |
| <b>11 İşlemci, bellek haritası ve register</b>           | <b>24</b> |
| 11.1 Mikrodenetleyicinin ana parçaları . . . . .         | 24        |
| 11.2 Memory-mapped I/O . . . . .                         | 24        |
| 11.3 Read-modify-write işlemi . . . . .                  | 25        |
| 11.4 Maske oluşturmak . . . . .                          | 25        |
| <b>12 GPIO: dijital giriş ve çıkış</b>                   | <b>26</b> |
| 12.1 Bir pin neden doğrudan çalışmaz? . . . . .          | 26        |
| 12.2 Mantıksal seviye fiziksel gerilimdir . . . . .      | 26        |
| 12.3 Atomik set/reset register'ı . . . . .               | 27        |
| <b>13 Timer, PWM ve dead-time</b>                        | <b>28</b> |
| 13.1 Timer nasıl sayar? . . . . .                        | 28        |
| 13.2 PWM duty nasıl oluşur? . . . . .                    | 28        |
| 13.3 Complementary PWM . . . . .                         | 28        |
| 13.4 Preload ve senkron güncelleme . . . . .             | 29        |
| <b>14 Kesme, ISR ve gerçek zaman</b>                     | <b>30</b> |
| 14.1 Polling ile kesme farkı . . . . .                   | 30        |
| 14.2 İyi ISR nasıl olmalı? . . . . .                     | 30        |
| 14.3 Yarış durumu örneği . . . . .                       | 31        |
| 14.4 Öncelik ve gecikme . . . . .                        | 31        |
| <b>15 ADC, akım ölçümü ve hata koruması</b>              | <b>32</b> |
| 15.1 ADC'nin görevi . . . . .                            | 32        |
| 15.2 PWM ile senkron örnekleme . . . . .                 | 32        |
| 15.3 Yazılım ve donanım koruması birlikte . . . . .      | 32        |
| <b>16 Hata ayıklama ve ölçüm araçları</b>                | <b>33</b> |
| 16.1 Derleyici uyarıları ilk savunmadır . . . . .        | 33        |
| 16.2 Debugger ile bakılacaklar . . . . .                 | 33        |
| 16.3 Osiloskop ve logic analyzer . . . . .               | 33        |

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>III BLDC Motor Kontrolüne Geçiş</b>                        | <b>35</b> |
| <b>17 BLDC motorun temel çalışma mantığı</b>                  | <b>36</b> |
| 17.1 Stator, rotor ve üç faz . . . . .                        | 36        |
| 17.2 Elektriksel ve mekanik açı . . . . .                     | 36        |
| 17.3 Torku ne belirler? . . . . .                             | 37        |
| <b>18 Üç faz inverter ve altı MOSFET</b>                      | <b>38</b> |
| 18.1 Bir yarım köprü . . . . .                                | 38        |
| 18.2 Üç yarım köprü bir arada . . . . .                       | 39        |
| 18.3 Üç fazın üç durumu . . . . .                             | 39        |
| <b>19 Altı adımlı komütasyon</b>                              | <b>40</b> |
| 19.1 Neden altı adım? . . . . .                               | 40        |
| 19.2 Birinci adımı fiziksel okuyalım . . . . .                | 40        |
| 19.3 Yön nasıl ters çevrilir? . . . . .                       | 40        |
| 19.4 Komütasyon zamanını kim belirler? . . . . .              | 41        |
| <b>20 Hall ve sensorless BEMF</b>                             | <b>42</b> |
| 20.1 IMU neden gerekli değildir? . . . . .                    | 42        |
| 20.2 Hall kodları . . . . .                                   | 42        |
| 20.3 Sadece üç faz kablosu varsa . . . . .                    | 42        |
| 20.4 60° olayından hız hesabı . . . . .                       | 43        |
| <b>21 Sürücü kartını tanımadan kod yazma</b>                  | <b>44</b> |
| 21.1 Veri sayfasında aranacak bilgiler . . . . .              | 44        |
| <b>22 Motor yazılımı için durum makinesi</b>                  | <b>45</b> |
| 22.1 Neden tek bir while döngüsü yetmez? . . . . .            | 45        |
| 22.2 Her durumun sorumluluğu . . . . .                        | 45        |
| 22.3 Durum geçiş koşulları . . . . .                          | 45        |
| <b>IV Basitten Zora Kodlama Uygulamaları</b>                  | <b>47</b> |
| <b>23 Seviye 1: PC'de gate maskesi simülasyonu</b>            | <b>48</b> |
| 23.1 Satırların mantığı . . . . .                             | 49        |
| 23.2 Kendin dene . . . . .                                    | 49        |
| <b>24 Seviye 2: Komütasyon tablosu</b>                        | <b>50</b> |
| 24.1 Faz durumlarını tanımlamak . . . . .                     | 50        |
| 24.2 Altı satırlık değişmez tablo . . . . .                   | 50        |
| 24.3 Bir faz durumunu gate bitine çevirmek . . . . .          | 50        |
| <b>25 Seviye 3: Fonksiyon işaretçisiyle donanım soyutlama</b> | <b>52</b> |
| 25.1 Problem . . . . .                                        | 52        |
| 25.2 Fonksiyon işaretçisi tipi . . . . .                      | 52        |
| 25.3 PC uygulaması . . . . .                                  | 52        |
| 25.4 Kontrol katmanının çağırısı . . . . .                    | 53        |
| <b>26 Seviye 4: ISR posta kutusu ve olay işleme</b>           | <b>54</b> |
| 26.1 Paylaşılan veri yapısı . . . . .                         | 54        |
| 26.2 Erken dönüşler neden kullanıldı? . . . . .               | 55        |
| 26.3 Bir event bayrağının sınırı . . . . .                    | 55        |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>27 Seviye 5: Hız hesabı ve hata yolu</b>     | <b>56</b> |
| 27.1 İlk ölçüm neden ayrı ele alınır? . . . . . | 56        |
| 27.2 Merkezi hata fonksiyonu . . . . .          | 57        |
| <b>28 Tam programın çalışma sırası</b>          | <b>58</b> |
| 28.1 main fonksiyonunun görevi . . . . .        | 58        |
| 28.2 Tek Hall olayının yolculuğu . . . . .      | 59        |
| 28.3 Kodun kavram haritası . . . . .            | 59        |
| <b>29 STM32'ye taşıma rehberi</b>               | <b>60</b> |
| 29.1 Hangi kod aynı kalır? . . . . .            | 60        |
| 29.2 Kavramsal STM32 backend . . . . .          | 60        |
| 29.3 HAL, LL veya register? . . . . .           | 61        |
| 29.4 ESP32 farkı . . . . .                      | 61        |
| <b>30 Laboratuvar bring-up planı</b>            | <b>62</b> |
| 30.1 Aşama 0: Doküman ve bağlantı . . . . .     | 62        |
| 30.2 Aşama 1: Motor bağlı değilken . . . . .    | 62        |
| 30.3 Aşama 2: Düşük enerji . . . . .            | 62        |
| 30.4 Aşama 3: Kapalı çevrime geçiş . . . . .    | 63        |
| <b>31 Sık hatalar ve teşhis yolu</b>            | <b>64</b> |
| <b>32 Mini alıştırmalar</b>                     | <b>65</b> |
| Alıştırma 1: Bit maskesi . . . . .              | 65        |
| Alıştırma 2: İşaretçi . . . . .                 | 65        |
| <b>33 Alıştırma çözümleri</b>                   | <b>66</b> |
| <b>34 Son kontrol listesi ve çalışma planı</b>  | <b>67</b> |
| 34.1 İlk hafta . . . . .                        | 67        |
| 34.2 İkinci hafta . . . . .                     | 67        |
| 34.3 Üçüncü hafta . . . . .                     | 67        |
| 34.4 Dördüncü hafta . . . . .                   | 67        |
| <b>A Kısa sözlük</b>                            | <b>69</b> |
| <b>B Başvuru kaynakları</b>                     | <b>71</b> |

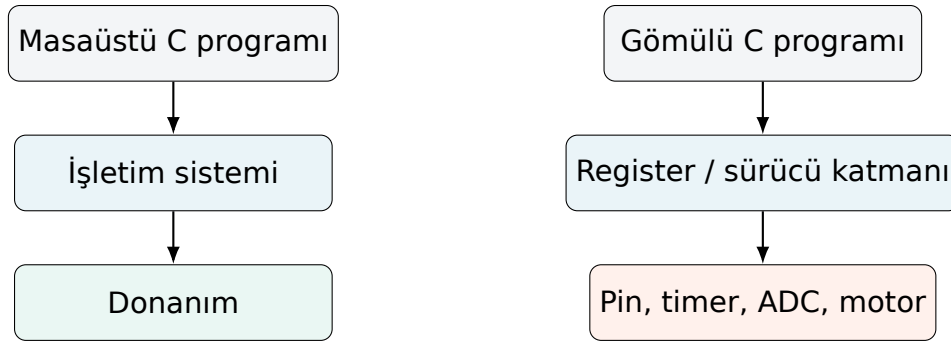
**Kısım I**

**C Dilinin Temeli**

## Gömülü sistem nedir?

### 1.1 Bilgisardaki program ile gömülü program arasındaki fark

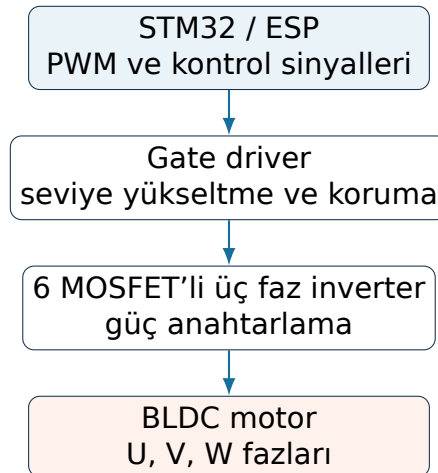
Masaüstü uygulamasında işletim sistemi; ekranı, dosyaları, belleği ve zamanı senin adına yönetir. Mikrodenetleyicide ise program çoğu zaman doğrudan donanımla konuşur. Bir pine 1 yazdığında fiziksel gerilim değişebilir; yanlış iki MOSFET'i açtığında kısa devre oluşabilir. Bu nedenle gömülü C'de bir satır kodun hem yazılımsal hem elektriksel karşılığı vardır.



Şekil 1.1: Masaüstü ve yalın gömülü programın sadeleştirilmiş farkı

### 1.2 Bu projedeki fiziksel zincir

BLDC projesinde bağlantı zinciri şöyledir:



### Neden?

Bu zinciri anlamadan kod yazmak tehlikelidir. Mikrodenetleyici motor akımını taşımaz; yalnızca güç katmanının ne zaman anahtarlanaacağını söyler. Bu yüzden “motor kablosunu STM’ye bağlarım” düşüncesi yanlıştır.

## 1.3 Bir mikrodenetleyici programının yaşamı

Enerji verildiğinde işlemci reset vektöründen başlar, başlangıç kodu RAM’i hazırlar ve sonra main() fonksiyonuna gelir. Basit bir yalın-metal program çoğunlukla hiç bitmeyen döngü içerir:

Listing 1.1: Bir gömülü programın en sade iskeleti

```
1 #include <stdint.h>
2
3 int main(void)
4 {
5     donanimi_hazirla();
6
7     while (1)
8     {
9         olaylari_isle();
10    }
11 }
```

### Sözdizimi

int main(void) ifadesinde int, fonksiyonun dönüş tipidir; main fonksiyonun adıdır; parantez içindeki void, parametre almadığını açıkça belirtir. Süslü parantezler fonksiyon gövdesini sınırlar. Her ifade çoğunlukla noktalı virgülle biter.

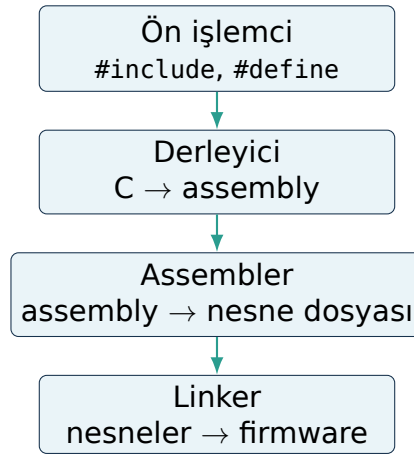
### Bölüm özeti

- Mikrodenetleyici kodu fiziksel donanım durumunu değiştirebilir.
- MCU, gate driver ve inverter aynı şey değildir.
- Yalın-metal uygulamada while (1) ana çalışma döngüsüdür.

## Derleme süreci ve ilk C programı

### 2.1 Kaynak kod makine koduna nasıl dönüşür?

Yazdığın .c dosyası işlemcinin doğrudan anlayacağı biçimde değildir. Araç zinciri dört temel aşamada çalışır:



Şekil 2.1: C kaynak kodunun firmware'e dönüşmesi

#### Neden?

"Undefined reference" hatası genellikle sözdizimi hatası değil, linker hatasıdır: fonksiyon bildirilmiştir ama tanımı bağlanan dosyalarda yoktur. "Unknown type name" veya eksik noktalı virgül ise çoğunlukla derleyici aşamasında yakalanır.

### 2.2 İlk programı parçalayalım

Listing 2.1: Değişken, işlem ve fonksiyon çağırısı

```

1 #include <stdint.h>
2 #include <stdio.h>
3
4 int main(void)
5 {
6     uint32_t sayac = 0u;
7
8     sayac = sayac + 1u;
9     printf("sayac = %u\n", (unsigned)sayac);
10

```

```

11     return 0;
12 }

```

| Parça               | Anlamı                                                                                  |
|---------------------|-----------------------------------------------------------------------------------------|
| #include <stdint.h> | Sabit genişlikli tamsayı tiplerinin bildirimlerini ekler.                               |
| uint32_t            | Tam 32 bitlik işaretli tamsayı tipidir.                                                 |
| sayac               | Değişkenin programcı tarafından seçilen adıdır.                                         |
| = 0u                | İlk değer atamasıdır. u, sabitin işaretli olduğunu belirtir.                            |
| sayac + 1u          | Mevcut değer okunur, bir eklenir.                                                       |
| printf(...)         | PC testinde ekrana yazar; yalın MCU'da kendiliğinden ekran oluşturmaz.                  |
| return 0;           | Programın başarılı bittiğini bildirir; gömülü ana döngüde çoğu zaman bu satıra ulaşmaz. |

## 2.3 Bildirim, ilk değer verme ve atama farklıdır

```

1 uint32_t hiz;          /* Bildirim: hiz 1adnda alan 1ayr. */
2 uint32_t hedef = 0;    /* Bildirim + ilk ğdeer verme. */
3
4 hiz = 1200;            /* Atama: var olan ğşdeikeni ğşdeitir. */

```

### Dikkat

Otomatik yerel değişken ilk değer verilmeden okunursa belirsiz değer içerir. Derleyici uyarılarını açmak bu tür hataların önemli bir bölümünü erkenden yakalar: -Wall -Wextra -Wpedantic.

## Tipler, değişkenler ve operatörler

### 3.1 Neden sabit genişlikli tipler?

C standardında `int` tipinin kaç bit olduğu hedefe göre değişebilir. Register ve haberleşme paketi işlerken genişliğin kesin olması gerekir. Bu yüzden `stdint.h` tipleri tercih edilir.

| Tip                   | Yaklaşık aralık          | Projede kullanım                    |
|-----------------------|--------------------------|-------------------------------------|
| <code>uint8_t</code>  | 0 ... 255                | Hall kodu, byte, küçük durum değeri |
| <code>int8_t</code>   | -128 ... 127             | -1 gibi geçersiz tablo değeri       |
| <code>uint16_t</code> | 0 ... 65535              | ADC sonucu, küçük timer             |
| <code>uint32_t</code> | 0 ... $2^{32} - 1$       | Register, zaman sayacı, RPM         |
| <code>int32_t</code>  | $-2^{31} ... 2^{31} - 1$ | İşaretli hata ve fark hesabı        |
| <code>uint64_t</code> | Çok geniş                | Ara hesapta taşmayı önleme          |
| <code>float</code>    | Yaklaşık 7 basamak       | Fiziksel büyüklük, gerektiğinde     |

#### Neden her yerde float kullanmıyoruz?

Bazı mikrodenetleyiciler kayan nokta işlemlerini yavaş yapar; ayrıca karşılaştırma ve gerçek zamanlı süre açısından tamsayı daha öngörülebilir olabilir. Örneğin duty değerini yüzde yerine 0-1000 arası tamsayıyla tutmak mümkündür. Yine de ölçüm ve kontrol algoritmasında gerektiğinde float kullanmak yanlış değildir; hedef işlemci ve süre bütçesi belirleyicidir.

### 3.2 İşaretli ve işaretsiz sayıların tuzağı

```
1 uint32_t a = 0u;
2 a = a - 1u; /* Sonuç -1 ğdeil, 4294967295 olur. */
```

İşaretsiz aritmetik modüler çalışır. Bu özellik timer taşmasını hesaplamak için faydalı olabilir:

```
1 uint32_t gecen = simdiki_tick - onceki_tick;
```

Sayaç bir kez taşsa bile iki değer arasındaki süre, uygun aralıkta kaldığı sürece doğru hesaplanır.

### 3.3 Temel operatörler

| Grup          | Operatörler       | Örnek anlam                                  |
|---------------|-------------------|----------------------------------------------|
| Aritmetik     | + - * / %         | Toplama, çıkarma, çarpma, bölme, kalan       |
| Karşılaştırma | == != < > <= >=   | İki değeri karşılaştırır; sonuç doğru/yanlış |
| Mantıksal     | &&    !           | Koşulları birleştirir                        |
| Bit düzeyi    | &   ^ ~ << >>     | Register bitlerini değiştirir                |
| Atama         | = + = - = & =   = | Değişkene yeni değer yazar                   |
| Adres         | & *               | Adres alma ve işaret edilen değere erişim    |
| Üye erişimi   | . ->              | Yapı nesnesi veya yapı işaretçisi üyesi      |

#### En sık başlangıç hatası

= atama yapar, == karşılaştırır. `if (state = MOTOR_RUN)` yazmak `state`'i değiştirir ve çoğu zaman koşulu doğru yapar. Doğrusu `if (state == MOTOR_RUN)` biçimidir.

### 3.4 Tür dönüşümü ve taşıma

RPM hesabında önce çarpma yapılırsa 32 bit ara değer taşabilir. Bu yüzden en az bir terim 64 bit yapılır:

```
1 uint64_t pay = 60ull * timer_frequency_hz;
2 uint32_t rpm = (uint32_t)(pay / payda);
```

Parantez içindeki `uint32_t` açık tür dönüşümüdür. Sonucun bu tipe sığdığına programcı karar vermiş olur.

## Karar yapıları ve döngüler

### 4.1 if, else if ve else

```
1 if (akim_ma > akim_limiti_ma)
2 {
3     motoru_kapat();
4 }
5 else if (baslat_komutu != 0u)
6 {
7     motoru_baslat();
8 }
9 else
10 {
11     motoru_beklet();
12 }
```

Koşul sırası önemlidir. Güvenlik kontrolü başlatma komutundan sonra olsaydı motor kısa süre de olsa tehlikeli durumda çalıştırılabilirdi.

### 4.2 switch ile durum seçmek

```
1 switch (motor_state)
2 {
3     case MOTOR_IDLE:
4         /* Bekle. */
5         break;
6
7     case MOTOR_RUN:
8         motoru_islet();
9         break;
10
11    case MOTOR_FAULT:
12        gate_cikislarini_kapat();
13        break;
14
15    default:
16        motor_state = MOTOR_FAULT;
17        break;
18 }
```

**Sözdizimi**

case belirli bir değere karşılık gelen dalı seçer. break, o daldan çıkmayı sağlar. Unutulursa yürütme sonraki case'e düşer. Bazen bu bilerek yapılır; başlangıçta ise çoğunlukla hatadır.

**4.3 for ve while**

```
1 for (uint32_t i = 0u; i < 6u; ++i)
2 {
3     tabloyu_kontrol_et(i);
4 }
5
6 while (1)
7 {
8     motor_control_process();
9 }
```

for üç parçadan oluşur: başlangıç, devam koşulu ve her tur sonundaki değişiklik. while ise yalnızca koşula sahiptir.

**Bloklayan gecikme**

Uzun delay() çağrıları başlangıç denemesinde kolaydır fakat motor kontrolünde ana döngünün hata ve sensör olaylarını zamanında işlemesini engelleyebilir. Gerçek zamanlı projede timer, durum makinesi ve kesme tabanlı zamanlama tercih edilir.

## Fonksiyonlar ve programı parçalara ayırmak

### 5.1 Fonksiyon neden vardır?

Fonksiyon, tek bir görevi isimlendirir. Aynı işlemi kopyalamayı önler, test edilebilirliği artırır ve ana akışı okunur hale getirir.

```

1 static uint32_t rpm_hesapla(
2     uint32_t timer_hz,
3     uint32_t sector_ticks,
4     uint8_t pole_pairs)
5 {
6     if ((sector_ticks == 0u) || (pole_pairs == 0u))
7     {
8         return 0u;
9     }
10
11     uint64_t pay = 60ull * timer_hz;
12     uint64_t payda = 6ull * pole_pairs * sector_ticks;
13     return (uint32_t)(pay / payda);
14 }
```

#### Fonksiyon imzasını oku

static fonksiyonun bu kaynak dosyaya özel olduğunu; uint32\_t geri dönüş tipini; rpm\_hesapla adını; parantez içindekiler ise parametreleri belirtir. return, sonucu çağırana koda döndürür.

### 5.2 Değer ile geçirme

C'de parametreler varsayılan olarak kopyalanır:

```

1 void bir_artir(uint32_t x)
2 {
3     x = x + 1u;
4 }
5
6 uint32_t sayi = 5u;
7 bir_artir(sayi); /* sayi hala 5 */
```

Çağıranın değişkenini değiştirmek için adresi gönderilir; bu konu işaretçiler bölümünde açıklanacaktır.

### 5.3 Tek sorumluluk ilkesi

İyi bir motor kontrol modülünde görevler ayrılır:

- `read_sensors()`: ölçümleri alır.
- `select_step()`: komütasyon adımını seçer.
- `write_gates()`: donanıma uygular.
- `enter_fault()`: güvenli hata durumuna geçirir.

Bu ayırım sayesinde hesaplama kodu PC'de test edilirken gerçek MOSFET'lere enerji verilmez.

## Bellek ve işaretçiler

### 6.1 Değişken aslında nedir?

Bir değişken yalnızca isim değildir; bellekte belirli sayıda byte kaplayan bir bölgedir. İsim, derleme sırasında bu bölgeye erişmeyi kolaylaştırır. İşlemci çalışırken asıl önemli olan adrestir.

| Adres      | Bellekteki değer |
|------------|------------------|
| 0x20000020 | 0x78             |
| 0x20000021 | 0x56             |
| 0x20000022 | 0x34             |
| 0x20000023 | 0x12             |

uint32\_t sayi = 0x12345678

Şekil 6.1: 32 bit bir değişkenin little-endian bellekte örnek görünümü

### 6.2 Adres alma: & operatörü

```
1 uint32_t hiz = 1200u;
2 uint32_t *hiz_adresi = &hiz;
```

Satırları parçalara ayıralım:

- `uint32_t *`: 32 bitlik işaretsiz bir değerın adresini tutabilen tip.
- `hiz_adresi`: İşaretçi değişkeninin adı.
- `&hiz`: `hiz` değişkeninin bellekteki adresi.

### 6.3 Adresteki değere erişme: \* operatörü

```
1 *hiz_adresi = 1500u;
```

Bu satır “`hiz_adresi`’nin gösterdiği yere 1500 yaz” demektir. Sonuç olarak `hiz` değişkeni 1500 olur.



İşaretçi de bir değişkendir.

Gösterilen nesne

### Motor kodunda işaretçi neden gerekli?

`motor_control_start(&motor)` çağrısında yapının kopyası değil adresi gönderilir. Fonksiyon aynı motor nesnesinin durumunu değiştirir. Büyük yapıların kopyalanması önlenir ve donanım register adreslerine erişmek mümkün olur.

## 6.4 NULL işaretçi

NULL, geçerli bir nesneyi göstermeyen işaretçiye temsil eder:

```

1 void motor_start(MotorControl *motor)
2 {
3     if (motor == NULL)
4     {
5         return;
6     }
7
8     motor->state = MOTOR_RUN;
9 }
  
```

`motor->state`, şu ifadenin daha kısa yazımıdır:

```

1 (*motor).state = MOTOR_RUN;
  
```

Önce `*motor` ile yapı bulunur, sonra `.state` üyesine erişilir.

### Dikkat

NULL veya geçersiz bir adresi `*` ile açmak tanımsız davranıştır. PC'de program çökebilir; mikrodenetleyicide HardFault oluşabilir veya daha kötüsü yanlış çevre birimi değişebilir.

## 6.5 const ile işaretçinin dört hali

| Bildirim                             | Anlam                                                                                         |
|--------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>uint32_t *p</code>             | Değer ve işaretçinin yönü değişebilir.                                                        |
| <code>const uint32_t *p</code>       | Gösterilen değer bu işaretçi üzerinden değiştirilemez; işaretçi başka yere yönlendirilebilir. |
| <code>uint32_t *const p</code>       | İşaretçi başka adrese yönlendirilemez; gösterilen değer değiştirilebilir.                     |
| <code>const uint32_t *const p</code> | Ne adres ne de değer bu isim üzerinden değiştirilebilir.                                      |

Donanım örneğinde sık görülen biçim:

```
1 static volatile uint32_t *const gate_register =  
2   (volatile uint32_t *)0x40010000u;
```

Burada işaretçi hep aynı donanım adresini gösterir, fakat adresteki register değeri değişebilir.

## Diziler, tablolar ve sınır güvenliği

### 7.1 Dizi nedir?

Dizi aynı tipte art arda yerleştirilmiş nesneler topluluğudur:

```
1 uint8_t hall_sequence[6] =
2 {
3     0x01u, 0x05u, 0x04u,
4     0x06u, 0x02u, 0x03u
5 };
```

Geçerli indeksler 0 ile 5 arasındadır. Altı elemanlı dizide `hall_sequence[6]` yoktur.

#### Sınır dışına yazma

C dizinin sınırını çalışma anında otomatik kontrol etmez. Sınır dışı yazma başka değişkeni, stack'i veya dönüş adresini bozabilir. İndeksin her zaman tablo boyutundan küçük olduğunu doğrula.

### 7.2 Dizi adı ve işaretçi ilişkisi

Çoğu ifadeye dizi adı ilk elemana işaretçiye dönüşür:

```
1 uint8_t values[3] = {10u, 20u, 30u};
2 uint8_t *p = values;
3
4 uint8_t a = values[1]; /* 20 */
5 uint8_t b = *(p + 1); /* 20 */
```

İşaretçi aritmetiğinde `p + 1`, bir byte değil, gösterilen tipin bir elemanı kadar ilerler.

### 7.3 Tablo tabanlı kontrol

Altı adımı altı ayrı if ile yazmak mümkündür; ancak tablo daha açık ve denetlenebilirdir:

```
1 static const int8_t hall_to_step[8] =
2 {
3     -1, 0, 4, 5, 2, 1, 3, -1
4 };
5
6 int8_t step = hall_to_step[hall_code];
```

**Neden?**

Hall kodu 3 bit olduğu için 0-7 arasındır; bu yüzden sekiz elemanlı tablo doğal bir eşlemedir. -1, 000 ve 111 gibi geçersiz girişleri ifade eder. Tablo, kablo sırası değiştiğinde kontrol akışını yeniden yazmadan eşlemeyi değiştirmeyi sağlar.

**7.4 sizeof ile eleman sayısı**

```
1 #define ARRAY_LEN(a) (sizeof(a) / sizeof((a)[0]))
2
3 for (size_t i = 0u; i < ARRAY_LEN(hall_sequence); ++i)
4 {
5     /* ... */
6 }
```

Bu makro yalnızca gerçek dizi aynı kapsamdayken güvenlidir. Fonksiyon parametresine geçen dizi işaretçiye dönüştüğü için orada toplam boyut kaybolur.

## struct, enum ve typedef

### 8.1 enum: sayılara anlamlı ad vermek

```
1 typedef enum
2 {
3     MOTOR_IDLE,
4     MOTOR_ALIGN,
5     MOTOR_STARTUP,
6     MOTOR_RUN,
7     MOTOR_FAULT
8 } MotorState;
```

Derleyici arka planda tamsayı kullanır, fakat kodda 3 yerine MOTOR\_RUN görülür. Bu hata olasılığını azaltır.

### 8.2 struct: ilişkili verileri bir arada tutmak

```
1 typedef struct
2 {
3     MotorState state;
4     uint8_t current_step;
5     uint16_t duty_per_mille;
6     uint32_t measured_rpm;
7     uint32_t fault_flags;
8 } MotorControl;
```

Bir nesne oluşturmak:

```
1 MotorControl motor =
2 {
3     .state = MOTOR_IDLE,
4     .current_step = 0u,
5     .duty_per_mille = 0u,
6     .measured_rpm = 0u,
7     .fault_flags = 0u
8 };
```

Noktalı ilk değer verme, hangi değerın hangi üyeye gittiğini açıkça gösterir.

### 8.3 typedef ne yapar?

typedef yeni bir tip oluşturmaz; mevcut tipe okunabilir bir takma ad verir:

```
1 typedef uint32_t Tick;  
2 typedef void (*FaultCallback)(uint32_t flags);
```

İkinci satır bir fonksiyon işaretçisi tipine ad verir. Bu konu donanım soyutlama bölümünde kullanılacaktır.

### Neden?

Motorun state, RPM ve fault değerlerini ayrı global değişkenler yapmak yerine tek MotorControl yapısında tutmak; birden fazla motor eklemeyi, fonksiyonlara bağlam göndermeyi ve test yazmayı kolaylaştırır.

## static, const, volatile ve kapsam

### 9.1 Kapsam ve yaşam süresi farklıdır

**Kapsam**, bir isme kodun nerelerinden erişilebildiğini; **yaşam süresi**, nesnenin bellekte ne kadar kaldığını anlatır.

```
1 static uint32_t module_counter; /* Dosya 1kapsam, program boyu şyaam. */
2
3 static void helper(void)      /* 1Yalnız bu .c 1dosyasından çağırılır. */
4 {
5     static uint32_t call_count; /* Yerel kapsam, program boyu şyaam. */
6     uint32_t temporary = 0u;    /* Blok 1kapsam, çağır süresince şyaam. */
7
8     ++call_count;
9     ++temporary;
10 }
```

### 9.2 Dosya seviyesinde static

static bir global değişken veya fonksiyonda kullanılırsa sembolü dosya dışına kapatır. Bu, modülün iç ayrıntılarını saklar:

```
1 static void enter_fault(MotorControl *motor)
2 {
3     motor->state = MOTOR_FAULT;
4 }
```

Başka dosyaların yalnızca public fonksiyonları kullanması, bağımlılığı azaltır.

### 9.3 const: değişmez niyet

```
1 static const CommutationStep forward_table[6] =
2 {
3     /* ... */
4 };
```

const, tablonun kazara değiştirilmesini derleme hatasına dönüştürür. Gömülü linker çoğunlukla bunu Flash'taki salt okunur bölüme koyar.

### 9.4 volatile: program dışı değişiklik

```
1 static volatile uint8_t event_pending;
2
3 void hall_isr(void)
4 {
5     event_pending = 1u;
6 }
7
8 int main(void)
9 {
10     while (1)
11     {
12         if (event_pending != 0u)
13         {
14             event_pending = 0u;
15             process_event();
16         }
17     }
18 }
```

Derleyici, normal akışta değişmediğini düşündüğü bir değeri tekrar okumayabilir. Kesme, DMA veya donanım register'ı bu değeri dışarıdan değiştirebildiği için volatile her erişimin gerçekten yapılmasını ister.

#### volatile ne sağlamaz?

volatile; atomiklik, karşılıklı dışlama, işlem sırası kilidi veya çok alanlı tutarlı anlık görüntü sağlamaz. ISR üç değişkeni güncellerken ana döngü bunların yarısını eski, yarısını yeni okuyabilir. Kritik bölüm, atomik protokol veya kuyruk gerekebilir.

## 9.5 extern: başka derleme birimindeki sembol

Header'da:

```
1 extern volatile uint32_t system_ticks;
```

Yalnız bir .c dosyasında tanım:

```
1 volatile uint32_t system_ticks = 0u;
```

Her header'a doğrudan tanım yazmak linker'da çoklu tanım hatası oluşturabilir. Yine de mümkün olduğunca global extern yerine erişim fonksiyonları ve bağlam yapıları tercih edilir.

## Header dosyaları ve katmanlı mimari

### 10.1 Bildirim ve tanım

- **Bildirim:** Bir isim ve tipin var olduğunu söyler.
- **Tanım:** Fonksiyon gövdesini veya nesnenin gerçek depolamasını oluşturur.

Listing 10.1: motor\_control.h içinde public bildirim

```
1 #ifndef MOTOR_CONTROL_H
2 #define MOTOR_CONTROL_H
3
4 #include <stdint.h>
5
6 typedef struct MotorControl MotorControl;
7
8 void motor_control_init(MotorControl *motor);
9 void motor_control_start(MotorControl *motor);
10 void motor_control_process(MotorControl *motor);
11
12 #endif
```

Listing 10.2: motor\_control.c içinde tanım

```
1 #include "motor_control.h"
2
3 void motor_control_start(MotorControl *motor)
4 {
5     /* Fonksiyonun gerçek gövdesi. */
6 }
```

### 10.2 Include guard neden vardır?

Bir header dolaylı olarak birkaç kez eklenebilir. `#ifndef`, `#define` ve `#endif` üçlüsü içeriğin derleme biriminde yalnız bir kez işlenmesini sağlar.

### 10.3 Açılı ve tırnaklı include farkı

```
1 #include <stdint.h>           /* Sistem/standart header. */
2 #include "motor_control.h"    /* Projeye ait header. */
```

Arama yollarının kesin davranışı araç zincirine bağlıdır; genel niyet bu ayrımdır.

## 10.4 Bu proje için önerilen katmanlar

**Uygulama: başlat/durdur, hedef hız, telemetri**

**Kontrol: state machine, komütasyon, hız hesabı**

**Donanım soyutlama: PWM, capture, ADC, fault**

**STM32/ESP çevre birimleri ve motor sürücü**

### Neden?

Kontrol algoritmasının içinde doğrudan belirli STM32 register'ı kullanılırsa aynı algoritmayı PC'de test etmek veya ESP'ye taşımak zorlaşır. Donanım katmanı değişir, kontrol mantığı mümkün olduğunca değişmeden kalır.

### Birinci kısım kontrol listesi

Şunları açıklayabiliyorsan mikrodenetleyici bölümüne geçebilirsin:

- `uint32_t *p = &x;` satırında `*` ve `&` ne yapar?
- `motor->state` neden nokta yerine ok operatörü kullanır?
- `static const` bir komütasyon tablosuna ne kazandırır?
- ISR'in değiştirdiği bayrak neden `volatile` olur ve bu neden tek başına yeterli değildir?
- Header ile `.c` dosyasının sorumluluğu nedir?

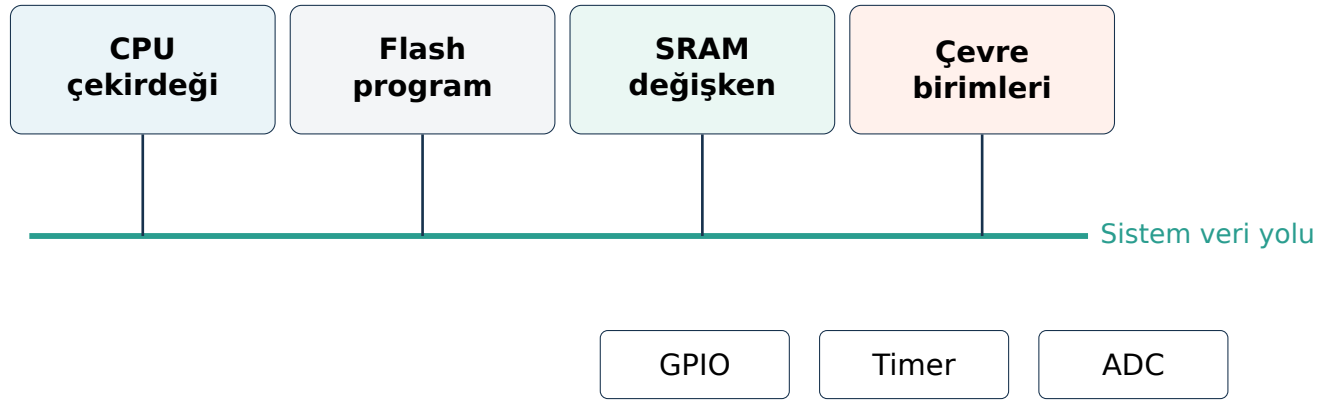
## **Kısım II**

# **Mikrodenetleyici ve Donanımla Konuşmak**

## İşlemci, bellek haritası ve register

### 11.1 Mikrodenetleyicinin ana parçaları

Bir STM32 veya ESP yalnızca CPU değildir. Aynı yonga içinde program belleği, RAM ve çevre birimleri bulunur.



Şekil 11.1: Mikrodenetleyicinin kavramsal iç yapısı

### 11.2 Memory-mapped I/O

Çevre birimi register'ları bellek adresi uzayına yerleştirilir. CPU açısından RAM okumaya benzeyen bir yükleme işlemi, timer sayacını okuyabilir; bir yazma işlemi GPIO pinini değiştirebilir.

| Adres bölgesi      | Kavramsal içerik      | Örnek kullanım        |
|--------------------|-----------------------|-----------------------|
| Flash bölgesi      | Program ve sabit veri | Komütasyon tablosu    |
| SRAM bölgesi       | Çalışma verisi        | Motor state, ölçümler |
| Peripheral bölgesi | Donanım register'ları | GPIO, TIM, ADC        |

Gerçek adresleri elle ezberlemek yerine üreticinin CMSIS header'ları kullanılır:

```

1 TIM1->CCR1 = duty_ticks;
2 GPIOA->BSRR = (1u << 5);
  
```

**TIM1->CCR1 nasıl okunur?**

TIM1, timer register yapısına işaretçidir. ->CCR1, bu yapının Capture/Compare Register 1 üyesini seçer. Atama, bu donanım register'ına değer yazar.

**11.3 Read-modify-write işlemi**

Bir register'ın yalnız belirli bitini değiştirmek için önce mevcut değer okunabilir:

```
1 register_value |= bit_mask; /* Bitleri 1 yap. */
2 register_value &= ~bit_mask; /* Bitleri 0 yap. */
3 register_value ^= bit_mask; /* Bitleri tersle. */
```

Fakat donanım register'larında bu yöntem her zaman güvenli değildir. Bazı bitler "1 yazarak temizle", bazıları yalnız okunur olabilir. Ayrıca ISR aynı anda register'ı değiştirirse güncelleme kaybolabilir. Referans kılavuzundaki register davranışı okunmalıdır.

**11.4 Maske oluşturmak**

```
1 #define BIT(n) (1u << (n))
2
3 uint32_t mask = BIT(0) | BIT(3);
```

İki bit alanına değer yerleştirmek:

```
1 #define MODE_Pos 4u
2 #define MODE_Msk (0x3u << MODE_Pos)
3
4 reg = (reg & ~MODE_Msk) | ((new_mode << MODE_Pos) & MODE_Msk);
```

Adımlar:

1. `reg & ~MODE_Msk`: Eski alanı temizler.
2. `new_mode << MODE_Pos`: Yeni değeri doğru bit konumuna kaydırır.
3. İkinci maske, komşu bitlere taşmayı engeller.
4. `|` yeni alanı register'a ekler.

## GPIO: dijital giriş ve çıkış

### 12.1 Bir pin neden doğrudan çalışmaz?

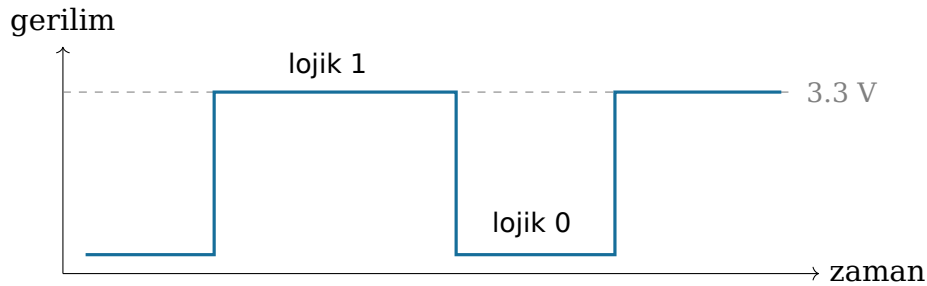
Mikrodenetleyici pinleri çok amaçlıdır. Bir pini kullanmadan önce genellikle şu ayarlar yapılır:

1. İlgili GPIO portunun clock'u açılır.
2. Pin modu input, output veya alternate function seçilir.
3. Çıkış tipi push-pull/open-drain belirlenir.
4. Hız ve pull-up/pull-down ayarlanır.
5. Gerekirse alternate function numarası seçilir.

#### Clock neden açılıyor?

Çevre birimleri kullanılmadığında clock'ları kapatılarak güç tüketimi azaltılır. Clock kapalıyken register yazısı etkisiz kalabilir. Başlangıçta "kodu yazdım ama pin değişmiyor" sorununun yaygın sebeplerinden biridir.

### 12.2 Mantıksal seviye fiziksel gerilimdir



Şekil 12.1: Bir dijital çıkışın gerilim karşılığı

MCU çıkışı gate driver'ın lojik girişine gidebilir; doğrudan MOSFET gate'ini sürmek bile çoğu güç uygulamasında uygun değildir. Gate'in hızlı şarj/deşarjı, high-side seviye yükseltme ve koruma için sürücü gerekir.

## 12.3 Atomik set/reset register'ı

STM32 ailesindeki birçok GPIO, BSRR benzeri set/reset register'ı sunar. Böylece read-modify-write yarışından kaçınılır:

```
1 GPIOA->BSRR = (1u << 5);          /* PA5 set. */  
2 GPIOA->BSRR = (1u << (5u+16u)); /* PA5 reset. */
```

Kesin register adı ve davranışı seçilen MCU'nun referans kılavuzundan doğrulanmalıdır.

## Timer, PWM ve dead-time

### 13.1 Timer nasıl sayar?

Timer giriş clock'unu prescaler ile böler, sayaç 0'dan ARR değerine kadar ilerler. Basitleştirilmiş güncelleme frekansı:

$$f_{update} = \frac{f_{timer}}{(PSC + 1)(ARR + 1)}$$

Örnek olarak timer clock'u 80 MHz, PSC =0 ve ARR =3999 ise:

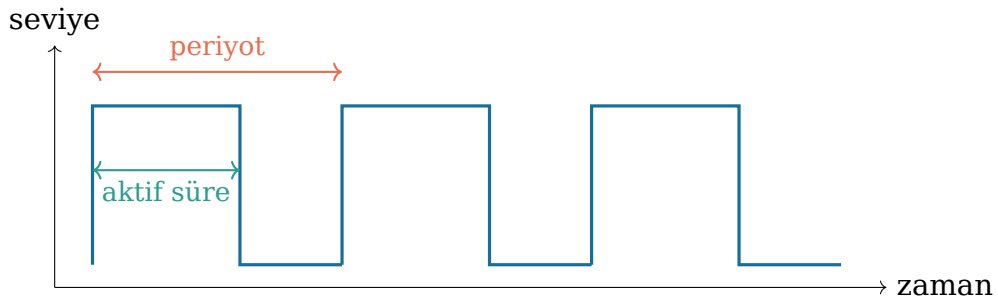
$$f_{PWM} = \frac{80\,000\,000}{1 \cdot 4000} = 20\,000 \text{ Hz}$$

Gerçek STM32 timer clock hesabında APB prescaler etkisi olabileceği için clock tree kontrol edilmelidir.

### 13.2 PWM duty nasıl oluşur?

Edge-aligned PWM'de sayaç CCR değerinden küçükken çıkış aktif seçilebilir:

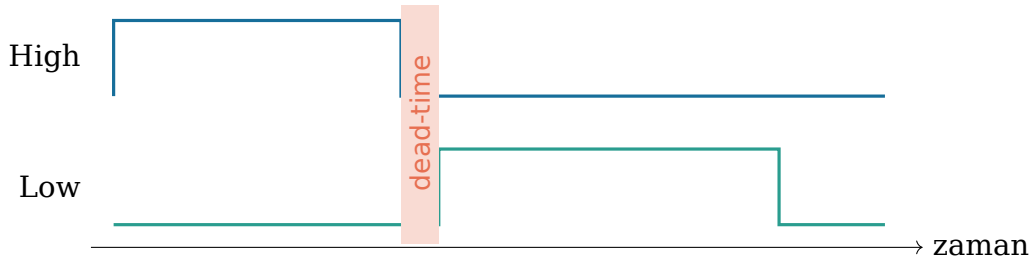
$$D = \frac{CCR}{ARR + 1}$$



Şekil 13.1: Yaklaşık yüzde 60 duty değerli edge-aligned PWM

### 13.3 Complementary PWM

Bir yarım köprüde high-side ve low-side çıkışlar tamamlayıcı olabilir. Fakat biri kapanır kapanmaz diğeri açmak tehlikelidir; MOSFET'in kapanma süresi sıfır değildir.



Şekil 13.2: İki anahtarın kapalı tutulduğu dead-time aralığı

### Shoot-through

Aynı fazın high-side ve low-side MOSFET'i birlikte açılırsa DC+ ile GND arasında çok düşük dirençli yol oluşur. Buna shoot-through denir. Yazılım maskesi hatalı deseni engelleyebilir, fakat geçiş anındaki gecikmeyi yalnız donanımsal dead-time ve sürücü interlock'u güvenilir biçimde yönetir.

## 13.4 Preload ve senkron güncelleme

Üç fazın compare değerlerini farklı anlarda etkinleştirmek kısa süreli yanlış anahtar deseni oluşturabilir. Gelişmiş timer'larda preload register'larına yazılır ve update/commutation olayıyla bütün yeni değerler aynı sınırdan etkinleştirilir.

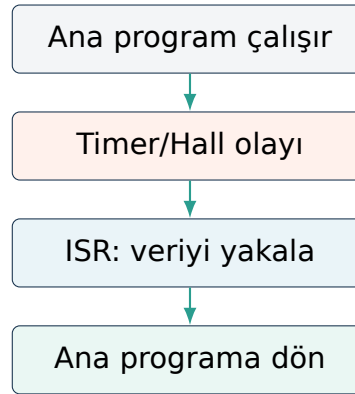
Motor kontrolünde timer seçerken aranan özellikler:

- Complementary çıkışlar
- Programlanabilir dead-time
- Break/fault girişi
- Preload ve senkron update
- Center-aligned PWM seçeneği
- ADC tetikleme olayı

## Kesme, ISR ve gerçek zaman

### 14.1 Polling ile kesme farkı

Polling'de ana döngü sürekli bir bayrağı sorar. Kesmede donanım olay olduğunda CPU'yu ISR'a yönlendirir.



### 14.2 İyi ISR nasıl olmalı?

ISR mümkün olduğunca kısa ve süre olarak öngörülebilir olmalıdır:

```

1 void HALL_IRQHandler(void)
2 {
3     uint8_t hall = read_hall_pins();
4     uint32_t capture = read_capture_timer();
5
6     hall_mailbox.code = hall;
7     hall_mailbox.capture = capture;
8     hall_mailbox.pending = 1u;
9
10    clear_interrupt_flag();
11 }
  
```

ISR içinde kaçınılması gerekenler:

- Uzun bloklayan döngü veya gecikme
- Dinamik bellek ayırma
- Yavaş printf çağrısı
- Sonlanma süresi belirsiz haberleşme
- Gereksiz ağır kayan nokta hesabı

### 14.3 Yarış durumu örneği

Ana döngü capture değerinin ilk yarısını okurken ISR yeni değer yazarsa tutarsız veri oluşabilir. 32 bit Cortex-M üzerinde hizalı 32 bit tek erişim genellikle atomik olsa bile birden fazla alanın birlikte tutarlı olması garanti değildir.

Basit kritik bölüm:

```
1 disable_interrupts();
2 local_code = mailbox.code;
3 local_capture = mailbox.capture;
4 mailbox.pending = 0u;
5 enable_interrupts();
```

Kritik bölüm çok kısa tutulmalıdır; aksi halde başka önemli kesmeler gecikir.

### 14.4 Öncelik ve gecikme

Motor fault kesmesi, kullanıcı butonu kesmesinden daha kritik olabilir. Ancak yalnızca öncelik numarasını değiştirmek yeterli değildir; seçilen MCU'da öncelik bitlerinin ve RTOS kurallarının nasıl çalıştığı doğrulanmalıdır.

## ADC, akım ölçümü ve hata koruması

### 15.1 ADC'nin görevi

ADC analog gerilimi sayısal değere çevirir. 12 bit ADC ve 3.3 V referans için ideal yaklaşım:

$$V_{in} = \frac{ADC_{raw}}{4095} V_{ref}$$

Akım şönt üzerinden ölçülüyorsa:

$$I = \frac{V_{shunt}}{R_{shunt} G}$$

Burada  $G$ , akım yükseltecinin kazancıdır.

### 15.2 PWM ile senkron örnekleme

Anahtarlama kenarları elektriksel gürültü üretir. ADC örneğini PWM periyodunun sakın bir noktasında timer ile tetiklemek daha güvenilir sonuç verir. Rastgele anda yazılım çağrısıyla okumak, komütasyon gürültüsünü ölçüme karıştırabilir.

### 15.3 Yazılım ve donanım koruması birlikte

| Koruma          | Donanım yolu                           | Yazılım yolu                       |
|-----------------|----------------------------------------|------------------------------------|
| Aşırı akım      | Comparator → gate driver / timer break | ADC ölçümü, fault kaydı            |
| Düşük gerilim   | Sürücü UVLO                            | ADC ile DC bus izleme              |
| Aşırı sıcaklık  | Driver shutdown / sensör               | Sıcaklık eşiği ve azaltma          |
| Geçersiz sektör | Gate interlock                         | Hall/BEMF doğrulama                |
| Watchdog        | Bağımsız watchdog reset                | Periyodik sağlıklı çalışma işareti |

#### Dikkat

Yazılımın ADC okuyup motoru kapatmasını tek aşırı akım koruması yapmak yavaştır. Şiddetli kısa devrede donanımsal comparator ve gate shutdown yolu, CPU'dan bağımsız ve çok daha hızlı olmalıdır.

## Hata ayıklama ve ölçüm araçları

### 16.1 Derleyici uyarıları ilk savunmadır

PC testlerinde örnek bayraklar:

```
1 gcc -std=c11 -Wall -Wextra -Wpedantic \
2   main.c motor_control.c motor_hw.c -o motor_test
```

Gömülü projede de IDE'nin uyarı seviyesini yüksek tut. Uyarıyı susturmak yerine sebebini anlamaya çalış.

### 16.2 Debugger ile bakılacaklar

- Breakpoint: Programı seçilen satırda durdurur.
- Step over: Fonksiyonu tek adım olarak yürütür.
- Step into: Fonksiyonun içine girer.
- Watch: Değişkeni sürekli izler.
- Peripheral view: Register'ları isimleriyle gösterir.
- Call stack: Fonksiyon çağrı zincirini gösterir.

### 16.3 Osiloskop ve logic analyzer

Motor projesinde yazılım debugger'ı tek başına yeterli değildir. PWM frekansı, complementary çıkış, dead-time ve sektör sırası osiloskopa doğrulanır. Hall veya dijital sürücü sinyalleri logic analyzer ile daha uzun süre kaydedilebilir.

#### Ölçüm toprağı

Osiloskop probunun toprak klipsi birçok masaüstü osiloskopta koruyucu toprağa bağlıdır. Yanlış noktaya bağlamak kısa devre oluşturabilir. Güç katmanı ölçümünde diferansiyel prob, izolasyon ve kurum laboratuvar prosedürü gerekir.

#### Donanım kısmı kontrol listesi

- Register'ın bellek adresinden erişilen donanım durumu olduğunu açıklayabiliyorum.
- Bir biti set, clear ve mask işlemleriyle değiştirebiliyorum.

- Timer clock, ARR, PSC, CCR ve duty ilişkisini biliyorum.
- Complementary PWM ile dead-time'ın neden farklı kavramlar olduğunu biliyorum.
- ISR'ın kısa tutulması ve paylaşılan verinin korunması gerektiğini biliyorum.
- Aşırı akım korumasında donanım yolunun zorunlu olduğunu biliyorum.

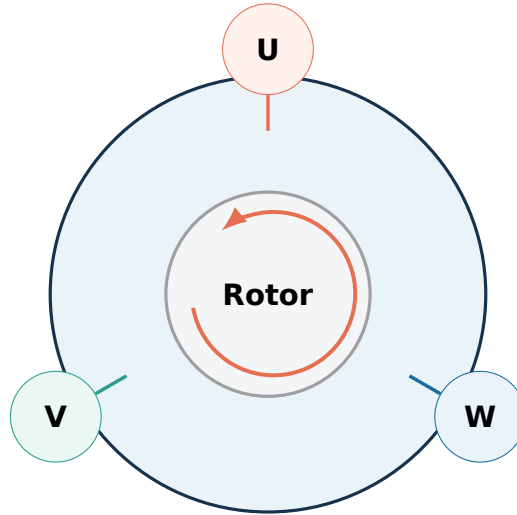
## **Kısım III**

# **BLDC Motor Kontrolüne Geçiş**

## BLDC motorun temel çalışma mantığı

### 17.1 Stator, rotor ve üç faz

BLDC motorun statorunda U, V ve W olarak adlandırılan üç faz sargısı bulunur. Rotor kalıcı mıknatıs taşır. Sargılardaki akımı doğru sırada değiştirerek dönen manyetik alan oluştururuz; rotor bu alanı takip eder.



Şekil 17.1: Üç faz ve dönen mıknatıslı kısmın kavramsal görünümü

### 17.2 Elektriksel ve mekanik açı

Bir kutup çiftli motorda bir mekanik tur bir elektriksel tura karşılık gelir.  $p$  kutup çiftli motorda:

$$\theta_e = p \theta_m$$

Altı adımlı komütasyonda her sektör 60 elektriksel derecedir. 7 kutup çiftinde bir mekanik turda  $6 \times 7 = 42$  sektör olayı görülür.

#### Neden?

“Motorun 60 derecelik aralıkları” ifadesi çoğunlukla milin her 60 mekanik derecesi anlamına gelmez. Komütasyon algoritmasındaki 60 derece elektriksel açıdır; kutup çifti sayısı mekanik karşılığını değiştirir.

### 17.3 Torku ne belirler?

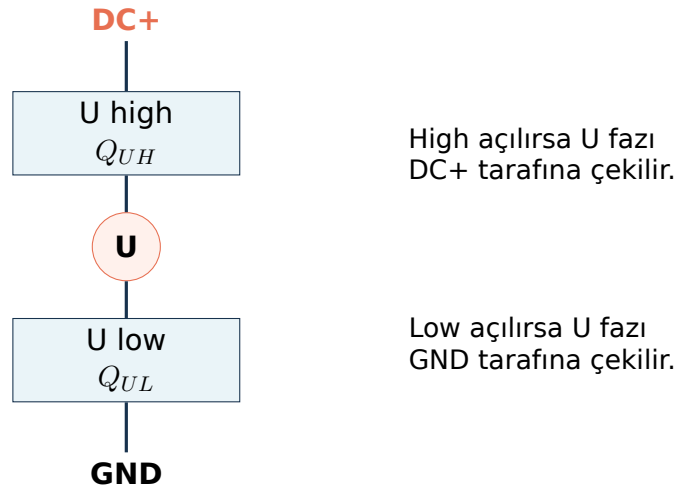
Basitleştirilmiş olarak motor torku faz akımıyla ilişkilidir. PWM duty artırıldığında uygulanan ortalama gerilim ve koşullara bağlı olarak akım artabilir. Ancak duty doğrudan tork değildir; sargı direnci, endüktans, hızla oluşan back-EMF, besleme gerilimi ve yük de etkilidir.

Bu nedenle “duty yüzde 50 ise hız kesin yüzde 50” veya “yüksek duty her zaman güvenlidir” denemez.

## Üç faz inverter ve altı MOSFET

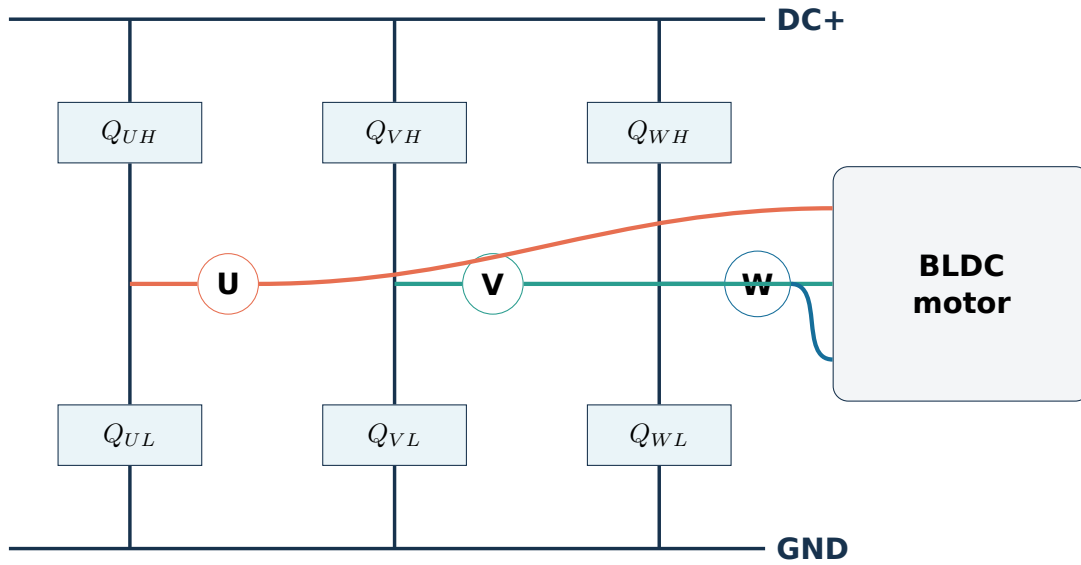
### 18.1 Bir yarım köprü

Her motor fazı bir high-side ve bir low-side MOSFET arasındaki ayrı orta noktaya bağlanır.



Şekil 18.1: U fazına ait tek yarım köprü

## 18.2 Üç yarım köprü bir arada



Şekil 18.2: U, V ve W orta noktaları birbirinden ayrı olan üç faz inverter

### Şemada kritik ayrıntı

U, V ve W inverter çıkışları aynı kablo değildir ve birbirine kısa devre edilmez. Her biri kendi yarım köprüsünün orta noktasıdır ve motorun ayrı faz terminaline gider.

## 18.3 Üç fazın üç durumu

Bir komütasyon anında faz için kavramsal durumlar:

- **SOURCE:** High-side ile DC+ tarafına enerji verir.
- **SINK:** Low-side ile akımı GND tarafına taşır.
- **FLOAT:** İki anahtar da kapalıdır; sensorless yöntemde BEMF ölçülebilir.

Gerçek PWM stratejisinde “SOURCE” anahtarı sürekli açık olmak zorunda değildir; ilgili aktif anahtar PWM’lenebilir. Serbest dolaşım akımının yolu kullanılan synchronous/asynchronous rectification stratejisine göre değişir.

## Altı adımlı komütasyon

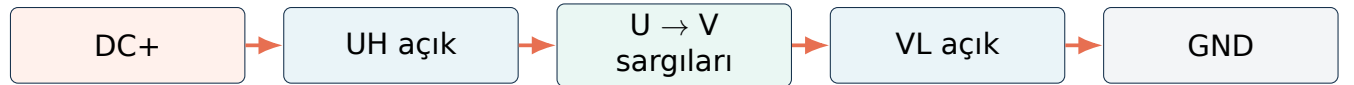
### 19.1 Neden altı adım?

Her adımda bir faz DC+ tarafında, bir faz GND tarafında, üçüncü faz boşta tutulur. Altı farklı yönlü faz çifti, elektriksel turu altı sektöre böler.

| Adım | Kaynak | Sink | Boşta | Anahtarlar | Maske |
|------|--------|------|-------|------------|-------|
| 1    | U      | V    | W     | UH + VL    | 0x09  |
| 2    | U      | W    | V     | UH + WL    | 0x21  |
| 3    | V      | W    | U     | VH + WL    | 0x24  |
| 4    | V      | U    | W     | VH + UL    | 0x06  |
| 5    | W      | U    | V     | WH + UL    | 0x12  |
| 6    | W      | V    | U     | WH + VL    | 0x18  |

### 19.2 Birinci adımı fiziksel okuyalım

Adım 1’de UH ve VL etkinleştirilir:



W fazı floating

Bit maskesi:

```

1 GATE_UH = 1u << 0; /* 000001 */
2 GATE_VL = 1u << 3; /* 001000 */
3
4 gate_mask = GATE_UH | GATE_VL; /* 001001 = 0x09 */
  
```

### 19.3 Yön nasıl ters çevrilir?

Komütasyon adımları ters sırada uygulanırsa dönen alanın yönü tersine döner. Ancak çalışan motorun yönünü aniden terslemek büyük akım ve mekanik darbe oluşturabilir. Önce kontrollü yavaşlatma veya durdurma gerekir.

## 19.4 Komütasyon zamanını kim belirler?

Üç temel yaklaşım vardır:

| Yöntem          | Bilgi kaynağı                            | Temel sorun                                  |
|-----------------|------------------------------------------|----------------------------------------------|
| Open-loop       | Yazılım belirli zaman aralığında ilerler | Yük altında rotor alanı takip etmeyebilir    |
| Hall sensörlü   | Üç dijital Hall kodu                     | Motor/kablo eşlemesi kalibre edilmelidir     |
| Sensorless BEMF | Floating faz gerilimi                    | Düşük hızda BEMF zayıftır; başlangıç gerekir |

## Hall ve sensorless BEMF

### 20.1 IMU neden gerekli değildir?

IMU, doğrusal ivme ve açısal hız ölçer; klasik BLDC komütasyonunda gerekli sensör değildir. Rotorun elektriksel konumu Hall sensörü, enkoder veya fazlardan oluşan back-EMF ile tahmin edilir.

### 20.2 Hall kodları

Üç Hall hattı sekiz ikili kombinasyon üretebilir. Tipik olarak altı geçerli durum vardır; 000 ve 111 hata kabul edilir. Örnek ileri sıra:

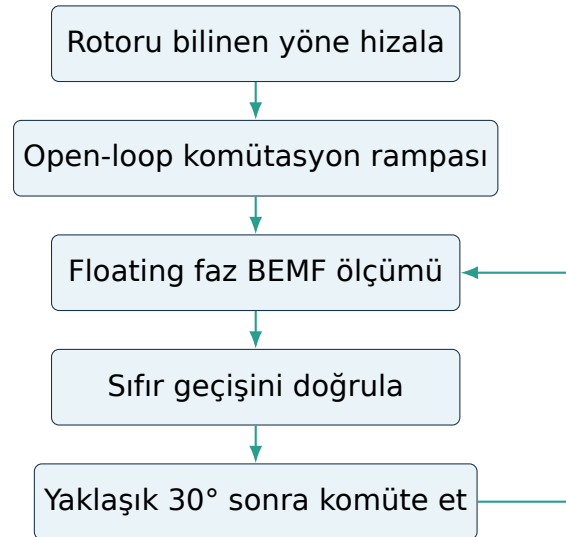
001 → 101 → 100 → 110 → 010 → 011

```
1 static const int8_t hall_to_step[8] =  
2 {  
3     -1, /* 000 */  
4     0, /* 001 */  
5     4, /* 010 */  
6     5, /* 011 */  
7     2, /* 100 */  
8     1, /* 101 */  
9     3, /* 110 */  
10    -1 /* 111 */  
11 };
```

Bu sıra evrensel değildir; Hall montajı ve U-V-W kablo sırası değişirse tablo değişir.

### 20.3 Sadece üç faz kablosu varsa

Motorundan yalnız U, V ve W çıkıyorsa dahili Hall bağlantısı yoktur. Sensorless altı adımın temel akışı:



Trapezoidal BLDC’de floating fazın BEMF’i sanal nötr/yarım DC bus seviyesini geçtiğinde yaklaşık sektör ortası, yani bir sonraki komütasyondan yaklaşık 30 elektriksel derece önce kabul edilebilir. Uygulamada blanking süresi, filtreleme, comparator/ADC gecikmesi ve hızla değişen zamanlama hesaba katılır.

### Sensorless başlangıç

Motor dururken BEMF yoktur. Bu nedenle sıfır hızda yalnız BEMF okuyarak rotor konumu bulunamaz. Hizalama ve kontrollü open-loop hızlanma gerekir. Faz sırası veya rampanın eğimi yanlışsa motor yalnız titreyebilir ya da aşırı akım çekebilir.

## 20.4 60° olayından hız hesabı

Bir mekanik turda  $6p$  sektör olayı vardır. Timer frekansı  $f_t$ , iki olay arasındaki tick sayısı  $N$  ise:

$$RPM = \frac{60f_t}{6pN}$$

Örneğin 1 MHz timer, 7 kutup çifti ve 2000 tick sektör süresinde:

$$RPM = \frac{60 \times 1\,000\,000}{6 \times 7 \times 2000} \approx 714$$

## Sürücü kartını tanımadan kod yazma

“Motor sürücü” sözü birkaç farklı kartı ifade edebilir. İlk iş kart üzerindeki entegre ve pin isimlerini belirlemektir.

| Kart tipi                   | MCU ne gönderir?                        | Komütasyonu kim yapar?   |
|-----------------------------|-----------------------------------------|--------------------------|
| Hazır ESC                   | Servo PWM, DShot veya üretici protokolü | ESC’nin kendi işlemcisi  |
| Entegre komütasyon sürücüsü | Hız/yön/enable veya SPI komutları       | Sürücü entegresi         |
| 3-PWM inverter sürücüsü     | Üç PWM ve kontrol pinleri               | MCU + sürücü mantığı     |
| 6-PWM gate driver           | UH/UL, VH/VL, WH/WL komutları           | Büyük ölçüde MCU timer’ı |
| Yalnız gate driver          | Gate sürme ve koruma; MOSFET’ler harici | MCU + harici güç katı    |

### 21.1 Veri sayfasında aranacak bilgiler

- Lojik giriş gerilimi STM32/ESP ile uyumlu mu?
- Giriş arayüzü 1-PWM, 3-PWM veya 6-PWM mi?
- Aktif-high/aktif-low girişler hangileri?
- Dahili dead-time ve shoot-through önleme var mı?
- Fault çıkışı open-drain mi, pull-up gerekiyor mu?
- Enable sırası ve bootstrap koşulları nedir?
- Akım ölçüm çıkışının kazancı ve ofseti nedir?
- Maksimum motor/bus gerilimi ve akımı nedir?

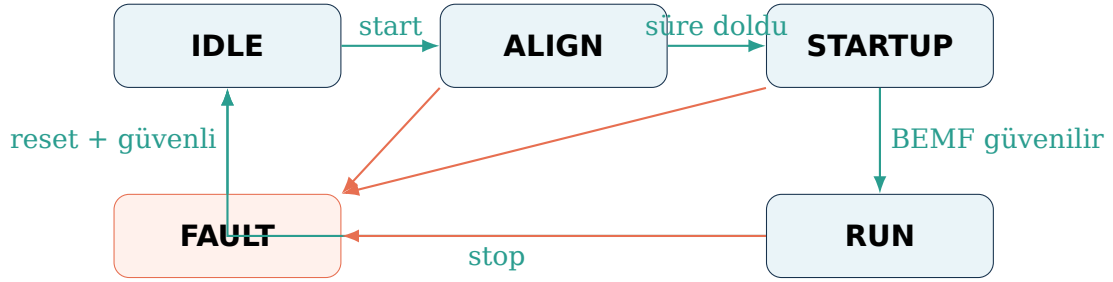
#### Dikkat

Kartın yalnız pin sayısına bakarak bağlantı yapılmaz. Aynı isimli girişler farklı sürücülerde farklı truth table’a sahip olabilir. Önce veri sayfası, ardından enerjisiz continuity kontrolü ve düşük gerilimli sınırlı akım testi yapılır.

## Motor yazılımı için durum makinesi

### 22.1 Neden tek bir while döngüsü yetmez?

Motor bir anda “kapalı” durumdan normal sensorless çalışmaya geçemez. Başlatma, hizalama, rampalama ve hata gibi farklı davranışlar gerekir. Durum makinesi bu davranışları açıkça ayırır.



Şekil 22.1: Sensorless motor için örnek durum makinesi

### 22.2 Her durumun sorumluluğu

**IDLE** Bütün gate’ler güvenli kapalıdır; start komutu beklenir.

**ALIGN** Düşük duty ile sabit vektör uygulanır, rotor bilinen konuma çekilir.

**STARTUP** Önceden belirlenen zaman rampasıyla open-loop altı adım ilerletilir.

**RUN** Hall/BEMF olayına göre kapalı çevrim komütasyon ve hız kontrolü yapılır.

**FAULT** Gate’ler kapatılır, hata sebebi saklanır; koşulsuz otomatik yeniden başlatılmaz.

### 22.3 Durum geçiş koşulları

Bir durum geçişi yalnız zaman dolmasına bağlı olmamalıdır. Örnek RUN geçişi için:

- Minimum hız aşılmış mı?
- Art arda yeterli sayıda geçerli BEMF olayı görüldü mü?
- Akım ve bus gerilimi güvenli mi?
- Komütasyon sırası beklenen yönde mi?

**Neden?**

Durum makinesi, motorun “bazen çalışıyor bazen titriyor” davranışını gözlenebilir hale getirir. Debugger veya UART logunda hangi durumda kaldığını görerek sorunun hizalama, rampalama veya BEMF algılama tarafında olduğunu ayırabilirsin.

## **Kısım IV**

# **Basitten Zora Kodlama Uygulamaları**

## Seviye 1: PC’de gate maskesi simülasyonu

Gerçek motorla başlamadan önce bit desenlerini PC’de üretmek en güvenli adımdır. Aşağıdaki program donanım sürmez; yalnız maskeyi hesaplar ve tehlikeli aynı-bacak desenini reddeder.

Listing 23.1: Tek dosyalık ilk simülasyon

```

1  #include <stdint.h>
2  #include <stdio.h>
3
4  enum
5  {
6      GATE_UH = 1u << 0,
7      GATE_UL = 1u << 1,
8      GATE_VH = 1u << 2,
9      GATE_VL = 1u << 3,
10     GATE_WH = 1u << 4,
11     GATE_WL = 1u << 5
12 };
13
14 static int has_shoot_through(uint32_t mask)
15 {
16     int u_bad = ((mask & GATE_UH) != 0u) &&
17                ((mask & GATE_UL) != 0u);
18
19     int v_bad = ((mask & GATE_VH) != 0u) &&
20                ((mask & GATE_VL) != 0u);
21
22     int w_bad = ((mask & GATE_WH) != 0u) &&
23                ((mask & GATE_WL) != 0u);
24
25     return u_bad || v_bad || w_bad;
26 }
27
28 int main(void)
29 {
30     uint32_t step_1 = GATE_UH | GATE_VL;
31
32     if (has_shoot_through(step_1) != 0)
33     {
34         printf("HATALI DESEN\n");
35         return 1;
36     }
37
38     printf("Adim 1 maskesi: 0x%02X\n",
39           (unsigned)step_1);

```

```
40     return 0;  
41 }
```

### 23.1 Satırların mantığı

1. Header'lar tip ve printf bildirimlerini getirir.
2. İsimsiz enum, gate bit sabitlerini gruplaştırır.
3. static yardımcı fonksiyonu dosyaya özel yapar.
4. mask & GATE\_UH, UH bitinin maskede bulunup bulunmadığını ayırır.
5. != 0u, sonucu açık biçimde doğru/yanlış değerine çevirir.
6. &&, aynı bacağın iki anahtarının birlikte seçilip seçilmediğini sorar.
7. ||, üç fazdan herhangi birinde hata bulunmasını yeterli sayar.
8. GATE\_UH | GATE\_VL, iki ayrı biti tek maskede birleştirir.

### 23.2 Kendin dene

```
1 uint32_t bad = GATE_UH | GATE_UL;
```

Bu deseni kontrol fonksiyonuna gönderdiğinde hata üretmelidir. Böylece gerçek MOSFET olmadan koruma mantığını sınarsın.

## Seviye 2: Komütasyon tablosu

### 24.1 Faz durumlarını tanımlamak

```

1 typedef enum
2 {
3     PHASE_FLOAT,
4     PHASE_SOURCE_HIGH,
5     PHASE_SINK_LOW
6 } PhaseMode;
7
8 typedef struct
9 {
10     PhaseMode u;
11     PhaseMode v;
12     PhaseMode w;
13 } CommutationStep;

```

Bu yapı sayesinde 0x09 gibi bir sayının anlamını akılda tutmak zorunda kalmayız. Önce fiziksel durumu ifade eder, ardından bit maskesine çeviririz.

### 24.2 Altı satırlık değişmez tablo

```

1 static const CommutationStep forward_table[6] =
2 {
3     {PHASE_SOURCE_HIGH, PHASE_SINK_LOW,    PHASE_FLOAT},
4     {PHASE_SOURCE_HIGH, PHASE_FLOAT,      PHASE_SINK_LOW},
5     {PHASE_FLOAT,       PHASE_SOURCE_HIGH, PHASE_SINK_LOW},
6     {PHASE_SINK_LOW,    PHASE_SOURCE_HIGH, PHASE_FLOAT},
7     {PHASE_SINK_LOW,    PHASE_FLOAT,      PHASE_SOURCE_HIGH},
8     {PHASE_FLOAT,       PHASE_SINK_LOW,    PHASE_SOURCE_HIGH}
9 };

```

### 24.3 Bir faz durumunu gate bitine çevirmek

```

1 static uint32_t phase_to_gate_bits(
2     PhaseMode mode,
3     uint32_t high_bit,
4     uint32_t low_bit)
5 {
6     switch (mode)
7     {

```

```

8      case PHASE_SOURCE_HIGH:
9          return high_bit;
10
11     case PHASE_SINK_LOW:
12         return low_bit;
13
14     case PHASE_FLOAT:
15     default:
16         return 0u;
17 }
18 }

```

Fonksiyon U, V veya W'yi özel olarak bilmez. Çağırın kod o fazın high ve low bitlerini parametre olarak verir. Bu, aynı mantığın üç kez kopyalanmasını önler.

```

1 static uint32_t step_to_gate_mask(
2     const CommutationStep *step)
3 {
4     uint32_t mask = 0u;
5
6     mask |= phase_to_gate_bits(
7         step->u, GATE_UH, GATE_UL);
8
9     mask |= phase_to_gate_bits(
10        step->v, GATE_VH, GATE_VL);
11
12    mask |= phase_to_gate_bits(
13        step->w, GATE_WH, GATE_WL);
14
15    return mask;
16 }

```

### Sözdizimi

`const CommutationStep *step`, fonksiyonun yapıyı okuyacağını fakat bu işaretçi üzerinden değiştirmeyeceğini belirtir. `step->u`, işaret edilen yapının U üyesidir. `mask | = ...`, yeni gate bitlerini mevcut maskeye ekler.

## Seviye 3: Fonksiyon işaretçisiyle donanım soyutlama

### 25.1 Problem

PC testinde gate yazmak ekrana sayı basmalıdır; STM32'de timer register'larını güncellemelidir. Kontrol algoritmasına iki ayrı sürüm yazmak istemeyiz.

### 25.2 Fonksiyon işaretçisi tipi

```
1 typedef void (*MotorWriteGatesFn)(uint32_t gate_mask);
2 typedef uint32_t (*MotorReadTicksFn)(void);
3
4 typedef struct
5 {
6     MotorWriteGatesFn write_gates;
7     MotorReadTicksFn read_ticks;
8 } MotorHwOps;
```

İlk typedef sağdan sola şöyle okunabilir:

MotorWriteGatesFn; bir uint32\_t alan ve değer döndürmeyen fonksiyonu gösterebilen işaretçi tipidir.

### 25.3 PC uygulaması

```
1 static volatile uint32_t simulated_gate_register;
2
3 static void simulated_write_gates(uint32_t gate_mask)
4 {
5     simulated_gate_register = gate_mask;
6     printf("Gate register = 0x%02X\n",
7         (unsigned)simulated_gate_register);
8 }
9
10 static uint32_t simulated_read_ticks(void)
11 {
12     return simulated_tick_register;
13 }
14
15 static const MotorHwOps simulated_ops =
16 {
```

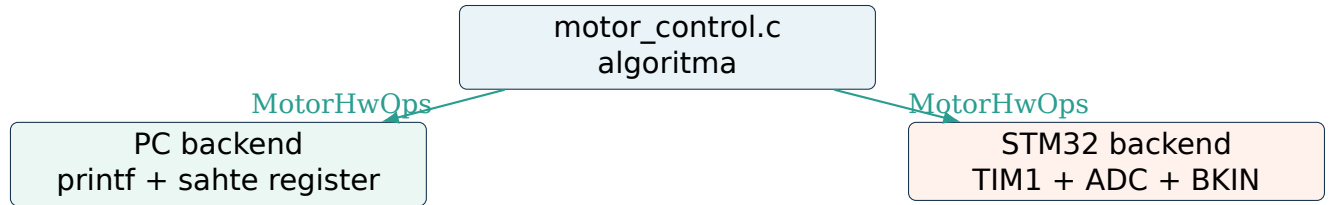
```
17     .write_gates = simulated_write_gates,  
18     .read_ticks = simulated_read_ticks  
19 };
```

## 25.4 Kontrol katmanının çağırısı

```
1 motor->hw->write_gates(gate_mask);
```

Bu satırda iki ok operatörü vardır:

1. `motor->hw`: Motor yapısındaki donanım arayüzü işaretçisini bulur.
2. `->write_gates`: Arayüz yapısındaki fonksiyon işaretçisini bulur.
3. `(gate_mask)`: İşaret edilen fonksiyonu çağırır.



## Seviye 4: ISR posta kutusu ve olay işleme

### 26.1 Paylaşılan veri yapısı

```

1 typedef struct
2 {
3     volatile uint8_t hall_code;
4     volatile uint8_t event_pending;
5     volatile uint32_t capture_ticks;
6 } HallIsrMailbox;
7
8 static HallIsrMailbox hall_mailbox;

```

ISR yalnız veriyi yakalar:

```

1 void motor_control_hall_isr(
2     uint8_t hall_code,
3     uint32_t capture_ticks)
4 {
5     hall_mailbox.hall_code = hall_code & 0x07u;
6     hall_mailbox.capture_ticks = capture_ticks;
7     hall_mailbox.event_pending = 1u;
8 }

```

Ana döngü ağır işi yapar:

```

1 void motor_control_process(MotorControl *motor)
2 {
3     if ((motor == NULL) ||
4         (motor->state != MOTOR_RUN))
5     {
6         return;
7     }
8
9     if (hall_mailbox.event_pending == 0u)
10    {
11        return;
12    }
13
14    /* Gerçek MCU'da ıksa kritik bölüm gerekir. */
15    uint8_t hall = hall_mailbox.hall_code;
16    uint32_t capture = hall_mailbox.capture_ticks;
17    hall_mailbox.event_pending = 0u;
18
19    int8_t step_index = hall_to_step[hall];
20
21    if (step_index < 0)

```

```
22 {  
23     enter_fault(motor, FAULT_INVALID_HALL);  
24     return;  
25 }  
26  
27 update_speed(motor, capture);  
28 apply_step(motor, (uint8_t)step_index);  
29 }
```

## 26.2 Erken dönüşler neden kullanıldı?

Geçersiz koşulları fonksiyon başında eleyince ana başarılı akış daha az iç içe girer. Sıra bilinçlidir:

1. İşaretçi geçerli mi?
2. Motor RUN durumunda mı?
3. Yeni olay var mı?
4. Hall kodu geçerli adım üretiyor mu?
5. Sonra ölçüm ve komütasyon yapılır.

## 26.3 Bir event bayrağının sınırı

Tek event\_pending bayrağı, ana döngü yavaşsa iki hızlı olayı tek olay gibi görebilir. Üretim kodunda olay kaybı kabul edilmiyorsa:

- monoton artan event counter,
  - ring buffer,
  - timer capture FIFO/DMA,
  - RTOS queue
- gibi mekanizmalar değerlendirilir.

## Seviye 5: Hız hesabı ve hata yolu

### 27.1 İlk ölçüm neden ayrı ele alınır?

Hız için iki zaman damgası gerekir. Sağlam yaklaşım ayrı bir geçerlilik bayrağı kullanmaktır:

```

1 static void update_speed(
2     MotorControl *motor,
3     uint32_t capture_ticks)
4 {
5     if (motor->has_previous_capture == 0u)
6     {
7         motor->last_capture_ticks = capture_ticks;
8         motor->has_previous_capture = 1u;
9         return;
10    }
11
12    uint32_t sector_ticks =
13        capture_ticks - motor->last_capture_ticks;
14
15    motor->last_capture_ticks = capture_ticks;
16
17    if ((sector_ticks == 0u) ||
18        (motor->pole_pairs == 0u))
19    {
20        return;
21    }
22
23    uint64_t numerator =
24        60ull * motor->timer_frequency_hz;
25
26    uint64_t denominator =
27        6ull * motor->pole_pairs * sector_ticks;
28
29    motor->measured_rpm =
30        (uint32_t)(numerator / denominator);
31 }
```

#### Neden?

last\_capture\_ticks == 0 değerini “ilk ölçüm yok” işareti yapmak zayıftır; gerçek timer değeri meşru biçimde 0 olabilir. Ayrı has\_previous\_capture bayrağı durumu açıkça temsil eder.

## 27.2 Merkezi hata fonksiyonu

```
1 static void enter_fault(  
2     MotorControl *motor,  
3     MotorFault fault)  
4 {  
5     motor->fault = fault;  
6     motor->state = MOTOR_FAULT;  
7     motor->hw->write_gates(0u);  
8 }
```

Her hata dalının gate kapatmayı unutması yerine güvenli davranış tek fonksiyonda toplanır. Gerçek projede buna hata zaman damgası, ölçülen akım, bus gerilimi ve son sektör de eklenebilir.

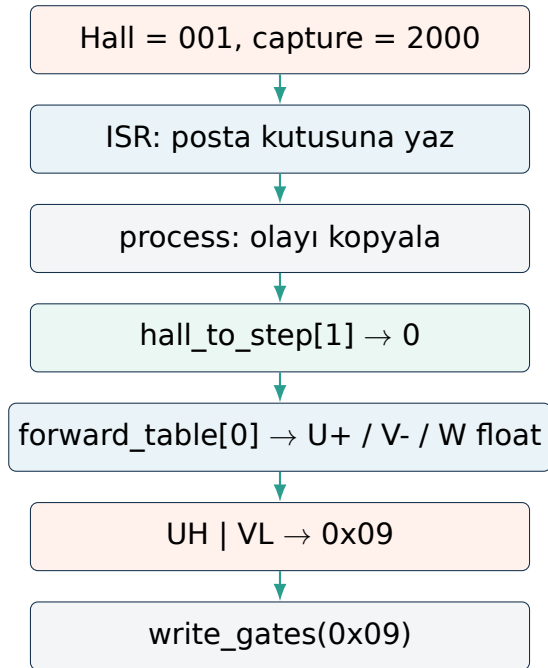
## Tam programın çalışma sırası

### 28.1 main fonksiyonunun görevi

```
1 int main(void)
2 {
3     MotorControl motor;
4
5     motor_hw_init();
6     motor_control_init(
7         &motor,
8         motor_hw_get_ops(),
9         1000000u,
10        7u);
11
12     motor_control_start(&motor);
13
14     while (1)
15     {
16         motor_control_process(&motor);
17     }
18 }
```

1. Stack üzerinde MotorControl motor nesnesi oluşturulur.
2. GPIO, timer, ADC ve fault donanımı hazırlanır.
3. &motor ile aynı nesnenin adresi init fonksiyonuna verilir.
4. Timer frekansı ve kutup çifti sayısı kontrol bağlamına kaydedilir.
5. Start isteği state machine'i başlatır.
6. Sonsuz döngü bekleyen olayları ve durum geçişlerini işler.

## 28.2 Tek Hall olayının yolculuğu



## 28.3 Kodun kavram haritası

| C kavramı            | Kod örneği              | Projede çözdüğü sorun                             |
|----------------------|-------------------------|---------------------------------------------------|
| İşaretçi             | MotorControl *motor     | Aynı kontrol nesnesini fonksiyonlarda değiştirmek |
| volatile             | ISR posta kutusu        | Kesme/donanım kaynaklı değişikliği görünür kılmak |
| static               | static void enter_fault | Modül içi ayrıntıyı gizlemek                      |
| const                | Komütasyon tablosu      | Kazara değişikliği önlemek                        |
| struct               | MotorControl            | İlişkili motor durumunu toplamak                  |
| enum                 | MotorState              | Sayılarla anlamlı isim vermek                     |
| Bit işlemi           | GATE_UH   GATE_VL       | Bir register/maskede çok anahtarı temsil etmek    |
| Fonksiyon işaretçisi | MotorHwOps              | Algoritmayı MCU'dan ayırmak                       |
| Header               | motor_control.h         | Public arayüzü bildirmek                          |

## STM32'ye taşıma rehberi

### 29.1 Hangi kod aynı kalır?

Genellikle şu kısımlar MCU'dan bağımsız tutulabilir:

- State machine ve fault mantığı
- Hall/BEMF sektör eşlemesi
- Komütasyon tablosu
- Hız hesabı
- Duty sınırlandırma ve üst seviye komutlar

Şu kısımlar hedefe özeldir:

- Clock tree ve pin alternate function ayarı
- TIM1/TIM8 register/HAL/LL çağrıları
- Complementary PWM, dead-time ve break konfigürasyonu
- ADC channel, trigger ve DMA ayarı
- Hall input capture veya comparator/EXTI ISR'ı

### 29.2 Kavramsal STM32 backend

```

1 static void stm32_write_gates(uint32_t mask)
2 {
3     /* Önce yeni compare ve channel-enable değerlerini
4        preload alanlarına hazırla. */
5
6     prepare_u_leg(mask);
7     prepare_v_leg(mask);
8     prepare_w_leg(mask);
9
10    /* Uygun timer ıolaynda hepsini birlikte şetkinletir. */
11    trigger_commutation_event();
12 }

```

Bu fonksiyonun içinde sıradan altı GPIO'yu art arda açıp kapatmak uygun değildir. Geçişlerin timer ile senkron, dead-time'lı ve break korumalı yapılması gerekir.

### 29.3 HAL, LL veya register?

| Yaklaşım          | Avantaj                           | Dezavantaj                                                  |
|-------------------|-----------------------------------|-------------------------------------------------------------|
| HAL               | Hızlı başlangıç, okunabilir API   | Ek yük ve bazı gelişmiş zamanlama ayrıntılarında sınırlılık |
| LL                | Register'a yakın, üretici desteği | HAL'dan daha fazla donanım bilgisi gerekir                  |
| Doğrudan register | Tam kontrol ve düşük ek yük       | Taşınabilirlik düşük, hata riski ve doküman ihtiyacı yüksek |

Başlangıç için CubeMX/HAL ile clock, GPIO ve temel timer kurulabilir; kritik komütasyon bölümünde LL veya kontrollü register erişimi gerekebilir. Kurumun mevcut sürücü mimarisi ve kod standardı önceliklidir.

### 29.4 ESP32 farkı

ESP32 ailesinde motor kontrolü için MCPWM benzeri çevre birimleri bulunabilir; kesin modül hedef modele göre değişir. Üst seviye algoritma aynı kalırken timer, dead-time, fault ve ADC sürücüleri ESP-IDF arayüzlerine taşınır.

## Laboratuvar bring-up planı

Motoru ilk denemede doğrudan besleme ve yüksek duty ile çalıştırmak yerine aşamalı doğrulama yapılır.

### 30.1 Aşama 0: Doküman ve bağlantı

1. Motorun anma gerilimi, akımı, kutup çifti ve bağlantıları bulunur.
2. Sürücü kartının tam parça numarası ve giriş modu belirlenir.
3. MCU ile sürücü arasında ortak referans/izolasyon yapısı doğrulanır.
4. U, V, W çıkışlarının motor fazlarına gittiği kontrol edilir.
5. Fault, enable ve akım sense hatları belirlenir.

### 30.2 Aşama 1: Motor bağlı değilken

1. Gate komutları kapalı başlıyor mu?
2. PWM frekansı doğru mu?
3. High/low complementary mi?
4. Dead-time osiloskopta var mı?
5. Fault tetiklenince çıkış donanımsal kapanıyor mu?
6. Altı adım sırası düşük frekansta izlenebiliyor mu?

### 30.3 Aşama 2: Düşük enerji

1. Akım limitli ve mümkün olan düşük uygun bus gerilimi kullanılır.
2. Motor mekanik olarak güvenli sabitlenir, yük ayrılır.
3. Çok düşük duty ile hizalama denenir.
4. Faz sırası ve dönme yönü gözlenir.
5. Akım şekli ve sürücü sıcaklığı izlenir.

### 30.4 Aşama 3: Kapalı çevrime geçiş

1. Hall sırası veya BEMF olayları yalnız gözlenir ve loglanır.
2. Olayların komütasyon tablosuyla sırası doğrulanır.
3. Open-loop rampanın takip edildiği düşük hızda kanıtlanır.
4. Güvenilir olay sayısından sonra RUN'a geçilir.
5. Hız ve akım limitleri adım adım yükseltilir.

#### İlk enerji verme

Bu adımlar kurum laboratuvar prosedürünün yerine geçmez. Güç elektroniği konusunda yetkili bir mühendisin gözetimi, uygun sigorta, akım sınırlama, acil kapatma ve kişisel koruyucu ekipman gerekir.

## Sık hatalar ve teşhis yolu

| Belirti                         | Olası neden                                           | İlk güvenli kontrol                                            |
|---------------------------------|-------------------------------------------------------|----------------------------------------------------------------|
| Motor yalnız titriyor           | Faz/Hall sırası yanlış; startup hızlı; duty yetersiz  | Enerjiyi kes, sektör sırasını ve düşük hızlı dalgaları doğrula |
| Akım aniden yükseliyor          | Shoot-through, yanlış faz deseni, kilitli rotor       | Break/fault kaydını ve dead-time'ı motor bağlı değilken ölç    |
| Yön ters                        | Faz sırası veya komütasyon yönü ters                  | Tablo yönünü kontrollü değiştir; çalışırken tersleme yapma     |
| Hall 000/111                    | Kablo/pull-up/sensör besleme sorunu                   | Giriş seviyelerini multimetre/logic analyzer ile kontrol et    |
| BEMF bulunamıyor                | Hız düşük, floating faz yanlış, gürültü/blanking      | Open-loop hızı, ölçüm penceresi ve sanal nötrü kontrol et      |
| RPM çok yanlış                  | Kutup çifti, timer clock veya sektör katsayısı yanlış | Formülü ölçülen event frekansı ile elle doğrula                |
| MCU reset oluyor                | Besleme çökmesi, EMI, watchdog, HardFault             | Reset cause register, besleme dalgası ve fault logunu incele   |
| Kod optimize edilince bozuluyor | Eksik volatile, yarış durumu, tanımsız davranış       | Uyarılar, ISR paylaşımı ve sınır ihlallerini incele            |

## Mini alıştırmalar

### Alıştırma 1: Bit maskesi

UH bit 0, VL bit 3 ise iki anahtarı seçen maskeyi ikilik ve hexadecimal yaz.

### Alıştırma 2: İşaretçi

```
1 void stop_motor(MotorControl motor)
2 {
3     motor.state = MOTOR_IDLE;
4 }
```

Bu fonksiyon çağırının motor durumunu neden değiştirmez? İmzasını düzelt.

### Alıştırma 3: volatile

ISR new\_sample bayrağını 1 yapıyor, ana döngü okuyup sıfırlıyor. Bayrak neden volatile olmalı? Neden volatile bütün yarış sorunlarını çözmez?

### Alıştırma 4: Timer

72 MHz timer clock'u, PSC =0 ve 18 kHz PWM için ARR yaklaşık kaç olmalıdır?

### Alıştırma 5: Elektriksel açı

5 kutup çiftli motorda bir 60 elektriksel derece sektör kaç mekanik derecedir? Bir mekanik turda kaç sektör olayı vardır?

### Alıştırma 6: Hata tasarımı

Geçersiz Hall kodunda yalnız return yapmak neden yetersizdir? Güvenli hata fonksiyonu hangi işlemleri yapmalıdır?

## Alıştırma çözümleri

### Çözüm 1

$$(1 \ll 0) | (1 \ll 3) = 000001 | 001000 = 001001_2 = 0x09$$

### Çözüm 2

Parametre değer ile geçirildiği için yapının kopyası değişir. Adres gönderilmelidir:

```

1 void stop_motor(MotorControl *motor)
2 {
3     if (motor != NULL)
4     {
5         motor->state = MOTOR_IDLE;
6     }
7 }
```

### Çözüm 3

volatile, ana döngünün bayrağı gerçekten bellekten tekrar okumasını ister. Ancak ISR'ın iki olayının tek bayrağa sıkışmasını, çok alanlı verinin tutarsız kopyalanmasını veya read-modify-write yarışını çözmez.

### Çözüm 4

$$ARR + 1 = \frac{72\,000\,000}{18\,000} = 4000 \Rightarrow ARR = 3999$$

### Çözüm 5

$$\theta_m = \frac{60}{5} = 12^\circ$$

Bir mekanik turda  $6 \times 5 = 30$  sektör olayı vardır.

### Çözüm 6

Yalnız dönmek önceki gate desenini aktif bırakabilir. Hata fonksiyonu en azından gate'leri kapatmalı, state'i FAULT yapmalı ve hata nedenini saklamalıdır. Donanım break yolu ayrıca bağımsız kapanma sağlamalıdır.

## Son kontrol listesi ve çalışma planı

### 34.1 İlk hafta

- C tipleri, fonksiyonlar, diziler ve işaretçiler için küçük PC programları yaz.
- Bit maskesiyle altı gate desenini üret ve test et.
- struct, enum, static, const, volatile örneklerini debugger ile gözle.
- Çok dosyalı projeyi GCC ile derle.

### 34.2 İkinci hafta

- STM32/ESP clock, GPIO ve timer örneği kur.
- PWM'i önce LED/logic analyzer üzerinde doğrula.
- Input capture veya periyodik timer kesmesi yaz.
- ISR ile ana döngü arasında güvenli event aktarımı kur.

### 34.3 Üçüncü hafta

- Sürücü veri sayfasından pin ve giriş modunu çıkar.
- Motor bağlı değilken complementary PWM, dead-time ve fault kapatmayı ölç.
- Komütasyon tablosunu sürücü truth table'ına uyarlayıp simüle et.
- Akım ölçümünü kalibre et.

### 34.4 Dördüncü hafta

- Yetkili gözetiminde düşük enerjili open-loop test yap.
- Hall veya BEMF olaylarını logla ve sırayı doğrula.
- Hız hesabı, fault durumları ve restart politikasını test et.
- Son olarak duty/hız kontrolünü kontrollü artır.

**Projeye hazır sayılman için**

- Motorun neden MCU'ya doğrudan bağlanmadığını açıklayabilmelisin.
- Altı adımın her birinde akım yolunu çizebilmelisin.
- Bir gate maskesini bitlerden üretebilmelisin.
- Timer PWM, dead-time, break ve input capture kavramlarını ayırabilmelisin.
- Hall ile yalnız üç kablolu sensorless motorun yazılım farkını bilmelisin.
- ISR'ı kısa tutup ana işleme veri aktarabilmelisin.
- Geçersiz durumda gate'leri güvenli kapatan tek hata yolun olmalı.

## Kısa sözlük

| Terim           | Kısa açıklama                                                                        |
|-----------------|--------------------------------------------------------------------------------------|
| ADC             | Analog gerilimi sayısal değere çeviren çevre birimi.                                 |
| Back-EMF / BEMF | Dönen motorun fazlarında oluşan ters elektromotor kuvvet.                            |
| BLDC            | Elektronik komütasyonlu kalıcı mıknatıslı fırçasız DC motor.                         |
| Bootstrap       | High-side gate sürmek için kullanılan şarj pompası/kondansatör yaklaşımı.            |
| Break / BKIN    | Timer PWM çıkışlarını hata anında donanımsal kapatan giriş.                          |
| Capture         | Bir olay anındaki timer sayacını donanımın kaydetmesi.                               |
| Commutation     | Faz akım yollarının rotor konumuna göre değiştirilmesi.                              |
| Dead-time       | Aynı yarım köprüde iki anahtarın da kapalı tutulduğu kısa süre.                      |
| Duty cycle      | PWM periyodunun aktif geçen oranı.                                                   |
| Gate driver     | MCU lojik sinyalini MOSFET gate'ini uygun hız/seviyede sürececek hale getiren devre. |
| GPIO            | Genel amaçlı dijital giriş/çıkış pini.                                               |
| Hall sensor     | Manyetik alanla rotor konum sektörü hakkında dijital bilgi veren sensör.             |
| ISR             | Kesme olduğunda çalışan kısa servis fonksiyonu.                                      |
| Linker          | Ayrı nesne dosyalarını ve adres yerleşimini tek firmware'de birleştiren araç.        |
| MOSFET          | İnverterde hızlı güç anahtarı olarak kullanılan transistör.                          |
| Open-loop       | Geri bildirim doğrulanmadan önceden belirlenmiş komutla ilerleme.                    |
| Pole pair       | Bir kuzey ve bir güney manyetik kutuptan oluşan çift.                                |
| PWM             | Darbe genişliği modülasyonu; ortalama güç/gerilim kontrolü yöntemi.                  |
| Register        | Donanım durum ve ayarlarının bit alanlarıyla tutulduğu özel konum.                   |

| Terim            | Kısa açıklama                                                                                 |
|------------------|-----------------------------------------------------------------------------------------------|
| Shoot-through    | Aynı yarım köprüde üst ve alt anahtarın birlikte ilettime girerek DC bus'ı kısa devre etmesi. |
| State machine    | Sistemin sınırlı durumlar ve açık geçiş kurallarıyla modellenmesi.                            |
| Translation unit | Ön işlem sonrasında tek bir .c dosyası ve dahil ettiklerinin oluşturduğu derleme birimi.      |
| Watchdog         | Yazılım kilitlenirse sistemi sıfırlayabilen bağımsız zamanlayıcı.                             |

## Başvuru kaynakları

Gerçek kart üzerinde çalışırken genel internet örneğinden önce şu birincil dokümanlar kullanılmalıdır:

1. Seçilen STM32/ESP modelinin datasheet'i ve reference manual'ı.
2. Motor sürücü/gate driver entegresinin datasheet'i ve evaluation board şeması.
3. Motorun gerilim, akım, kutup çifti ve sensör bağlantı bilgileri.
4. Kartın gerçek şeması, pinout'u ve kurumun mevcut driver kodu.
5. ARM Cortex-M kullanılıyorsa ilgili ARM Architecture/Cortex-M generic user guide.
6. C dili için ISO C standardına uyumlu güvenilir başvuru kitabı ve derleyici kılavuzu.

### Son hatırlatma

Bu dökümandaki register adresleri ve sürücü fonksiyonları kavramsaldır. Gerçek değerler kart seçilmeden belirlenemez. Datasheet'te doğrulanmayan bir pin modu veya gate deseni gerçek donanımda kullanılmamalıdır.

# Pin, Timer, UART ve Hertz

STM32 üzerinde bir pinin ne olduğu, timer'ın nasıl saydığı, iki cihazın UART ile nasıl konuştuğu ve bütün bunları birbirine bağlayan saat frekansı.

## 01 · Pinler

Alternate function, GPIO modları, pull-up/pull-down.

## 02 · Timer'lar

Sayaç, modlar, PWM ve duty cycle.

## 03 · UART

TX/RX, baud rate, 8N1 veri çerçevesi.

## 04 · Hertz

Frekans, periyot, PSC/ARR ve PLL.

## 01 Pinler gerçekte nedir?

Bir STM32'de yüzlerce pin vardır, ama hepsi aynı değildir. Pinler çoklu fonksiyon içeren yapılardır: aynı pin hem GPIO çıkışı, hem UART TX, hem timer PWM çıkışı olabilir. Hangi rolü üstleneceği yazılım tarafından seçilir.

Pin adı iki parçadan oluşur: port harfi ve pin numarası. PA0 port A'nın 0. pini, PB4 port B'nin 4. pini, PC13 port C'nin 13. pinidir.

Aynı pin bir anda yalnızca **bir** fonksiyonda çalışır. Seçim CubeMX'te yapılır.



- Dijital çıkış — LED yak
- Dijital giriş — buton oku
- Analog okuma — potansiyometre

- GPIO → UART\_TX / UART\_RX
- GPIO → Timer PWM çıkışı
- ADC → Buzzer / motor kontrol

- USART
- TIM
- TIM

ŞEKİL 1 — Tek bir pinin alabileceği roller (alternate function).

#### ALTERNATE FUNCTION (AF)

Hangi pinin hangi çevre birimi için kullanılacağını CubeMX ayarlar. UART kullanmak istiyorsanız o pinleri USART moduna çevirmeniz gerekir; GPIO olarak bırakılırsa UART çalışmaz. Pin doğru, mod yanlışsa hiçbir hata mesajı almazsınız — yalnızca veri gelmez.

### 1.1 Ana pin grupları

| Fonksiyon    | Açıklama                         | Örnek pinler       |
|--------------|----------------------------------|--------------------|
| GPIO         | Herhangi bir dijital giriş/çıkış | PA4, PB4, PB5      |
| UART / USART | Seri iletişim (TX, RX)           | PA2 (TX), PA3 (RX) |
| ADC          | Analog okuma (0–3,3 V)           | PA0, PA1           |
| Timer / PWM  | Zamanlama veya hız kontrolü      | PA6, PA7           |
| I2C          | Çift telli iletişim (SDA, SCL)   | PB6, PB7           |
| SPI          | Dört telli yüksek hızlı iletişim | PA5, PA6, PA7      |
| Güç          | Besleme ve şase                  | 3V3, 5V, GND       |
| SWD          | Debug ve programlama             | PA13, PA14         |
| Boot / reset | Sistem başlatma                  | BOOT0, NRST        |

### 1.2 GPIO modları

|           |                                            |                        |                                    |
|-----------|--------------------------------------------|------------------------|------------------------------------|
| OUTPUT_PP | Çıkış, push-pull — HIGH/LOW verir          | ANALOG                 | Analog okuma (ADC)                 |
| OUTPUT_OD | Çıkış, open-drain — pull-up ile kullanılır | IT_RISING / IT_FALLING | Giriş + kesme üretir (EXTI)        |
| INPUT     | Giriş — pin durumunu okur                  | AF_PP / AF_OD          | Alternate function (UART, TIM vb.) |

#### PUSH-PULL (PP)

HIGH verildiğinde pin 3,3 V'a, LOW verildiğinde 0 V'a sürülür. Her iki seviye de aktif olarak üretilir.

Çoğu durumda kullanılan varsayılan çıkış modu.

#### OPEN-DRAIN (OD)

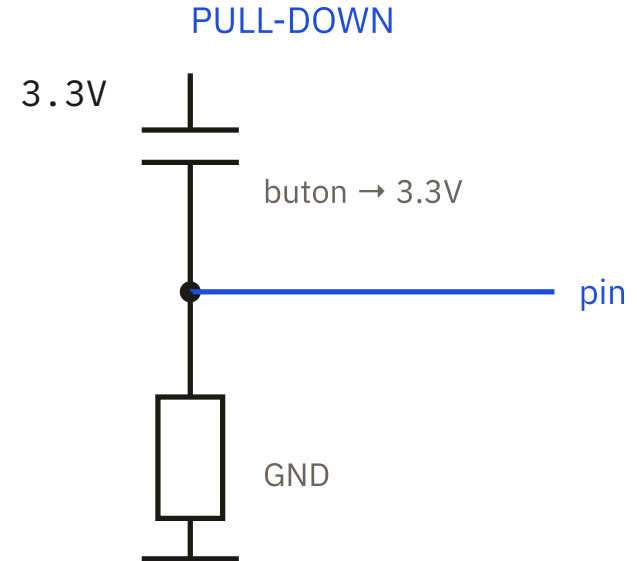
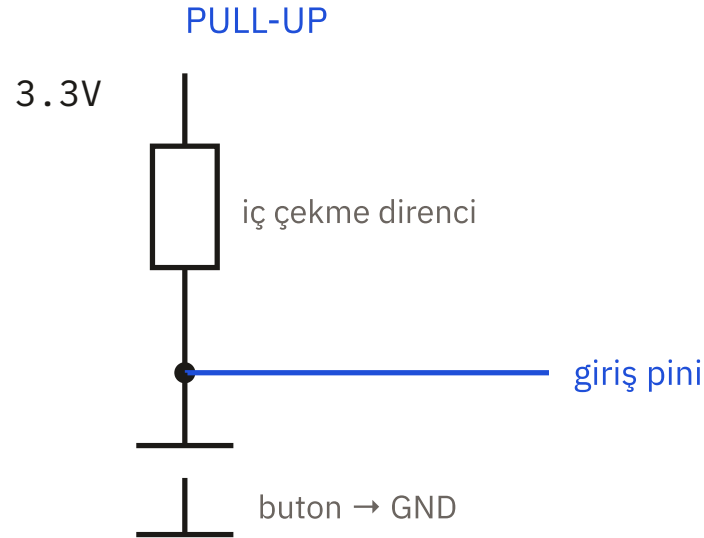
LOW verildiğinde pin 0 V'a çekilir. HIGH verildiğinde ise pin sürülmez, açık kalır — seviyeyi harici pull-up direnci belirler.

I2C gibi “wire-AND” protokollerinde zorunludur.

### 1.3 Pull-up ve pull-down nedir?

Bir giriş pini hiçbir yere bağlı değilse *float* durumdadır ve okunan değer belirsizdir; gürültüyle 0 veya 1 görünebilir. Pull-up direnci pini boştayken 3,3 V'a, pull-down direnci ise 0 V'a çeker. Böylece “basılmamış” hâlin ne okunacağı garanti edilir.

Bu projede Hall pinlerinde GPIO\_PULLUP var; boşta yüksek, sensör bastığında düşük.



ŞEKİL 2 — Pull-up'ta boşta seviye HIGH, buton LOW verir. Pull-down'da tersi geçerlidir.

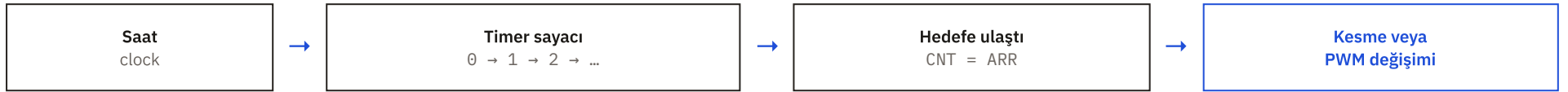
## HIZLI TEKRAR · BÖLÜM 01

1. Bir pini UART için ayırmayı unutursanız program derlenir mi, çalışır mı?
2. Open-drain çıkış neden tek başına HIGH üretmez?
3. Pull-up'lı bir butonda basılmamış hâl hangi seviyedir?

## 02 Timer'lar

Timer, içinde bir sayaç (counter) bulunan çevre birimidir. Saat sinyalinden beslenir, belirli aralıklarla otomatik olarak sayar ve CPU'yu meşgul etmez. Sayaç hedef değere ulaştığında kesme üretir veya bir çıkış pininin seviyesini değiştirir.

Timer'ın değeri, işi **yazılım olmadan** yapmasıdır. CPU yalnızca sonucu okur.



ŞEKİL 3 — Timer'ın çalışma döngüsü.

### 2.1 Timer modları

| Mod           | Ne yapar                              | Örnek                      |
|---------------|---------------------------------------|----------------------------|
| Basic         | Sadece sayar, kesme üretir            | Her 1 s'de zamanlayıcı     |
| PWM           | Belirli oranda aç/kapa sinyali üretir | Motor hızı, LED parlaklığı |
| Encoder       | A/B sinyallerini donanımda sayar      | Motor pozisyon ve hızı     |
| Input capture | Sinyalin değişim anını/süresini ölçer | Frekans ölçümü             |
| One-pulse     | Tetiklenince tek darbe üretir         | Darbe üretme               |
| Watchdog      | Program takılırsa sistemi sıfırlar    | Güvenlik                   |

## 2.2 G4 ailesindeki timer'lar

| Timer                                                               | Özellikler                                   | Kullanım alanı          |
|---------------------------------------------------------------------|----------------------------------------------|-------------------------|
| <a href="#">TIM1</a> , <a href="#">TIM8</a> , <a href="#">TIM20</a> | Gelişmiş, 16-bit, PWM + komütasyon + encoder | Motor kontrolü (BLDC)   |
| <a href="#">TIM2</a> ... <a href="#">TIM5</a>                       | Genel amaçlı, 32/16-bit, encoder + PWM       | Pozisyon ve PWM         |
| <a href="#">TIM6</a> , <a href="#">TIM7</a>                         | Basic, 16-bit, yalnızca zamanlama            | Periyodik kesme         |
| <a href="#">LPTIM</a>                                               | Düşük güç timer'ı                            | Uyku modunda zaman      |
| <a href="#">SysTick</a>                                             | ARM çekirdeğinin zamanlayıcısı               | ms sayacı (HAL_GetTick) |

## 2.3 PWM ve duty cycle

PWM (Pulse Width Modulation), bir pini hızla açıp kapatarak ortalama olarak “orta seviye” bir güç üretmektir. Duty cycle, bir periyot içinde sinyalin ne kadar süre açık kaldığının yüzdesidir. Frekans sabit tutulur, yalnızca açık kalma oranı değişir; motor hızı veya LED parlaklığı bu oranla ayarlanır.

Periyot = 1 / frekans. Duty cycle periyodu değiştirmez, yalnızca içindeki açık süreyi değiştirir.

100%



tam güç

75%



parlak

50%



yarı parlak

25%



soluk

0%



kapalı



bir periyot — bütün satırlarda aynı

ŞEKİL 4 — Aynı frekans, farklı duty cycle. Ortalama gerilim açık kalma oranıyla değişir.

#### BU PROJEDE

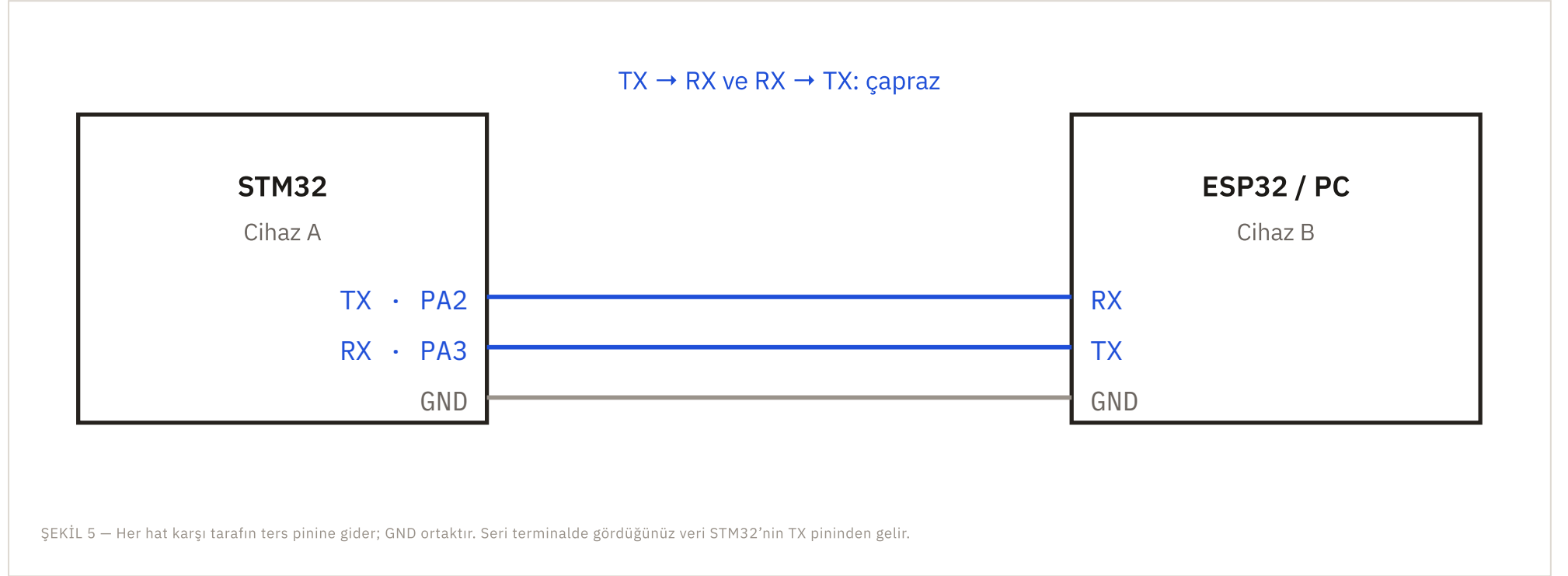
Motor PWM ile değil, GPIO aç/kapa ile sürülüyor: her 6-step adımında ilgili pinler elle HIGH/LOW yapılıyor. Gerçek motor kontrolünde ise sürücüye PWM verilir ve duty cycle doğrudan motor hızını belirler.

### 03 RX / TX pinleri: UART

UART (Universal Asynchronous Receiver/Transmitter), iki cihaz arasında iki tel üzerinden seri veri taşır. TX pini veri gönderir, RX pini veri alır. En yaygın seri iletişim yöntemidir ve ayrı bir saat teli gerektirmez — bu yüzden “asenكرون”dur.

Bağlantının tek kritik kuralı çaprazlamadır: A cihazının TX’i, B cihazının RX’ine gider. Ayrıca iki taraf ortak bir GND paylaşmak zorundadır.

Ortak GND olmadan seviyeler referanssız kalır ve iletişim çalışmaz. Bu **şarttır**, tercih değil.



### 3.1 Baud rate

Baud, saniyede gönderilen bit sayısıdır. Alıcı ve verici *aynı* baud ile çalışmak zorundadır; aksi hâlde veri anlamsız karakterlere dönüşür.

115200 → 115.200 bit/s

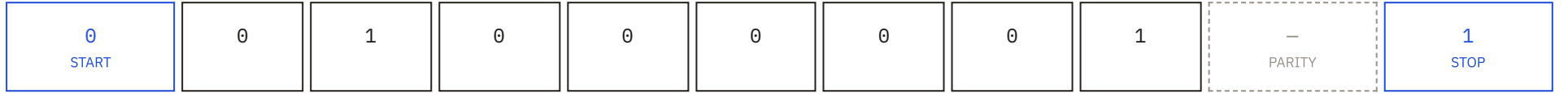
9600 → 9.600 bit/s

#### BU PROJEDE

115200 baud kullanılıyor. Düşük baud değerleri daha yavaştır, ancak uzun ve gürültülü hatlarda daha güvenilirdir.

### 3.2 Veri çerçevesi: 8N1

Bir karakter, örneğin 'A' = 0x41, hat üzerine şu sırayla çıkar: bir start biti, sekiz veri biti, isteğe bağlı parity biti ve bir ya da iki stop biti.



ŞEKİL 6 – 'A' = 0x41 = 0100 0001. Ortadaki sekiz kutu veri bitleridir.

WordLength 8 · Parity None · StopBits 1 → 8N1

### 3.3 Gönderme fonksiyonları

```
// Senkron – gönderme bitene kadar bekler (bloke edici)
HAL_UART_Transmit(&hlpuart1, data, size, timeout);

// Kesme tabanlı – göndermeye baslar, arka planda tamamlar
HAL_UART_Transmit_IT(&hlpuart1, data, size);

// DMA tabanlı – en verimli, büyük veriler için
HAL_UART_Transmit_DMA(&hlpuart1, data, size);
```

#### NEDEN LPUART1?

Bu projede USART yerine LPUART1 (Low Power UART) kullanılıyor. Aynı işi yapar, fakat düşük güç tüketir; PA2 = TX, PA3 = RX pinlerine bağlıdır. Düşük güç mimarisi nedeniyle üst baud sınırı daha düşüktür, 115200 civarında kalır.

#### HIZLI TEKRAR · BÖLÜM 03

1. TX-TX bağlarsanız ne olur?
2. Bir taraf 9600, diğeri 115200 baud ile çalışırsa terminalde ne görürsünüz?
3. 8N1 çerçevesinde bir karakter için hat üzerinde toplam kaç bit gider?

## 04 Hertz: frekans, periyot ve saat

Hertz, bir olayın saniyede kaç kez tekrarlandığını gösterir. Frekans tekrar sayısıdır, periyot ise tek bir tekrarın süresidir; ikisi birbirinin tersidir. Frekans büyüdükçe periyot küçülür.

Periyot = 1 / frekans. Zamanlama hesaplarında hemen her adım bu iki satır arasında gidip gelir.

1 Hz

periyot 1 s

10 Hz

periyot 100 ms

100 Hz

periyot 10 ms

170 MHz

periyot 5,88 ns

### 4.1 CPU saati ve dağıtımı

STM32 hızlı çalışabilmek için bir saat sinyali kullanır. Bu sinyal çekirdekten çevre birimlerine kadar bütün bloklara dağıtılır; her blok kendi bus'ı üzerinden beslenir.

**SYSCCLK**  
**170 MHz**

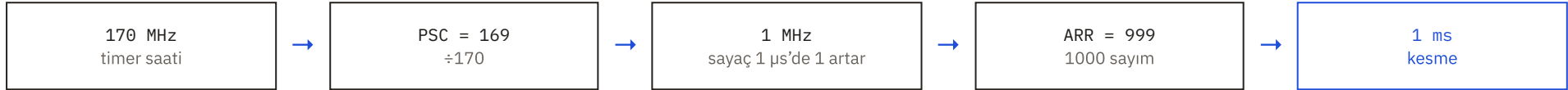
|                                   |          |
|-----------------------------------|----------|
| → CPU — komutları çalıştırır      | 170 MHz  |
| → AHB bus — GPIO ve bellek        | 170 MHz  |
| → APB1 bus — timer'lar buna bağlı | bölünmüş |
| → APB2 bus — UART, ADC, TIM       | bölünmüş |

ŞEKİL 7 — Saat dağıtımı. Bir çevre birimi çalışmıyorsa ilk kontrol edilecek şey saatinin açık olup olmadığıdır.

#### 4.2 Timer'da Hz: prescaler ve auto-reload

Timer sayacı, timer saatinden beslenir; saat ne kadar hızlıysa sayaç o kadar hızlı artar. Prescaler (PSC) gelen frekansı böler, auto-reload (ARR) ise sayacın hangi değerde sıfırlanacağını belirler. İkisi birlikte kesme aralığını verir.

PSC ve ARR register'larına yazılan değer, istenen bölme oranından **bir eksiktir**: 170 bölme için PSC = 169.



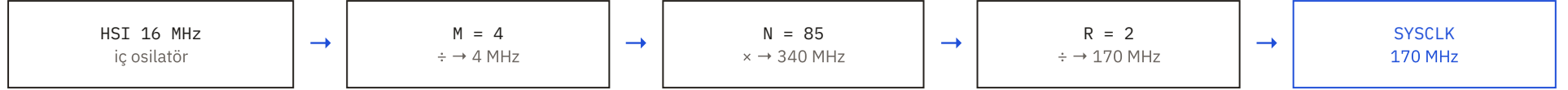
ŞEKİL 8 — 1 ms periyodik kesme:  $1000 \times 1 \mu s = 1 ms$ .

#### KESME FREKANSI FORMÜLÜ

$$f_{kesme} = f_{timer} / ((PSC + 1) \times (ARR + 1))$$

### 4.3 PLL ve saat kaynakları

CPU 170 MHz'e iç ya da dış osilatörden gelen düşük frekansı PLL ile çarparak ulaşır. PLL içinde önce bir bölme (M), sonra bir çarpma (N), sonra tekrar bir bölme (R) yapılır.



ŞEKİL 9 – PLL zinciri. Kaynak olarak HSI (16 MHz, iç) veya HSE (8/25 MHz, dış kristal) seçilebilir.

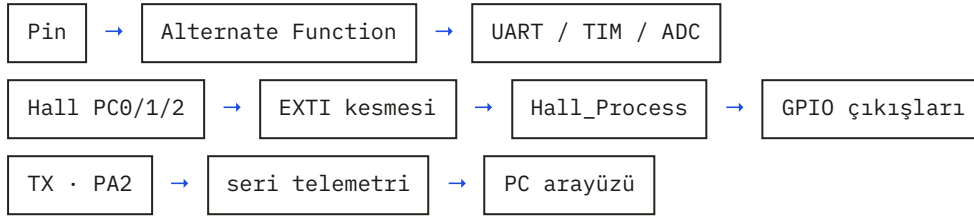
### 4.4 Bu projedeki zaman büyüklükleri

| Kavram                          | Anlamı                        | Bu projede                   |
|---------------------------------|-------------------------------|------------------------------|
| <a href="#">SYSCLK</a>          | İşlemci saat hızı             | 170 MHz                      |
| <a href="#">HAL_GetTick()</a>   | SysTick'in saydığı milisaniye | Zaman ölçümü (now_ms)        |
| <a href="#">UART baud</a>       | Seri veri hızı                | 115200 baud                  |
| <a href="#">Elektriksel RPM</a> | Elektriksel dönüş hızı        | 6 adım × komütasyon frekansı |
| <a href="#">Frekans</a>         | Saniyedeki tekrar sayısı      | Hall geçişleri               |

## 05 Her şey nasıl birbirine bağlanıyor?

CLOCK  
170 MHz

- **CPU** komutları çalıştırır
- **Timer** sayaç, PWM, encoder arayüzü
- **GPIO** pinleri okur ve sürer
- **UART** veri gönderir (baud = bit/s)



ŞEKİL 10 — Saat her şeyi besler; pin fonksiyonu çevre birimini seçer; kesme yazılım akışını tetikler.

### ANAHTAR TERİMLER

|                               |                                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------|
| <b>Alternate function</b>     | Bir pinin GPIO dışında bir çevre birimine (UART, TIM, ADC) atanması.                 |
| <b>Push-pull / open-drain</b> | Çıkışın her iki seviyeyi de sürmesi / yalnızca LOW'a çekmesi.                        |
| <b>Float</b>                  | Hiçbir yere bağlı olmayan, okunan değeri belirsiz giriş pini durumu.                 |
| <b>Duty cycle</b>             | Bir PWM periyodunda sinyalin açık kaldığı sürenin yüzdesi.                           |
| <b>PSC / ARR</b>              | Timer'ın prescaler ve auto-reload register'ları; birlikte sayma periyodunu belirler. |
| <b>Baud rate</b>              | UART'ta saniyede taşınan bit sayısı; iki tarafta aynı olmak zorundadır.              |

|               |                                                                                    |
|---------------|------------------------------------------------------------------------------------|
| <b>8N1</b>    | 8 veri biti, parity yok, 1 stop biti — en yaygın UART çerçeve biçimi.              |
| <b>SYSCLK</b> | Sistem saat frekansı; CPU ve bus'lar buradan beslenir.                             |
| <b>PLL</b>    | Düşük frekanslı osilatör sinyalini bölüp çarparak yüksek sistem saati üreten blok. |

Not: Pin alternatif fonksiyon numaraları, timer genişlikleri, bus bölme oranları ve PLL sınırları kullanılan STM32 modelinin datasheet ve reference manual belgelerinden doğrulanmalıdır.

# 2

KISIM İKİ

## İlk STM32 Uygulaması

Teoriyi ilk kez çalışan bir projeye dönüştürme aşaması. LED'i PWM ile sürmek, butonu kesme ile okumak ve programı enum + switch-case ile yazılmış bir state machine olarak kurmak — ilerideki komütasyon mantığının aynı iskeleti burada oturuyor.

---

BU KISIMDA

- 03 LED PWM Kontrolü ve Buton ile State Machine  
TIM2 PWM · EXTI/NVIC kesme · 4 durumlu state machine

# LED PWM Kontrolü ve Buton ile State Machine

TIM2 tabanlı yazılımsal PWM, EXTI/NVIC kesme sistemi ve enum + switch-case ile yazılmış 4 durumlu bir state machine'in uçtan uca çalışma mantığı.

**01** Mikrodenetleyici genel bakış

---

**02** Saat sistemi (clock tree)

---

**03** CubeMX yapılandırması

---

**04** TIM2 ile yazılımsal PWM

---

**05** EXTI + NVIC ile buton kesmesi

---

**06** State machine tasarımı

---

**07** Tam kod ve hızlı tekrar

---

**08** Test: 12 soru, cevaplı

---

Blok diyagram ve pinout, STM32F401xB/xC veri sayfasından (DS9716 Rev 11) alınmıştır; diğer şemalar bu proje için çizilmiştir.

# 01

## Mikrodenetleyici genel bakış

Hangi çip, hangi bacaklar, hangi çevre birimleri. Kodu okumadan önce elimizdeki donanımı tanıyoruz.

# STM32F401CCU6 künyesi

|                 |                               |
|-----------------|-------------------------------|
| Çekirdek        | Cortex-M4 + FPU, maks. 84 MHz |
| Flash           | 256 KB                        |
| SRAM            | 64 KB                         |
| Paket           | UFQFPN48 (7×7 mm)             |
| Zamanlayıcı     | TIM1 + 7 genel amaçlı         |
| Bu projede saat | HSI = 16 MHz (PLL kapalı)     |

## SOMUT KARŞILIK

84 MHz'lik bir çipi 16 MHz'de çalıştırmak, 240 km/h yapan arabayı şehir içinde 60'ta sürmek gibi. Bir LED'i soldurmak ve bir butonu okumak için motoru sonuna kadar açmaya gerek yok.

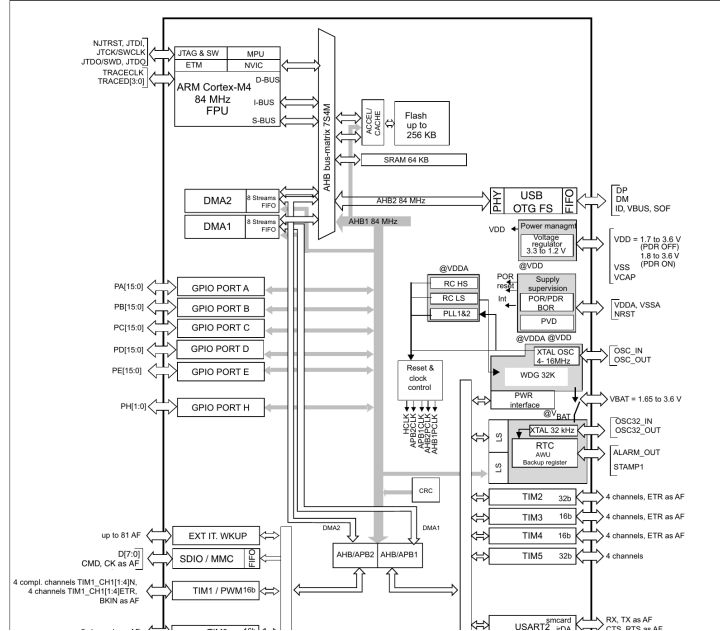
Kazanç: daha az güç, çok daha basit  
SystemClock\_Config, Flash için bekleme durumu yok.

# İç blok diyagramı

Description

STM32F401xB STM32F401xC

Figure 3. STM32F401xB/STM32F401xC block diagram



## DİYAGRAMDA NEREYE BAKMALI

- 1 Sol kolon: **GPIO PORT A** ve **GPIO PORT C** – PA0 ve PC13 buradan çıkıyor.
- 2 Sağ alt: **TIM2 32b** satırı – APB1 hattına bağlı, projedeki tek zamanlayıcı.
- 3 Sol üst: **NVIC** kutusu – hem TIM2 hem EXTI kesmesi CPU'ya buradan giriyor.
- 4 Sol alt: **EXT IT. WKUP** – EXTI hatlarının çipteki karşılığı.

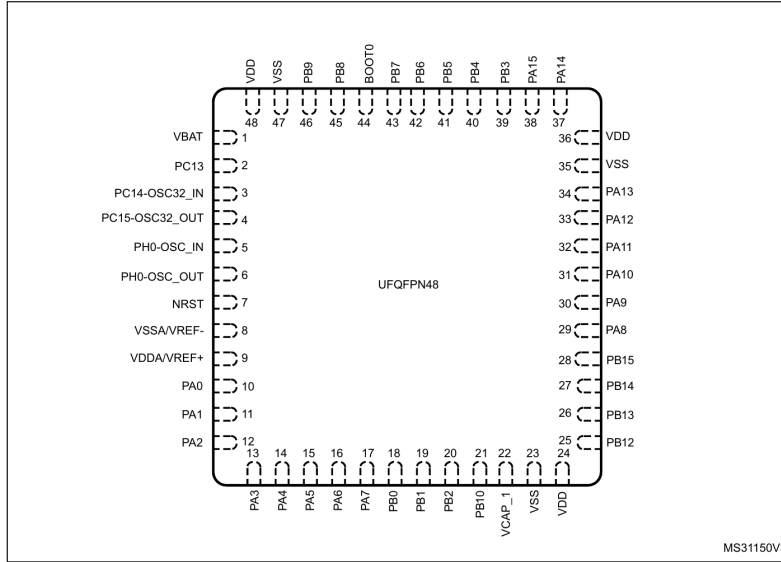
Kaynak: STM32F401xB/xC Datasheet, DS9716 Rev 11, Figure 3

# UFQFPN48 pin yerleşimi

STM32F401xB STM32F401xC

Pinouts and pin description

Figure 11. STM32F401xB/STM32F401xC UFQFPN48 pinout



1. The above figure shows the package top view.

## Pin 2 – PC13

LED çıkışı. Sol kenarda, Üstten ikinci bacak.

## Pin 10 – PA0

KEY girişi. Yine sol kenarda, VDDA/VREF+ bacağının hemen altında.

Şema Üstten görünüştür. Çipi kartla karşılaştırırken pin 1 nokta işaretini referans alın.

Kaynak: DS9716 Rev 11, Figure 11

## Aktif-low LED bağlantısı

Black Pill kartında LED'in anodu 3.3 V'a, katodu PC13'e bağlı. Akım pinden dışarı değil, pinin **içine** akar. Yani devre ancak pin LOW iken tamamlanır.

### PIN LOW = LED YANIK

```
HAL_GPIO_WritePin(LED_GPIO_Port,  
                  LED_Pin, GPIO_PIN_RESET);
```

Katot 0 V'a çekilir, 3.3 V üzerinden akım geçer.

### PIN HIGH = LED SÖNÜK

```
HAL_GPIO_WritePin(LED_GPIO_Port,  
                  LED_Pin, GPIO_PIN_SET);
```

Pinin iki ucu da 3.3 V; potansiyel farkı yok, akım yok.

Bu ters mantık ilerideki PWM kodunda kritik: **pwm\_phase < brightness** koşulu sağlandığında pini SET değil RESET yapıyoruz.

# 02

## Saat sistemi (clock tree)

TIM2'nin kaç Hz'de saydığını bilmeden PWM frekansı hesaplanamaz. Her şey saat ağacının tepesinden başlar.

# HSI'den TIM2CLK'a



**Sınavlık kural:** APB1 prescaler 1 ise  $TIMxCLK = PCLK1$ . Prescaler 1'den büyükse  $TIMxCLK = 2 \times PCLK1$ . Burada bölen 1 olduğu için çarpan devreye girmiyor.

# Saat ağacı = fabrikanın şaft hattı

## BENZETME

Eski bir fabrikada tavanda tek bir ana şaft döner. Her tezgâh bu şafta kayış ve kasnakla bağlanır; kasnak oranı o tezgâhın hızını belirler.

HSI = motor. SYSCCLK = ana şaft. AHB ve APB prescaler'ları = kasnaklar. TIM2 = tezgâhlardan biri. Motor hızını değiştirirseniz bütün tezgâhlar etkilenir; sadece kasnağı değiştirirseniz yalnız o tezgâh.

## PRATİKTE NE DEMEK

- PLL'i açıp 84 MHz'e çıkıydınız, **aynı PSC/ARR değerleriyle** LED 5,25 kat hızlı solardı.
- Bu yüzden "PWM frekansım neden yanlış?" sorusunun cevabı çoğu zaman kodda değil, CubeMX'in Clock Configuration sekmesindedir.
- 16 MHz'de Flash bekleme durumu gerekmez: **FLASH\_LATENCY\_0**. Yüksek frekansta bu değer artar; unutulursa çip kilitlenir.

**Neden PLL kullanmadık?** PLL saati 84 MHz'e çıkarmak içindir. Bir LED soluma efekti ve tek buton okuması için gereksiz; 16 MHz HSI hem yeterli hem de kurulumu çok daha basit.

# 03

## CubeMX yapılandırması

Grafik arayüzde yapılan her ayar .ioc dosyasına yazılır ve "Generate Code" ile HAL init fonksiyonlarına dönüşür.

# GPIO ayarları

| PIN  | KULLANIM   | MOD                       | PULL    |
|------|------------|---------------------------|---------|
| PC13 | LED çıkışı | GPIO_Output (Push-Pull)   | No pull |
| PA0  | KEY girişi | GPIO_EXTI0 (falling edge) | Pull-up |

## NEDEN PULL-UP

Butona basılmadığında PA0 hiçbir yere bağlı değildir; havada kalan giriş pini antene dönüşür ve rastgele HIGH/LOW okur (floating). Dahili pull-up direnci pini boştayken 3.3 V'a "yapıştırır". Basınca pin GND'ye çekilir, temiz bir HIGH→LOW geçişi oluşur.

## NEDEN FALLING EDGE

Basma anı düşen kenar, bırakma anı yükselen kenardır. Sadece falling seçilince her basış tek olay üretir; "both edges" seçilseydi bir basıp bırakma iki kesme doğururdu.

# TIM2 ve NVIC ayarları

## TIM2

|                 |                         |
|-----------------|-------------------------|
| Saat kaynağı    | Internal Clock – 16 MHz |
| Prescaler (PSC) | 15 → bölen 16           |
| Period (ARR)    | 99 → 100 adım           |
| Counter Mode    | Up                      |
| Başlatma        | HAL_TIM_Base_Start_IT() |

## NVIC

|            |                   |
|------------|-------------------|
| EXTI0_IRQn | Aktif • öncelik 0 |
| TIM2_IRQn  | Aktif • öncelik 0 |

İkisi de aynı öncelikte olduğu için biri diğerini kesemez, sıraya girerler. LED soluma için bu yeterli.

**USER CODE** Bu projede main.c'de CubeMX'in **USER CODE BEGIN/END** etiketleri yoktu. Bir pin ayarı değiştirip "Generate Code" deneysdi PWM, fade ve state machine kodunun tamamı silinebilirdi. Kural: elle düzenlenen HAL projelerinde bu etiketler asla silinmez, tüm özel kod aralarına yazılır.

# 04

## TIM2 ile yazılımsal PWM

Zamanlayıcı sadece düzenli aralıklarla kesme üretir; pini yakıp söndürme işini kod yapar.

# Donanımsal PWM yerine yazılımsal PWM

## DONANIMSAL (kullanılmadı)

TIM2'nin CH1..CH4 çıkışları belirli pinlere bağlıdır. CCR registerine bir sayı yazarsınız, darbe genişliğini donanım üretir. CPU hiç uğraşmaz.

Sorun: PC13, TIM2'nin PWM çıkış kanallarından biri değil. Karttaki LED donanım PWM'e bağlanamıyor.

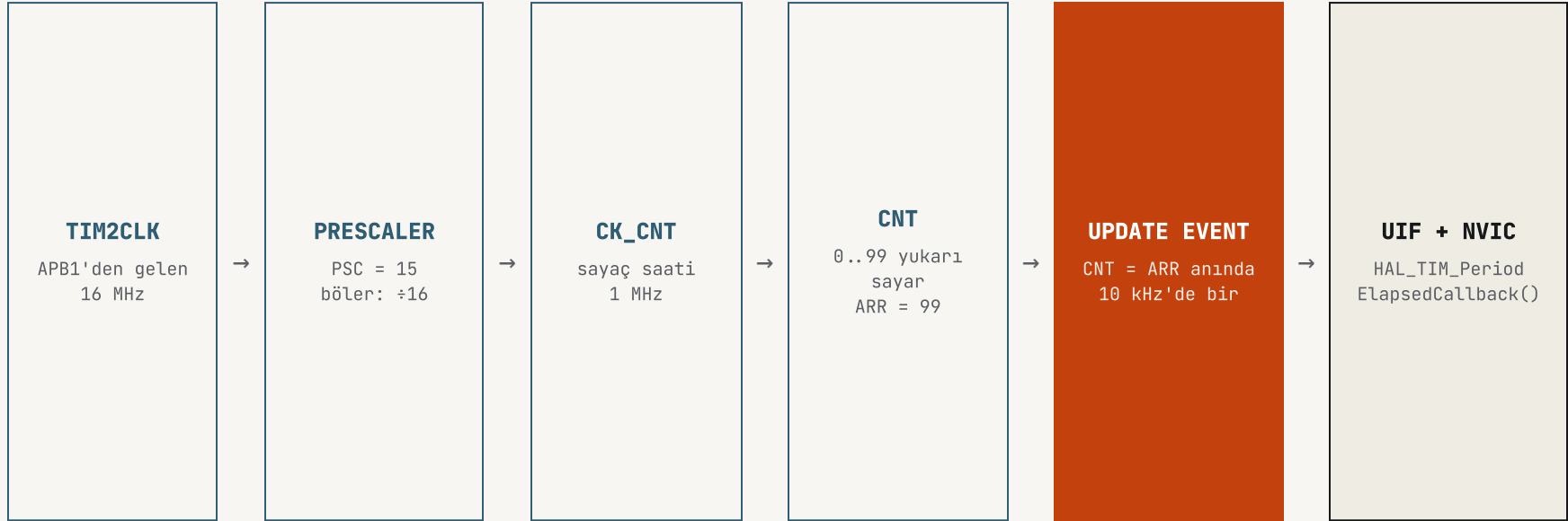
## YAZILIMSAL (bu proje)

TIM2 yalnızca metronom: her 100  $\mu$ s'de bir kesme atar. Kesme içinde bir sayaç (pwm\_phase) döner ve pin elle SET/RESET yapılır.

Bedeli: saniyede 10.000 kesme, yani sürekli CPU yükü. Kazancı: **herhangi bir** GPIO pininde PWM.

**Somut:** Donanımsal PWM, kurduktan sonra kendi kendine çalan bir müzik kutusudur. Yazılımsal PWM ise metronom eşliğinde her vuruşta elle tuşa basmaktır – aynı sesi çıkarır, ama eliniz sürekli meşguldür.

## TIM2 iç zinciri: PSC → CNT → ARR → UEV



Zincirin ilk beş kutusu **donanımda** kendiliğinden döner. Biz sadece `HAL_TIM_Base_Start_IT(&htim2)` diyerek çarkı başlatıyoruz.

UIF = Update Interrupt Flag. Donanım bayrağı kaldırır, NVIC bunu CPU'ya iletir, HAL bayrağı temizleyip callback'imizi çağırır.

# Prescaler ve ARR'ı somutlaştırmak

## **PRESCALER = dişli kutusu**

Motor saniyede 16 milyon tur atıyor ve sayacın bunu takip etmesini istemiyorsunuz. Araya 16:1 dişli koyarsınız: motor 16 tur atınca çıkış mili 1 tur atar.  $PSC = 15$  tam olarak budur – **bölen =  $PSC + 1$ .**

Neden +1? Çünkü  $PSC = 0$  "bölme yok" demektir; register sıfırdan başlar.

## **ARR = kilometre sayacının turu**

Eski araba kilometre sayacı 999999'a gelince 000000'a döner. ARR o "en büyük sayı"dır. CNT 99'a gelip sıfıra döndüğü an bir tur bitmiştir ve zil çalar: **Update Event.**

Adım sayısı =  $ARR + 1 = 100$ . Sıfır da bir adımdır.

### **TEK CÜMLE**

Prescaler ne kadar sık sayılacağını, ARR kaç kadar sayılacağını belirler. Kesme frekansı ikisinin çarpımına bölünür.

## Frekans hesabı, dört adım

01 • sayaç saati

$16.000.000 \div (15+1)$

**1 MHz**

02 • kesme frekansı

$1.000.000 \div (99+1)$

**10 kHz**

03 • PWM periyodu

$100 \text{ adım} \times 100 \mu\text{s}$

**10 ms**

04 • PWM frekansı

$1 \div 10 \text{ ms}$

**100 Hz**

CK\_CNT – CNT bu hızda artar

her 100  $\mu\text{s}$ 'de bir callback

pwm\_phase bir tur atar

göz titremeyi fark etmez

**Eşik bilgisi:** insan gözü yaklaşık 60-80 Hz üzerinde kırpmayı ayırt edemez. 100 Hz güvenli tarafta; 30 Hz'e düşseydi LED gözle görülür şekilde titrerdi.

## Zaman ekseninde: CK\_CNT, CNT, Update Event

TIM2CLK



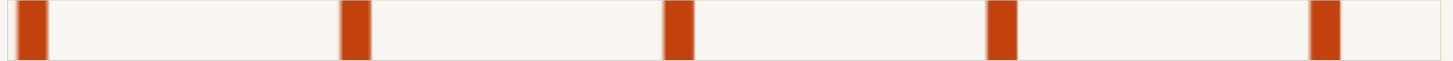
CK\_CNT  
(örnekte ÷4)



CNT  
(ARR = 1)



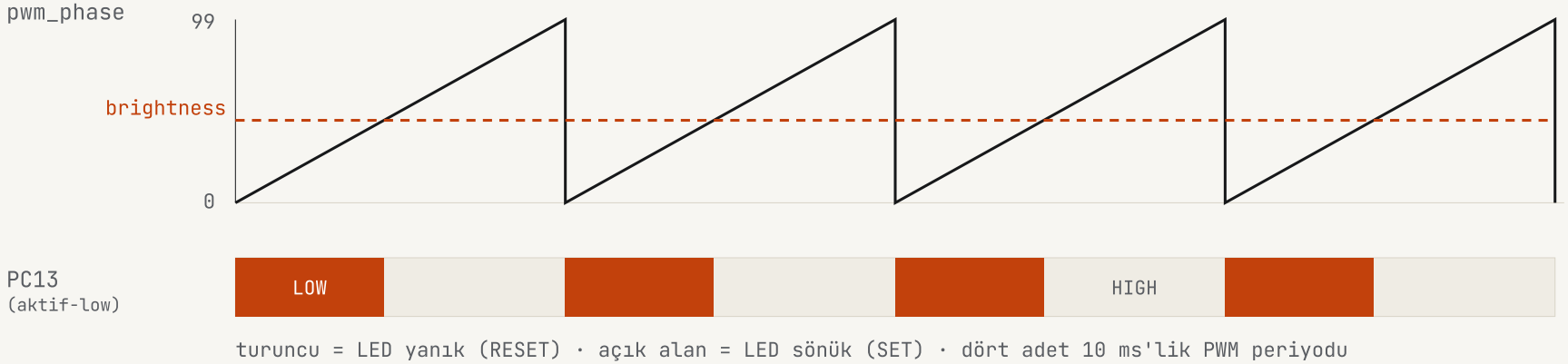
UEV



CNT, ARR'a ulaştı → 0'a döner, UIF bayrağı set edilir, kesme tetiklenir

Okunabilirlik için burada bölme oranı 1:4 ve ARR = 1 çizildi. Gerçek projede oran 1:16 (PSC = 15) ve ARR = 99'dur; mantık birebir aynı, sadece süreler uzun.

## pwm\_phase ve brightness ilişkisi



**Okuma biçimi:** testere dişi her turda 0'dan 99'a tırmanır. Kesikli çizginin **altında** kaldığı sürece LED yanar. Çizgiyi yukarı kaydırmak (`brightness`'ı artırmak) turuncu blokları genişletir; göz bunu "daha parlak" olarak okur.

## PWM üreten kısım

```
/* TIM2 her 100 us'de bir cagirir */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    /* ...fade mantigi... */

    pwm_phase++;
    if (pwm_phase ≥ 100) { pwm_phase = 0; }

    if (pwm_phase < brightness)
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin,
                           GPIO_PIN_RESET); // yanar
    else
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin,
                           GPIO_PIN_SET);   // soner
}
```

- 1 Fonksiyon saniyede 10.000 kez çalışır. İçine uzun iş koymak yasak: döngü, delay, printf yok.
- 2 **pwm\_phase** 0..99 arası sürekli döner; PWM'in yatay eksenidir.
- 3 **brightness** 0..100 arası eşiktir; grafikteki yatay kesikli çizgidir.
- 4 **brightness = 0** → koşul hiç sağlanmaz, LED hep sönmük. **brightness = 100** → hep sağlanır, LED tam parlak.

---

HAL bu callback'i tüm timer'lar için ortak çağırır. Projede tek timer olduğundan htim kontrolü yapılmadı; ikinci bir timer eklenirse `if (htim->Instance == TIM2)` şart olur.

# 05

## EXTI + NVIC ile buton kesmesi

PA0'a basılmasından bizim switch-case'imizin çalışmasına kadar sinyalin izlediği yol.

# Kesme zinciri: bacadan koda



Üst sıra çipte kendiliğinden olur. Alt sıradaki ilk iki fonksiyonu CubeMX ve HAL yazar; elle dokunduğumuz tek yer en sağdaki callback'tir.

# Kesme = kapı zili, polling = pencereden bakmak

## POLLING

Misafir gelecek diye her on saniyede bir pencereden bakarsınız. Çoğu bakış boşa gider; Üstelik tam bakmadığınız anda gelirse kaçırsınız.

```
while (1) {  
    if (HAL_GPIO_ReadPin(...)) { ... }  
}
```

## KESME

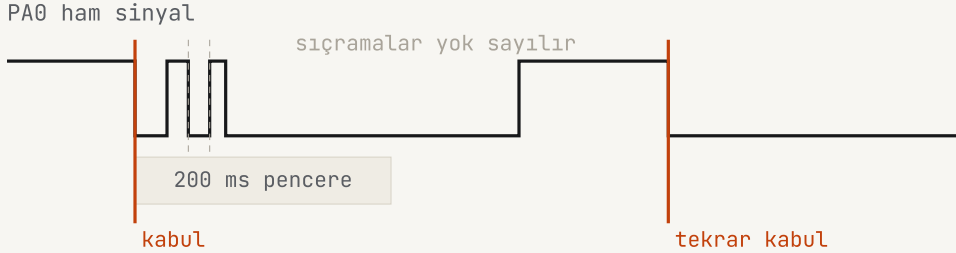
Zili takarsınız ve işinize dönersiniz. Zil çaldığı an ne yapıyorsanız bırakıp kapıya gider, sonra kaldığınız yerden devam edersiniz.

```
while (1) { }    // bos  
// her sey callback'lerde
```

## BENZETMENİN SINIRI

Kapıya giderken elinizdeki tepsiyi masaya bırakmanız gerekir; CPU da register'larını yığına (stack) kaydeder. Bu yüzden kesme fonksiyonları kısa olmalı: kapıda uzun sohbet ederseniz mutfakta bir şey yanar.

## Debounce: 200 ms'lik sessizlik penceresi



Mekanik kontaklar kapanırken birkaç milisaniye titrer. Her titreşim ayrı bir falling edge, yani ayrı bir kesmedir: tek basışta state machine iki üç durum birden atlayabilir.

**Somut:** Kapı zilinin düğmesi gevşek olsa bir basışta zil üç kez çalardı. Çözüm zili değiştirmek değil; "son çalıştan bu yana 200 ms geçmediyse duymazdan gel" demek.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    static uint32_t last_tick = 0;
    uint32_t now = HAL_GetTick();

    if (GPIO_Pin != KEY_Pin)    { return; }
    if ((now - last_tick) < 200) { return; }

    last_tick = now;

    switch (led_state) { /* durum gecisi */
    }
}
```

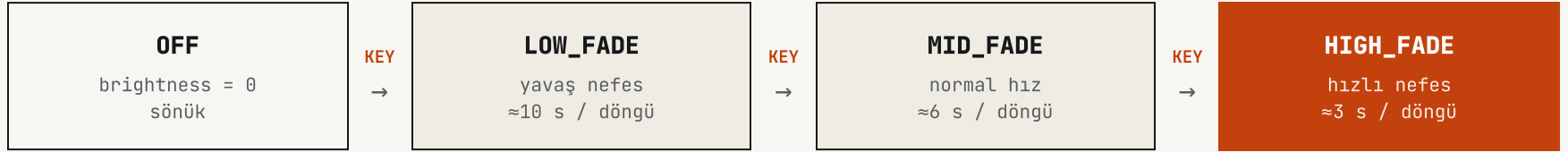
**static** burada zorunlu: fonksiyondan çıkınca değer silinmesin, bir sonraki kesmede hatırlansın. HAL\_GetTick() ise SysTick sayacından açılıştan bu yana geçen milisaniyeyi döndürür.

# 06

## State machine tasarımı

Dört durum, tek değişken, iki switch-case: biri geçişleri, diğeri davranışı yönetir.

## Dört durum, döngüsel geçiş



----- KEY → başa dön (OFF): brightness = 0, fade\_up = 1, fade\_counter = 0 ile garanti sönük başlangıç -----

```
typedef enum {  
    LED_STATE_OFF = 0,  
    LED_STATE_LOW_FADE,  
    LED_STATE_MID_FADE,  
    LED_STATE_HIGH_FADE  
} LedState_t;
```

**Neden enum?** 0/1/2/3 yazmak yerine isim kullanmak kodu okunur yapar, derleyici yazım hatasını yakalar ve switch içinde eksik durum bırakırsanız uyarı verir.

## Durumun PWM'e etkisi

| DURUM               | fade_step_ticks | 1 KADEME SÜRESİ                        | TAM DÖNGÜ (0→100→0)    |
|---------------------|-----------------|----------------------------------------|------------------------|
| LED_STATE_OFF       | —               | —                                      | brightness sabit 0     |
| LED_STATE_LOW_FADE  | 500             | $500 \times 100 \mu s = 50 \text{ ms}$ | $\approx 10 \text{ s}$ |
| LED_STATE_MID_FADE  | 300             | $300 \times 100 \mu s = 30 \text{ ms}$ | $\approx 6 \text{ s}$  |
| LED_STATE_HIGH_FADE | 150             | $150 \times 100 \mu s = 15 \text{ ms}$ | $\approx 3 \text{ s}$  |

**Nasıl çalışıyor?** Her TIM2 kesmesinde fade\_counter bir artar. fade\_step\_ticks değerine ulaşınca brightness bir kademe değişir ve sayaç sıfırlanır.

Küçük değer (150) → brightness sık değişir → hızlı nefes.  
Büyük değer (500) → seyrek değişir → yavaş nefes. Sayı büyüdükçe LED yavaşlar; sezgiye ters gelen kısım budur.

## main.c özeti

```
/* Global degiskenler - hepsi volatile */
volatile LedState_t led_state = LED_STATE_OFF;
volatile uint8_t pwm_phase = 0; // 0-99
volatile uint8_t brightness = 0; // 0-100
volatile uint8_t fade_up = 1; // 1 = artiyor
volatile uint16_t fade_counter = 0;

/* main() icinde */
MX_GPIO_Init();
MX_TIM2_Init();
HAL_TIM_Base_Start_IT(&htim2);
while (1) { } // her sey kesmelerde

/* Buton kesmesi */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    /* debounce 200 ms + switch(led_state) */
}

/* TIM2 kesmesi, 10 kHz */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    /* 1) led_state'e gore fade_step_ticks sec
       2) fade_counter ile brightness'i kaydir
       3) pwm_phase < brightness ise yak, degilse sondur */
}
```

### volatile NEDEN ŞART

Derleyici, bir değişkenin döngü içinde değişmediğini görürse onu register'a alıp bir daha bellekten okumaz. Ama bu değişkenleri **kesme** değiştiriyor; derleyicinin bundan haberi yok.

volatile "bu değişkeni her seferinde bellekten oku" demektir. Unutulursa LED bir durumda takıllı kalır; hata gözle görülür ama kodda görünmez.

---

**while(1) neden boş?** Tüm iş iki kesme fonksiyonunda. Ana döngü boş kalır; ileride güç tasarrufu için buraya \_\_WFI() konabilir.

# Test öncesi kontrol listesi

- ✓ HSI = dahili 16 MHz RC osilatör; PLL kapalıysa SYSCLK = HSI.
- ✓ Kesme frekansı =  $TIM2CLK \div (PSC+1) \div (ARR+1) = 16 \text{ MHz} \div 16 \div 100 = 10 \text{ kHz}$ .
- ✓ APB1 prescaler 1 ise  $TIMxCLK = PCLK1$ ; 1'den büyükse  $TIMxCLK = 2 \times PCLK1$ .
- ✓ PC13 aktif-low: pin LOW = LED yanık, pin HIGH = LED sönmük.
- ✓ Yazılımsal PWM: `pwm_phase < brightness` → pin LOW, yani LED yanık.
- ✓ Update Event, CNT = ARR anında donanım tarafından üretilir ve UIF'i set eder.
- ✓ EXTI bir GPIO pinini kesme kaynağına çevirir; NVIC kesmeyi CPU'ya iletir.
- ✓ Falling edge = HIGH→LOW; KEY basılınca PA0 GND'ye çekildiği için tetiklenir.
- ✓ Pull-up olmadan boştaki giriş pini floating kalır; PA0'a dahili pull-up şart.
- ✓ Debounce = mekanik sıçramayı zaman penceresiyle (200 ms) filtrelemek.
- ✓ volatile: ISR ile paylaşılan değişkende derleyici önbelleklemesini engeller.
- ✓ USER CODE BEGIN/END dışındaki her şey "Generate Code"ta yeniden yazılır.

# 08

## Test: 12 soru

Cevaplar her sorunun hemen altında. Kendini gerçekten sınamak istiyorsan Tweaks panelinden "Cevapları göster"i kapat, sonra tekrar aç.

**SORU 1**

TIM2CLK = 16 MHz, PSC = 15, ARR = 99. TIM2 kesmesi hangi frekansta üretilir?

- A) 1 kHz
- B) 10 kHz
- C) 100 kHz
- D) 160 kHz

**CEVAP · B**

$16.000.000 \div 16 = 1 \text{ MHz}$ ;  $1.000.000 \div 100 = 10 \text{ kHz}$ . Her iki bölmede de "+1" unutulmamalı.

**SORU 2**

PC13 aktif-low bağlı. LED'i yakmak için hangi satır doğrudur?

- A) WritePin(..., GPIO\_PIN\_SET)
- B) WritePin(..., GPIO\_PIN\_RESET)
- C) TogglePin(...)
- D) ReadPin(...)

**CEVAP · B**

RESET pini 0 V'a çeker. LED'in anodu 3.3 V'ta olduğu için akım ancak o zaman geçer.

**SORU 3**

HCLK 16 MHz iken APB1 prescaler'ı 4 yapılsaydı TIM2CLK ne olurdu?

- A) 4 MHz
- B) 8 MHz
- C) 16 MHz
- D) 32 MHz

**CEVAP · B**

$PCLK1 = 16 \div 4 = 4$  MHz. Prescaler 1'den büyük olduğu için  $TIMxCLK = 2 \times PCLK1 = 8$  MHz.

**SORU 4**

PA0'da dahili pull-up etkinleştirilmezse en olası sonuç nedir?

- A) Buton hiç çalışmaz
- B) Rastgele kesmeler oluşur
- C) LED sürekli yanar
- D) TIM2 durur

**CEVAP · B**

Boştaki pin floating kalır, çevresel gürültüyle rastgele HIGH→LOW geçişleri okur; durum kendi kendine değişir.

**SORU 5**

ARR'ı 99'dan 199'a çıkarırsam LED'de ne değişir? Sayılarla açıkla.

**CEVAP**

Kesme frekansı 10 kHz'den 5 kHz'e düşer ( $1 \text{ MHz} \div 200$ ), callback her 200  $\mu\text{s}$ 'de çalışır. pwm\_phase hâlâ 0..99 döndüğü için PWM periyodu 20 ms, frekans 50 Hz olur; titreme sınırına yaklaşılr. Ayrıca tüm fade süreleri iki katına çıkar: HIGH\_FADE  $\approx 3 \text{ s}$  yerine  $\approx 6 \text{ s}$ .

**SORU 6**

Debounce eşiğini 200 ms yerine 20 ms yaparsam ne olur? Ya 2000 ms yaparsam?

**CEVAP**

20 ms: tipik kontak sıçraması 5-20 ms sürer, pencere dar kalır, bazı basışlar iki durum birden atlatır. 2000 ms: sıçrama kesin filtrelendir ama kullanıcı iki saniye içinde ikinci kez basamaz, arayüz takılmış gibi hissettirir. 200 ms ikisinin arasında güvenli bir orta noktadır.

**SORU 7 – bu blok ne yapar?**

```
fade_counter++;  
if (fade_counter ≥ fade_step_ticks) {  
    fade_counter = 0;  
    if (fade_up) { brightness++; }  
    else        { brightness--; }  
    if (brightness == 100) { fade_up = 0; }  
    if (brightness == 0)   { fade_up = 1; }  
}
```

**CEVAP**

Nefes alma efektinin motoru. Sayaç fade\_step\_ticks'e ulaştıkça brightness'ı bir artırır veya azaltır; 100'e varınca yön aşağı, 0'a inince yön yukarı döner. Yani brightness 0 ile 100 arasında üçgen dalga çizer.

**SORU 8 – fark nedir?**

```
HAL_TIM_Base_Start(&htim2);  
/* ...veya... */  
HAL_TIM_Base_Start_IT(&htim2);
```

Projede ikincisi kullanıldı. Birincisi kullanılsaydı LED ne yapardı?

**CEVAP**

Base\_Start sayacı çalıştırır ama kesme üretmez; UIF set olur, NVIC'e gitmez. Callback hiç çağrılmaz, brightness ve pwm\_phase hiç değişmez, LED sürekli sönük kalır. Sayaç arka planda döndüğü için hata "kod hiç çalışmıyor" gibi görünür.

## Aşağıdaki iki kod parçasında birer hata var. Bul.

### SORU 9

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    uint32_t last_tick = 0;
    uint32_t now = HAL_GetTick();

    if ((now - last_tick) < 200) { return; }
    last_tick = now;

    switch (led_state) { /* ... */ }
}
```

#### CEVAP

**static** eksik. last\_tick her çağrıda 0'a döner, koşul asla sağlanmaz, debounce hiç çalışmaz. Ayrıca GPIO\_Pin kontrolü de yok: başka bir EXTI hattı da bu callback'i tetikleyebilir.

### SORU 10

```
uint8_t brightness = 0;
LedState_t led_state = LED_STATE_OFF;

/* ISR bunlari degistiriyor,
diger kod okuyor */
```

Derleyici optimizasyonu (-O2) açıkken ne olur?

#### CEVAP

**volatile** eksik. Derleyici değeri register'da tutup bellekten yeniden okumayabilir; ISR'ın yaptığı değişiklik görünmez olur. Hata çoğu zaman -O0'da ortaya çıkmaz, optimizasyon açılınca belirir.

**SORU 11 – hesap zinciri**

Bölen = PSC + \_\_\_\_

CK\_CNT = 16 MHz ÷ \_\_\_\_ = 1 MHz

Kesme frekansı = 1 MHz ÷ \_\_\_\_ = 10 kHz

PWM frekansı = \_\_\_\_ Hz

**CEVAP**

1 · 16 · 100 · 100

**SORU 12 – kavram eşleştirme**

Pini kesme kaynağına çeviren birim: \_\_\_\_

Kesmeyi CPU'ya ileten denetleyici: \_\_\_\_

CNT = ARR anında üretilen olay: \_\_\_\_

ISR ile paylaşılan değişkenin niteleyicisi: \_\_\_\_

**CEVAP**

EXTI · NVIC · Update Event (UEV) · volatile

# Kartta denemek için üç deney

---

## PSC = 159 yap

CK\_CNT 100 kHz'e, PWM 10 Hz'e düşer.  
LED'in kırpmasını çıplak gözle gör;  
PWM frekansının neden yüksek  
tutulduğunu bir kerede anlarsın.

## volatile'ları kaldır

-02 ile derle. Efektin bozulup  
bozulmadığını izle, sonra assembly  
çıktısında değişkenin bellekten mi  
register'dan mı okunduğuna bak.

## Debounce'ı kapat

200 ms kontrolünü kaldır ve butona  
bas. Tek basışta iki üç durum birden  
atlandığını gör; sıçrama artık soyut  
bir kavram olmaktan çıkar.

# 3

KISIM ÜÇ

## BLDC ve Geri Besleme Teorisi

Motora geçmeden önce iki temel soru: Üç fazı hangi sırayla enerjilemek gerekir (altı adımlı komütasyon) ve rotorun şu an nerede olduğu nasıl bilinir (encoder ve Hall sensörleri). Bu kısım kontrol yazılımının dayandığı fiziği görselleştirir.

---

BU KISIMDA

### 04 Üç Fazlı BLDC Motoru Six-Step Sürme

Görsel anlatım sürümü — sıralı çizim kareleri

### 05 Encoder ve Hall Sensörleri

Pozisyon, hız ölçümü ve rotor sektörü

# Üç Fazlı BLDC Motoru STM32 veya ESP ile Six-Step Sürme

Görsel anlatım sürümü: her kavram 3-4 kareye bölünmüş sıralı çizimlerle.

Bu sürüm, orijinal rehberdeki metni takip eder; farkı, soyut kalan her kavramın bir film şeridine bölünmüş olmasıdır. Kareleri soldan sağa okuyun: her kare bir önceki karede eksik olan tek şeyi ekler.

## PROJE VARSAYIMI

Üç fazlı BLDC motor, güç uçları U-V-W. 60 elektriksel derecelik altı komütasyon durumu. Mikrodenetleyici motora değil, sürücü/inverter kartına bağlanır. Hall'lü ve sensorless olasılıkları ayrı ayrı anlatılır.

## ENERJİ VERMEDEN ÖNCE

Motor ve sürücü modeli doğrulanmadan, giriş doğruluk tablosu okunmadan, dead-time ve fault mekanizması osiloskopa görülmeden güç katına enerji verilmez. İlk denemeler düşük DC gerilimde, akım sınırlamalı kaynakla, gözetim altında yapılır.

## Önce zinciri kur: ne, neyi tetikliyor?

İlk okumada formül ezberlemeyin. Aşağıdaki zincir bu rehberin tamamının iskeletidir. Her bölüm bu zincirin bir halkasını açar.

|                                                                                        |                                                                                   |                                                                                |                                                                                         |                                                                                                  |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>1 · YAZILIM</b><br><b>Mikrodenetleyici PWM üretir</b><br>Karar verir, enerji vermez | <b>2 · ARAYÜZ</b><br><b>Gate driver anahtarları sürer</b><br>Seviye ve kapı akımı | <b>3 · GÜÇ</b><br><b>İnverter U-V-W'yi enerjilendirir</b><br>6 MOSFET, DC-link | <b>4 · SIRA</b><br><b>Komütasyon tablosu 6 durumu değiştirir</b><br>Dönen manyetik alan | <b>5 · GERİ BESLEME</b><br><b>Hall veya Back-EMF geçiş anını söyler</b><br>Doğru anda komütasyon |
|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|

Şekil 0.1 — Rehberin okuma zinciri. Bir halkayı atlarsanız sonraki halka soyut kalır.

## Ben tam olarak ne yapacağım?

Giriş aşamasında en çok karıştıran şey görev sırasıdır. Aşağıdaki akış, kart eline geldiği andan kapalı çevrim hız kontrolüne kadar izlenecek yolu tek bakışta verir. Her adımın ayrıntısı Bölüm 8'de.

| #   | YAPILACAK İŞ                                                                                                             | GEÇİŞ KRİTERİ               |
|-----|--------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| 0   | <b>Donanımı tanı.</b> Motor modeli, kutup çifti, sürücü tipi, giriş modu (3-PWM / 6-PWM / komut) yazılı hale gelir.      | Donanım kimlik kartı dolu   |
| 1   | <b>Pin eşleme tablosunu çıkar.</b> Datasheet'ten sinyal bağlantıları, aktif seviye, enable ve fault.                     | Tablo onaylandı             |
| 2   | <b>Güç katı kapalıyken PWM üret.</b> Üç senkron kanal, complementary çıkış, dead-time ölçümü.                            | Osiloskopta çakışma yok     |
| 3   | <b>Altı durumu enerjisiz test et.</b> Adımları elle ilerletip altı çıkışı logic analyzer ile izle.                       | Her adımda doğru faz çifti  |
| 4   | <b>Geri beslemeyi tek başına doğrula.</b> Hall kodunu elle çevirerek kaydet; sensorless ise BEMF ölçüm devresini incele. | Kod sırası tutarlı          |
| 5   | <b>Düşük enerjili ilk dönüş.</b> Sabitlenmiş motor, düşük gerilim, akım limiti, sınırlı duty.                            | Doğru yönde, adım kaybı yok |
| 6-7 | <b>RPM ölç, rampa ekle, kapalı çevrime geç.</b> Önce P, kararlı olunca I.                                                | Hedef-ölçülen RPM grafiği   |
| 8   | <b>Hata testlerini çalıştır.</b> Enable kesme, driver fault, Hall kopması, stall, düşük bus gerilimi, watchdog.          | Test sonuç tablosu          |

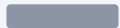
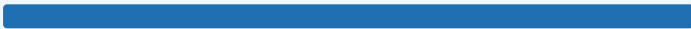
## Hangi kavram ne için var?

Kavramlar tek tek okunduğunda soyut kalır; her birinin bu projede somut bir görevi var.

| KAVRAM                 | BU PROJEDEKİ GÖREVİ                                                                | BÖLÜM |
|------------------------|------------------------------------------------------------------------------------|-------|
| Yarım köprü            | Bir fazı DC+ veya GND'ye bağlayan iki anahtar. Üç tanesi inverteri oluşturur.      | 3     |
| Dead-time              | Aynı koldaki iki anahtarın asla birlikte açılmamasını garanti eder.                | 3     |
| PWM / duty-cycle       | Motora uygulanan ortalama gerilimi, yani "ne kadar" bilgisini ayarlar.             | 3     |
| Altı komütasyon durumu | Hangi faz çiftinin enerjilendiğini, yani "hangi yöne" bilgisini taşır.             | 4     |
| Elektriksel açı        | Manyetik alanın açısı. Kutup çifti sayısı kadar hızlı döner; mekanik açı değildir. | 4     |
| Hall / Back-EMF        | Komütasyonun "ne zaman" değişeceğini söyler. Biri sensörle, biri ölçümle.          | 5     |
| Timer, preload, COM    | Altı çıkışın aynı anda ve glitch'siz değişmesini sağlar.                           | 6     |
| Timer tetiklemeli ADC  | Akımı ve BEMF'i gürültünün en az olduğu anda ölçer.                                | 6     |
| Durum makinesi         | Motorun hangi aşamada olduğunu ve fault'ta ne yapacağını tanımlar.                 | 7     |
| PI kontrolcü           | Yük değişse de ölçülen RPM'i hedefte tutar.                                        | 7     |
| Break / fault girişi   | Yazılımı beklemeden PWM'i donanımsal olarak kapatır.                               | 6, 10 |

#### ÇİZİMLERİN RENK KODU

Rehber boyunca **fazın rengi sabittir**, fazın o anki **durumu ise dolgu biçimiyle** gösterilir. Bir çizimi okurken önce rengi (hangi faz), sonra dolguyu (ne yapıyor) bakın.

|          |                                                                                     |                  |                                                                                       |                                            |
|----------|-------------------------------------------------------------------------------------|------------------|---------------------------------------------------------------------------------------|--------------------------------------------|
| <b>U</b> |  | U fazı · turuncu |  | dolu = kaynak, DC+ tarafı, PWM ile sürülür |
| <b>V</b> |  | V fazı · mavi    |  | çizgili = dönüş yolu, GND tarafı açık      |
| <b>W</b> |  | W fazı · yeşil   |  | kesikli = boş faz, iki anahtar da kapalı   |

# 1 · Büyük resim: sizden ne isteniyor?

Tek cümle: mikrodenetleyicinin timer ve geri besleme birimlerini kullanarak sürücüye uygun PWM ve komütasyon sinyalleri üretmek; motoru başlatmak, hızını ve yönünü kontrol etmek, hatada güvenli durdurmak.

## Motor neden doğrudan STM32 veya ESP'ye bağlanamaz?

Mikrodenetleyici pini 3.3 V mantık seviyesinde ve miliamper mertebesinde akım verir. Motor ise amperler ister. Aşağıdaki üç kare bu boşluğun nasıl kapatıldığını gösterir.



Şekil 1.1 — Kontrol yolu (küçük sinyaller) ile güç yolunun (motor akımı) ayrılması. U-V-W hatları MCU'ya değil inverter çıkışına bağlanır.

## TEMEL ZİHİNSEL MODEL

MCU bir orkestra şefidir: hangi anahtar, hangi sırada, ne kadar süre. Sesi çıkaran orkestra ise DC kaynak ve inverter'dır.

## Elinizdeki "motor sürücü" hangisi? Six-step'i kim yapıyor?

Bu sorunun cevabı yazılımınızın ne kadar iş yapacağını belirler. Dört olasılık var.

| Hazır ESC                                                        | Entegre 3 faz inverter                                       | Yalnız gate driver                                                 | Akıllı motor sürücü                              |
|------------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------|
| <b>SİZDEN İSTER</b><br>Throttle PWM, DShot, CAN veya UART komutu | <b>SİZDEN İSTER</b><br>3 veya 6 PWM, enable, bazen SPI ayarı | <b>SİZDEN İSTER</b><br>3 veya 6 PWM; harici MOSFET köprüsü gerekir | <b>SİZDEN İSTER</b><br>Hız, yön veya tork komutu |
| <b>Six-step: ESC içinde</b>                                      | <b>Six-step: sizin yazılımınız</b>                           | <b>Six-step: sizin yazılımınız + güç kartı</b>                     | <b>Six-step: sürücü entegrasyonu</b>             |

Şekil 1.2 — Turuncu kartlarda komütasyonu siz yazarsınız; gri kartlarda komut gönderirsiniz.

#### **KART GELDİĞİNDE İLK BAKIŞ**

Pin adlarında UH/UL, VH/VL, WH/WL veya INHA/INLA görüyorsanız kart doğrudan komütasyon bekleyen bir inverter olabilir. Yalnızca PWM, DIR, EN varsa komütasyon sürücünün içindedir. Kesin karar yalnızca datasheet ve kart şemasıyla verilir.

## 2 · Sıfırdan elektrik ve motor temelleri

Dört büyüklük, tek bir soru için: motora ne kadar enerji gidiyor ve nereye ısı olarak dönüyor?

**V** volt

Akımı hareket ettiren potansiyel farkı. PWM ile motora giden *ortalama* gerilim değiştirilir.

**I** amper

Sargıdan geçen yük akışı. BLDC'de **tork ile** doğrudan ilişkilidir.

**R** ohm

Akıma karşı koyar,  $I^2R$  kaybıyla ısı üretir.

**P** watt

Enerji aktarım hızı. DC-link kondansatörü enerji **depolar**; güç kesilse bile tehlikelidir.

### SAYISAL ÖRNEK

$$V = I \cdot R$$

$$P = I^2 \cdot R$$

Sargı yolunun eşdeğer direnci **0.5  $\Omega$**  ve üzerinden **4 A** geçiyorsa yalnız dirençten kaynaklı kayıp:

$$P = 4^2 \cdot 0.5 = 8 \text{ W}$$

8 W küçük bir motorda ciddi bir ısıtıcıdır. Kilitli veya yanlış komütasyonlu motorun neden saniyeler içinde ısındığı budur.

### DC kaynaktan dönen manyetik alan nasıl çıkar?

Besleme DC'dir, yani yönü değişmez. Dönen alanı üreten şey inverter'ın anahtarlama sırasıdır.

#### KARE 1

##### DC BESLEME

sabit gerilim,  
sabit yön

Elimizde tek yönlü bir DC bara var. Bu haliyle hiçbir şey dönmez.

#### KARE 2



akım: U → V

Inverter iki fazı seçer: biri kaynak, biri dönüş. Sargılarda belirli yönde akım akar.

#### KARE 3



İki sargının alanı toplanır ve belirli bir yöne bakan tek bir stator alanı oluşur.

#### KARE 4



alan 60° ilerledi

Bir sonraki duruma geçilince alan 60° atlar. Mıknatıslı rotor bu alanı kovalar: dönme budur.

Şekil 2.1 — DC beslemeden dönen alana. Çizim, motor geometrisinin sadeleştirilmiş gösterimidir.

## BLDC adındaki "DC" neden kafa karıştırır?



FIRÇALI DC

Akım yönünü **mekanik** fırçalar değiştirir. Motora sabit DC verilir, gerisini metal temaslar halleder. Aşınır, kıvılcım çıkarır.



BLDC

Aynı işi **elektronik** yapar: fırçanın görevi altı MOSFET ve sizin komütasyon tablonuza geçmiştir. Kaynak DC'dir, sargı gerilimi değil.

Şekil 2.2 — Aynı fikir, iki farklı komütasyon teknolojisi. "Yazılım fırçadır" cümlesi bu projeyi özetler.

### IMU NEDEN GEREKLİ DEĞİL?

IMU gövdenin doğrusal ivmesini ve açısal hızını ölçer. Six-step'in ihtiyacı ise şu tek bilgidir: *altı elektriksel durumdan hangisi uygulanmalı?* Bunu Hall sensörü, encoder, Back-EMF veya sürücünün kendi algılama devresi söyler. IMU bu iş için standart geri besleme elemanı değildir.

### 3 · Üç faz inverter ve motor sürücü

Tüm güç katı tek bir yapının üç kopyasıdır: yarım köprü. Önce onu tam anlayın, inverter kendiliğinden anlaşılır.

#### Yarım köprünün dört hali

Bir fazı DC+ veya GND'ye bağlamak için iki anahtar vardır: high-side ve low-side. Dört kombinasyonun üçü kullanılır, biri yasaktır.



Şekil 3.1 — U fazının yarım köprüsü. V ve W için aynı yapı tekrarlanır; üçü birlikte altı MOSFET'li inverter'ı oluşturur.



## Gate driver ne yapar, motor driver'dan farkı ne?

### GATE DRIVER'IN DÖRT GÖREVİ

- 1 MOSFET kapılarını yeterli akımla hızlı açıp kapatmak
- 2 High-side MOSFET için gereken yüksek kapı gerilimini üretmek
- 3 Bazı modellerde dead-time, düşük gerilim, aşırı akım ve sıcaklık koruması
- 4 Fault bilgisini mikrodenetleyiciye bildirmek

### İKİSİNİ KARIŞTIRMAYIN

**Gate driver** yalnız kapıları sürer; güç MOSFET'leri harici olabilir, yani güç kartı da sizin işiniz olur.

**Motor driver kartı** gate driver, MOSFET'ler, akım ölçümü ve korumayı birlikte barındırabilir. Kart şeması görülmeden bu iki terim birbirinin yerine kullanılmaz.

## Dead-time: yasak durumu imkânsız kılmak

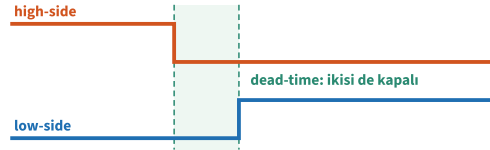
MOSFET anında kapanmaz. Üst anahtara "kapan" dedikten sonra gerçekten kapanması zaman alır. Bu yüzden alt anahtar hemen açılmaz.

### KARE 1 · DEAD-TIME YOK



Alt anahtar, üst anahtar tam kapanmadan açılırsa kısa devre akımı akar.

### KARE 2 · DEAD-TIME VAR



Araya kasıtlı bir boşluk konur. Bu sürede faz boştaadır, akım diyottan devam eder.

### KARE 3 · KİM EKLİYOR?



İkisi birlikte eklerse toplam süre beklediğinizden uzun olur: tork düşer, akım dalgalanır.

Dead-time tahminle seçilmez: MOSFET ve driver gecikmeleri, propagation delay farkı ve üretici önerisiyle belirlenir.

Şekil 3.3 — Tamamlayıcı (complementary) PWM sinyalleri arasına eklenen dead-time. Aşama 2'de bu boşluğu osiloskopa göreceksiniz.

## PWM ve duty-cycle: "ne kadar" ayarı

Anahtar ya tam açık ya tam kapalıdır; arada bir değer yoktur. Ortalama gerilim, açık kalma *oranıyla* ayarlanır.

### %25 duty



Kesikli çizgi ortalama gerilim. Yavaş dönüş, düşük tork.

### %50 duty



Frekans aynı, yalnız açık kalma süresi uzadı.

### %75 duty



Ortalama gerilim yükseldi; hız *artma eğiliminde*, garanti değil.

Şekil 3.4 — Aynı frekansta üç duty-cycle değeri.

#### SAYISAL ÖRNEK

$$D = t_{on} / T$$

$$V_{ort} \approx D \cdot V_{bus}$$

24 V bus, 20 kHz PWM ( $T = 50 \mu s$ ),  $t_{on} = 20 \mu s$  ise  $D = 0.40$  ve  $V_{ort} \approx 9.6 V$ .

Buradan RPM *çıkamaz*: gerçek hız yük, motor sabitleri, sürtünme ve besleme çökmesine bağlıdır. Bu yüzden Bölüm 7'de hız ölçülüp geri beslenir.

## 4 · Six-step: 60 derecelik altı durum

Bu bölümdeki 60 derece, motorun gövdesini altı dilime bölmek değildir. Manyetik alanın bir elektriksel turundaki sektördür. Karışıklığın kaynağı burasıdır.

### Elektriksel açı ile mekanik açı aynı şey değil

Kutup çifti sayısı  $p$ , bu iki açıyı birbirinden ayıran çarpandır. Dört kare bu ilişkiyi kurar.



Şekil 4.1 — Elektriksel açı ile mekanik açının ilişkisi (örnek:  $p = 7$ ).

#### SAYISAL ÖRNEK

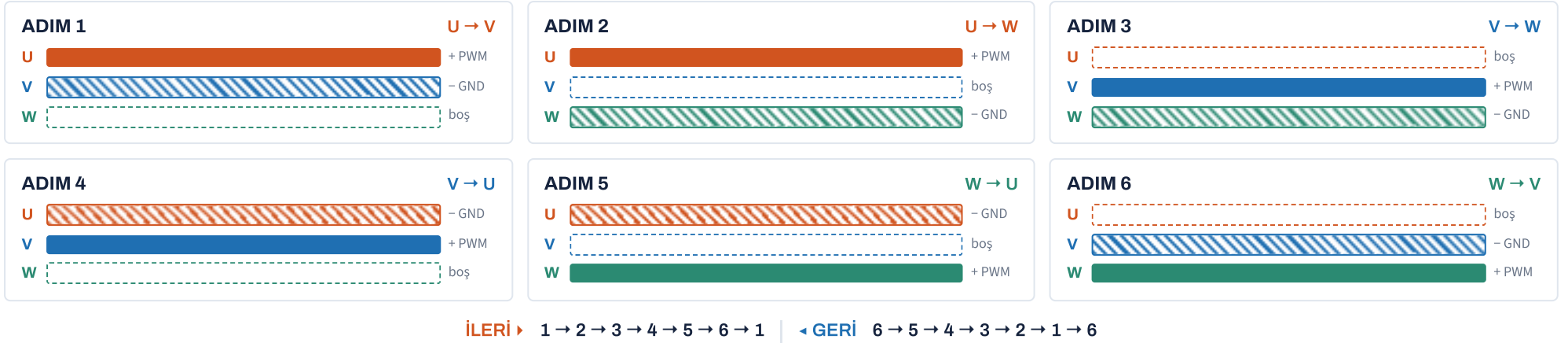
$$\theta_e = p \cdot \theta_m$$

$p = 7$  için mil  $1°$  dönerse alan  $7°$  elektriksel ilerler. Ters yönden bakınca: bir  $60°$  elektriksel sektör, mekanikte  $60/7 \approx 8.57°$ 'dir.

Yazılımda kontrol edilen şey milin açısı değil, bu **altı elektriksel durumdur**. "Rotor" kelimesi yalnız bu durumların doğru anda değişmesi için gereken fiziksel konumu tarif ederken kullanılır.

### Altı durum: her adımda hangi faz ne yapıyor?

Kural her adımda aynı: **bir faz pozitif, bir faz negatif, üçüncü faz boşta**. Altı adım bu üç rolün altı farklı dağılımıdır. Renkler fazı, dolgu biçimi rolü gösterir.

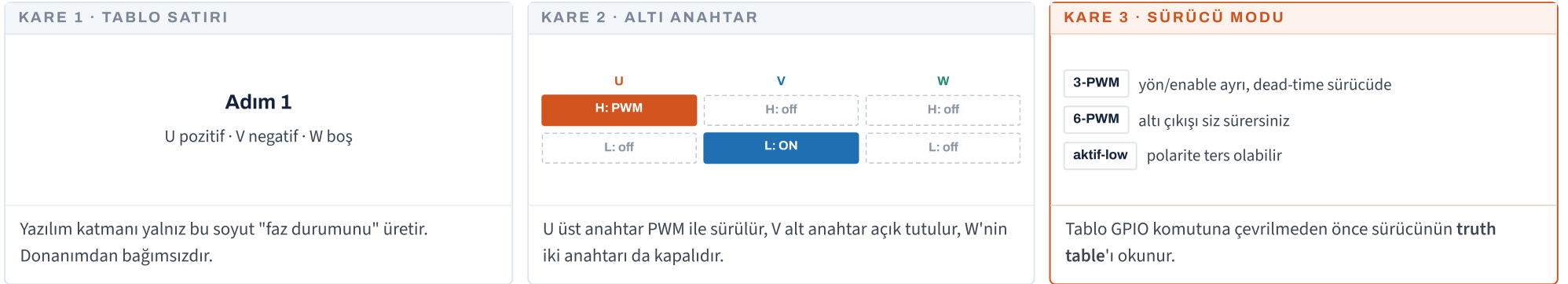


Şekil 4.2 — Altı komütasyon durumu ve çevrim yönü. Sıra tersine çevrilirse motor ters döner. Gerçek faz eşlemesi donanıma göre doğrulanmalıdır.

#### DİKKAT: SIRALAMA HATASININ BEDELİ

Faz isimleri veya sensör eşlemesi yanlışsa motor dönmez; titrer, ısınır ve aşırı akım çeker. Bu yüzden Aşama 3'te tablo *enerji verilmeden* logic analyzer ile doğrulanır.

### Bir durumu inverter anahtarlarına çevirmek



Şekil 4.3 — Soyut faz durumundan gerçek pin sinyaline üç adımda geçiş.

#### GPIO İLE RASTGELE ANAHTARLAMA YAPMAYIN

High-side ve low-side pinlerini sıradan GPIO yazmalarıyla farklı zamanlarda değiştirmek geçici shoot-through üretir. Motor kontrol timer'ının complementary PWM, preload/COM ve break özellikleri kullanılır. Tüm çıkışların güvenli başlangıç durumu, MCU reset olurken de tanımlı olmalıdır.

### Yön, hız ve tork üç farklı şeydir

#### YÖN

Komütasyon **sırası** belirler: 1→2→3 ileri, 6→5→4 geri.  
Duty ile ilgisi yoktur.

#### HIZ

Komütasyon **sıklığı** ve ortalama gerilimle ilişkilidir. Kesin hız için ölçülen RPM geri beslenir.

#### TORK

Temel olarak **faz akımıyla** ilişkilidir. Kilitli motor yüksek akım çeker ama dönmez.

Sadece duty vererek sabit hız elde edilmez: yük değişince hız da değişir. Sabit RPM istiyorsanız hızı ölçmek ve duty'yi kontrolcüye ayarlatmak zorunludur.

## 5 · Doğru komütasyon anı: Hall veya Back-EMF

Altı durumu biliyorsunuz. Kalan tek soru: bir sonraki duruma *ne zaman* geçilecek? Cevabı geri besleme verir ve hangi geri beslemeyi kullanacağınız motorun kablo sayısına bakılarak anlaşılır.

### 3 kalın kablo + ince sensör kabloları

KOLAY YOL

**Hall tabanlı six-step.** Rotorun sektörü dururken bile bilinir, motor doğru adımdan başlar. İlk çalışan sistemi kurmanın en kısa yolu budur.

### Yalnızca 3 kalın U-V-W kablosu

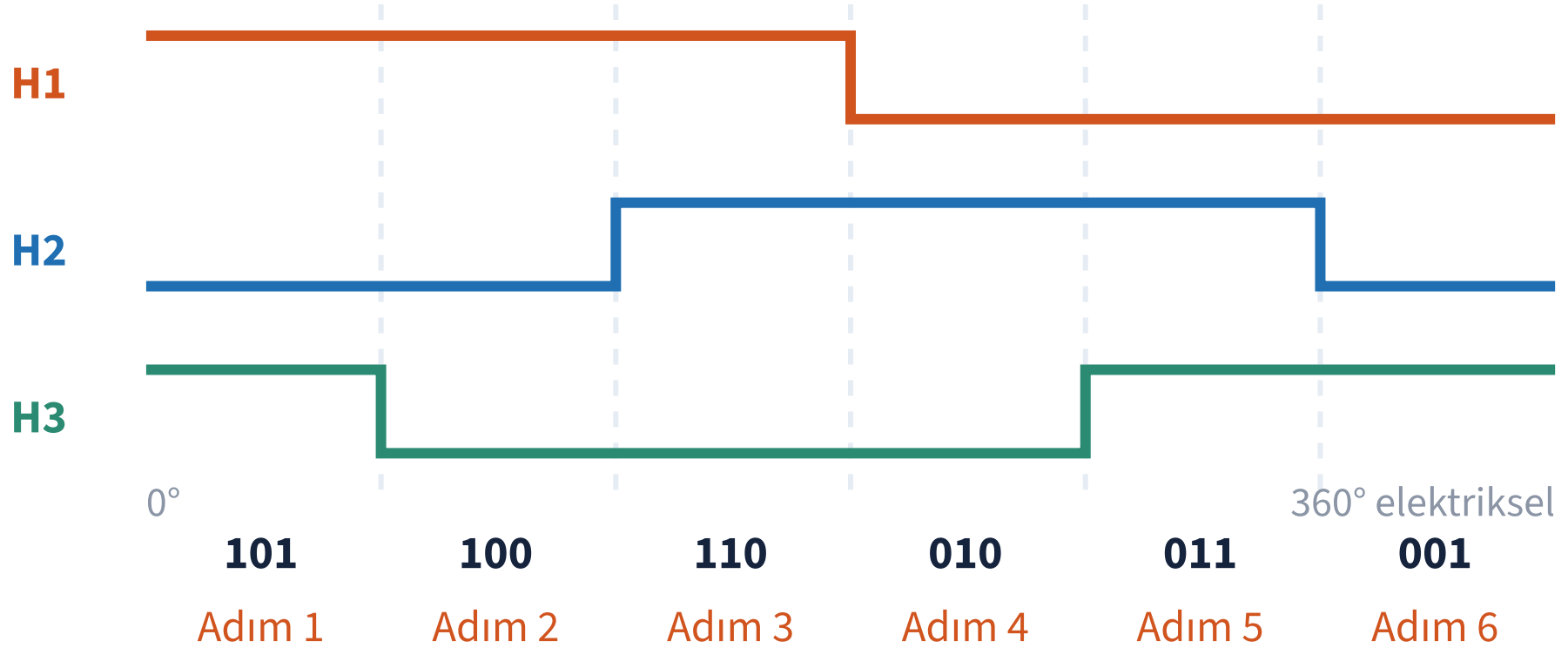
ZOR YOL

**Sensorless Back-EMF** veya sürücü içi algılama. Konum ölçümle tahmin edilir; motor dururken bu bilgi yoktur, bu yüzden ayrı bir başlatma dizisi gerekir.

Şekil 5.1 — Geri besleme yöntemini belirleyen ilk fiziksel kontrol: kabloları sayın. Hall sensörü bir IMU değildir; motor içinde manyetik alana göre dijital 0/1 üreten üç elemandır.

### Hall: üç dijital hat, altı geçerli kod

Üç Hall hattı birlikte üç bitlik bir sayı oluşturur. Sekiz kombinasyondan altısı geçerlidir; 000 ve 111 arıza demektir. Her 60 elektriksel derecede kod değişir ve bu değişim komütasyon komutudur.



Şekil 5.2 — Örnek Hall dalga şekilleri: üç hat 120° kaydırılmış, her 60°'de tek bir bit değişir. Kod sırası ve adım eşlemesi evrensel değildir; sensör yerleşimine göre değişir.

#### HALL TABLOSU İNTERNETTEN ALINMAZ

Hall kablo sırası, faz kablo sırası ve sensörlerin fiziksel yerleşimi üreticiye göre değişir. Gerçek eşleme motor datasheet'inden, çalışan bir karttan veya **düşük enerjili ölçümden** çıkarılır. Yazılımda bu eşleme sekiz elemanlı bir tabloya konur; 000 ve 111 girişleri "geçersiz" olarak işaretlenir.

#### HALL GEÇİŞİNDEN HIZ · SAYISAL ÖRNEK

$$\text{RPM} = 60 / (6 \cdot p \cdot \Delta t)$$

Bir elektriksel turda 6 sektör, bir mekanik turda p elektriksel tur vardır. p = 7 ve iki sektör olayı arası  $\Delta t = 1 \text{ ms}$  ise  $\text{RPM} = 60 / (6 \cdot 7 \cdot 0.001) \approx 1429$ .

Tek periyot gürültülü hız üretir; son birkaç geçişin hareketli ortalaması alınır. Filtre ağır olursa hız kontrolü geç tepki verir.

## Sensorless: boştaki fazı dinlemek

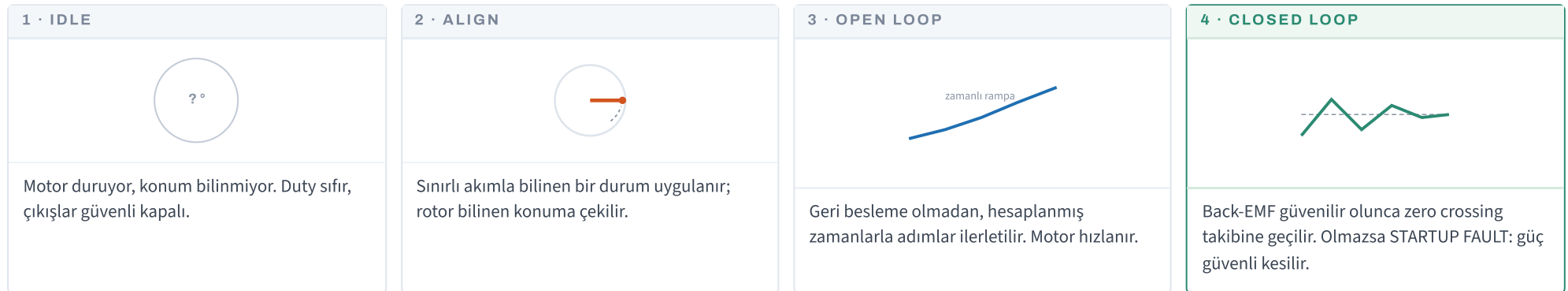
Hall yoksa rotorun yerini motorun kendisi söyler. Mıknatıslar dönerken sargılarda gerilim indükler; buna Back-EMF denir. Six-step'in güzelliği şudur: her adımda bir faz zaten boşta, yani ölçüme hazırdır.



Şekil 5.3 — Back-EMF ile komütasyon anının bulunması. Anahtarlama sonrasındaki güvenilmez demanyetizasyon bölgesi maskelenir. Dalga şekli öğretim amacıyla sadeleştirilmiştir; ST'nin resmi six-step açıklaması da aynı üç adımı tanımlar.

## Sensorless motor dururken kendini neden bulamaz?

Back-EMF genliği hızla artar; motor duruyorsa ölçülecek bir şey yoktur. Bu yüzden sensorless başlatma dört ayrı aşamadır ve her aşamada yazılım farklı davranır.



Şekil 5.4 — Sensorless başlatma akışı. Rampa çok hızlıysa rotor alanı takip edemez ve senkron kaybolur; çok yavaş veya duty çok düşükse yük aşılamaz; duty çok yüksekse aşırı akım ve mekanik sıçrama olur. Bu yüzden başlangıç parametreleri akım sınırı altında ayarlanır.

#### BU BÖLÜMÜN SONUCU

Ek Hall kabloları varsa Hall tabanlı six-step daha doğrudan başlar. Yalnız U-V-W varsa Back-EMF algılama veya sürücünün kendi sensorless özelliği araştırılır. Hangi yolun kullanılacağı motor ve sürücü modeli görülmeden kesinleştirilemez.

## 6 · Mikrodenetleyici çevre birimleri

Six-step yazılımının kalbi ana döngü değil, timer'dır. Bu bölümdeki her özellik tek bir amaca hizmet eder: altı çıkışın doğru anda, birlikte ve güvenli değişmesi.

Seçim hakkı varsa ve amaç algoritmayı kendiniz yazmaksa STM32 genellikle daha öğretici bir yol sunar: motor kontrole uygun ailelerde gelişmiş timer, hızlı ADC, comparator, op-amp ve donanımsal fault bağlantıları birlikte bulunur. ESP32 de MCPWM sayesinde complementary PWM, dead-time, capture ve fault/brake işlevlerini yapabilir; ekip ESP-IDF kullanıyorsa veya kablosuz telemetri önemliyse geçerli bir seçimdir. Karar "hangisi daha güçlü" sorusuyla değil, üç maddeyle verilir: **(1)** verilen sürücü kartıyla pin ve gerilim uyumu, **(2)** ekibin yazılım ekosistemi, **(3)** gereken timer/ADC/capture/fault özelliklerinin seçilen tam modelde bulunması.

| BAŞLIK               | STM32 YAKLAŞIMI                                     | ESP32 YAKLAŞIMI                                |
|----------------------|-----------------------------------------------------|------------------------------------------------|
| PWM birimi           | TIM1/TIM8 gibi advanced-control timer'lar           | MCPWM timer/operator/generator yapısı          |
| Complementary PWM    | CHx ve CHxN çıkışları                               | İki generator ve dead-time modülü              |
| Hall ölçümü          | Timer Hall/encoder modu veya input capture / EXTI   | MCPWM capture veya GPIO interrupt              |
| Sensorless ölçüm     | Timer tetiklemeli ADC, analog watchdog / comparator | ADC/comparator imkânı modele ve tasarıma bağlı |
| Fault                | BKIN/BRK ile donanımsal timer kapatma               | MCPWM fault/brake işlemleri                    |
| Hazır motor araçları | X-CUBE-MCSDK ve Motor Control Workbench             | ESP-IDF örnekleri ve özel uygulama kodu        |

Tablo 6.1 — Kavramsal karşılaştırma. Özellikler tam çip modelinden doğrulanmalıdır.

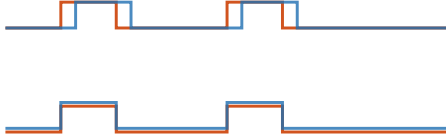
### Timer'da öğrenilecek dört özellik

Aşağıdaki dört kare, altı çıkışın neden "sıradan bir PWM" gibi ele alınamayacağını gösterir.

#### A · TEK ZAMAN TABANI

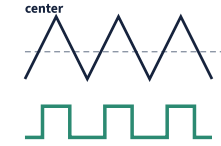
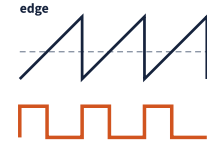
AYRI TIMER'LAR → KAYMA

TEK TIMER → HİZALİ



U, V ve W aynı timer'dan üretilir. Ayır, senkron olmayan timer'lar geçiş anlarını belirsizleştirir. STM32'de tipik eşleme TIM1\_CH1/CH1N = U kolu, CH2/CH2N = V, CH3/CH3N = W (yalnız örnek; gerçek pinler alternate function tablosundan seçilir).

#### B · EDGE vs CENTER ALIGNED



Edge-aligned'da sayaç tek yönde ilerler, darbe periyodun başına yapışır. Center-aligned'da sayaç yukarı-aşağı sayar, darbe ortalanır; akım ve Back-EMF ölçümü için sakın noktalar daha düzenli oluşur.

#### C · PRELOAD VE COM

PERİYOT ORTASINDA YAZMA



Duty veya komütasyon deseni periyodun ortasında değişirse kısa ya da uzamış darbe çıkar. Preload yeni değerleri gölge register'a yazar, tanımlı olayda hepsini **birlikte** uygular. Altı çıkışın atomik değişmesi böyle sağlanır.

#### D · BREAK / FAULT GİRİŞİ



Aşırı akım fault'u yazılım interrupt'ını *beklemez*. Çıkışlar donanımda güvenli seviyeye çekilir; CPU sonra sebebi kaydeder ve güvenli duruma geçer.

Şekil 6.1 — Motor kontrol timer'ının dört kritik özelliği. Register isimleri: PSC prescaler, ARR periyot sınırı, CCR1/2/3 kanal karşılaştırma (duty) değerleri. Ayrıntı seriye göre değişir; reference manual esastır.

#### SAYISAL ÖRNEK · CENTER-ALIGNED

$$f_{PWM} = f_{TIM} / [2(PSC+1)(ARR+1)]$$

$f_{TIM} = 170 \text{ MHz}$ ,  $PSC = 0$ ,  $ARR = 4249$  için:

$$f_{PWM} = 170.000.000 / (2 \cdot 1 \cdot 4250) = 20 \text{ kHz}$$

$ARR = 4249$  aynı zamanda duty çözünürlüğünüzdür: CCR değerini 0-4249 arasında ayarlarsınız, yani yaklaşık %0.024 adımlarla.

#### ADC: ölçümü doğru *ana* yerleştirmek

Ölçüm hatasının çoğu yanlış ADC ayarından değil, yanlış **zamandan** gelir. MOSFET anahtarlarken faz ve shunt sinyalleri yüksek frekanslı gürültüyle dolar.

### KARE 1 · YANLIŞ AN



Ana döngüde rastgele okumak, çoğu zaman gürültünün tepesini örneklemek demektir.

### KARE 2 · OTURMAYI BEKLE



Geçişten sonra sinyalin oturması için kısa ve *bilinen* bir süre beklenir.

### KARE 3 · TIMER TETİKLER



Timer compare olayı ADC'yi tetikler; ölçüm her periyotta aynı, tekrarlanabilir noktada alınır.

Şekil 6.2 — Akış: timer periyodu başlar → oturma beklenir → compare olayı ADC'yi tetikler → tamamlanma olayı ölçümü hızlı kontrol döngüsüne verir → yazılım ölçek, offset ve sınır kontrolünü uygular.

### SAYISAL ÖRNEK · SHUNT AKIMI

$$V_{ADC} = ADC_{raw} / (2^N - 1) \cdot V_{ref}$$
$$I = (V_{ADC} - V_{offset}) / (G \cdot R_{shunt})$$

12-bit ADC ( $N = 12$ ),  $V_{ref} = 3.3$  V, okunan değer 2300 ise  $V_{ADC} \approx 1.852$  V.

$V_{offset} = 1.65$  V, kazanç  $G = 20$ ,  $R_{shunt} = 0.01$   $\Omega$  için:

$$I = (1.852 - 1.65) / (20 \cdot 0.01) \approx 1.01 \text{ A}$$

Gerçek tasarımda tolerans, offset kayması, ADC referansı, doyumluk ve ortak-mod sınırları hesaba katılır.

## Capture ve sayaç taşması

Hall geçişleri veya Back-EMF olayları arasındaki süre timer capture değeriyle ölçülür. Tek tuzak sayaç sınırına gelip başa dönmesidir.

### NORMAL DURUM



İki olay aynı sayım turundayrsa fark doğrudan alınır.

### TAŞMA OLDUĞUNDA



Yeni değer eskiden küçükse fark negatif görünür. Doğru süre: (periyot - eski) + yeni + 1. Bu kontrol yapılmazsa RPM aniden sıçrar.

Interrupt içinde uzun işlem yapılmaz: olay zamanı kaydedilir, gerekli kısa komütasyon işi yapılır, ağır telemetri ana döngüye bırakılır.

## 7 · Yazılım mimarisi ve durum makinesi

Motor kontrol yazılımında işler farklı hızlarda koşar. Hepsini tek bir sonsuz döngüye koymak, en hızlı işi en yavaş işin insafına bırakmak demektir.

### AYNI PROJEDE BEŞ FARKLI ZAMAN ÖLÇEĞİ

|                    |                                                                                    |                               |
|--------------------|------------------------------------------------------------------------------------|-------------------------------|
| PWM ve hızlı ölçüm |   | mikrosaniye · her periyot     |
| Komütasyon         |   | olay tabanlı · hıza bağlı     |
| Hız kontrolü (PI)  |  | sabit periyot · milisaniye    |
| Telemetri ve komut |  | yavaş · ana döngü             |
| Fault              |   | anında ve en yüksek öncelikli |

Şekil 7.1 — Bir işi yavaş bir işin arkasına koymak, motorun senkron kaybetmesinin en yaygın yazılım sebebidir.

### KATMANLAR: HER KATMAN ALTINDAKİNİ KULLANIR, ÜSTÜNDEKİNİ BİLMEZ

#### Uygulama katmanı

start/stop, hedef RPM, yön komutu

#### Durum makinesi

hangi aşamadayız, fault var mı

#### Kontrol katmanı

rampa ve hız PI

#### Komütasyon katmanı

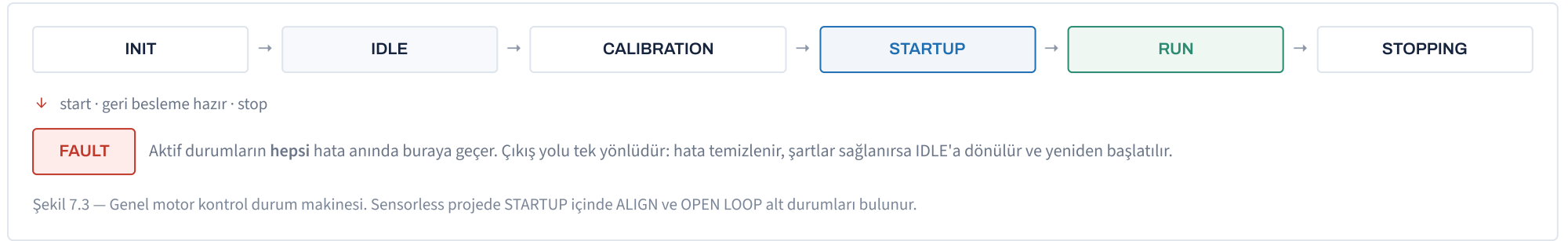
six-step tablosu · donanımdan bağımsız faz durumu

#### Donanım katmanı

timer, ADC, GPIO · sürücüye özel eşleme

Şekil 7.2 — Komütasyon tablosu saf "faz durumu" üretir (kaynak / dönüş / boş); onu gerçek timer çıkışlarına çevirmek yalnız donanım katmanının işidir. Bu ayırım sayesinde tablo ve durum makinesi bilgisayarda test edilebilir.

## Motor durum makinesi



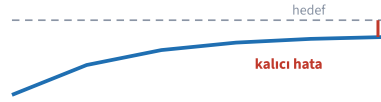
| DURUM       | O DURUMDA YAPILAN İŞLER                                                                                  |
|-------------|----------------------------------------------------------------------------------------------------------|
| INIT        | Clock, GPIO, timer, ADC ve sürücü arayüzü hazırlanır. PWM çıkışları güvenli kapalıdır.                   |
| IDLE        | Motor enerjisizdir, duty sıfırdır. Komut beklenir.                                                       |
| CALIBRATION | Akım sensörü offset'i, bus gerilimi ve girişlerin geçerliliği kontrol edilir.                            |
| STARTUP     | Hall varsa mevcut koddan ilk sektör seçilir; sensorless ise hizalama ve açık çevrim hızlandırma yapılır. |
| RUN         | Komütasyon geri beslemeyle sürdürülür; hız kontrolü ve sınırlar çalışır.                                 |
| STOPPING    | Duty kontrollü azaltılır veya uygulamanın gerektirdiği frenleme uygulanır.                               |
| FAULT       | PWM donanımsal kapatılır, fault sebebi kaydedilir, güvenli yeniden başlatma şartı beklenir.              |

Tablo 7.1 — Durumların sorumlulukları. Motorun her an tek bir durumda olması, hata ayıklamayı ölçülebilir hale getirir.

## Hız kontrolü: P ve I'yı sezgisel anlamak

Hata, hedef RPM ile ölçülen RPM'in farkıdır. PI kontrolcü bu hatadan bir duty komutu üretir. Üç kare, iki terimin ne yaptığını gösterir.

#### SADECE P · KÜÇÜK $K_p$



Yavaş yaklaşır ve hedefe tam oturmaz. Hata küçük kaldığı için düzeltme de küçüktür.

#### SADECE P · ÇOK BÜYÜK $K_p$



Sert tepki verir, hedefi aşar, salınır. Motorda ses ve akım dalgalanması duyulur.

#### P + I · DENGELİ



I terimi uzun süre kalan hatayı biriktirip kapatır. Duty sınırdayken birikmeyi durdurmak için **anti-windup** gerekir.

Şekil 7.4 — Aynı hedef, üç farklı ayar. İlk motor döndürmede PI zorunlu değildir: önce sabit ve sınırlı duty ile doğru komütasyon doğrulanır, RPM ölçümü güvenilir olduktan sonra kapalı çevrim eklenir.

#### SAYISAL ÖRNEK · PI

$$e = \text{RPM}_{\text{hedef}} - \text{RPM}_{\text{ölçülen}}$$

$$u = K_p \cdot e + K_i \cdot \int e \, dt$$

Hedef 1500, ölçülen 1400 RPM ise  $e = 100$ .  $K_p = 0.0002$  (duty/RPM) için P katkısı 0.02, yani duty +%2.

Çıkış her zaman sınırlanır:  $D_{\min} \leq D \leq D_{\max}$ . Duty sınırı dayandığında integral biriktirmeyi durdurun; aksi halde hata kapandıktan sonra motor hedefi uzun süre aşar.

#### INTERRUPT İÇİNDE YAPILMAYACAKLAR

- HAL\_Delay veya blocking bekleme
- Uzun printf, blocking UART aktarımı
- Fault çıkışını kapatıp uzun hesap yaptıktan sonra PWM durdurmak
- Belirsiz süreli döngüler
- Dinamik bellek ayırma
- Öncelik sırası belirsiz iç içe kesme kurgusu

ISR olayı zaman damgasıyla kaydeder, kısa ve deterministik işi yapar, gerisini bayrak veya kuyrukla ana göreve bırakır.

## 8 · Adım adım gerçekleştirme planı

Her aşamanın bir amacı ve bir geçiş kriteri vardır. Kriter sağlanmadan sonraki aşamaya geçilmez; hataların çoğu atlanan aşamadan gelir.

### Aşama 0 · Donanımı tanıma

Motoru döndürmeye çalışmadan önce bu kartı doldurun. Boş kalan her satır, sonraki aşamalarda bir hata olarak geri döner.

| SORULACAK BİLGİ                          | BULUNAN DEĞER / KAYNAK |
|------------------------------------------|------------------------|
| Motor tam model numarası                 |                        |
| Nominal ve maksimum gerilim              |                        |
| Sürekli ve tepe akımı                    |                        |
| Kutup çifti sayısı (p)                   |                        |
| Hall kabloları var mı?                   |                        |
| Sürücü kartı ve entegresi                |                        |
| Sürücü giriş modu: 3-PWM / 6-PWM / komut |                        |
| Logic gerilimi ve aktif seviye           |                        |
| Dead-time'ı kim ekliyor?                 |                        |
| Akım, BEMF ve fault çıkışları            |                        |
| STM32 veya ESP tam modeli                |                        |

Tablo 8.1 — Enerji vermeden önce doldurulacak donanım kimlik kartı.

## Aşama 1 · Datasheet'ten bağlantı tablosu

| İŞLEV          | SÜRÜCÜ PİNİ | MCU PİNİ | KONTROL NOTU                        |
|----------------|-------------|----------|-------------------------------------|
| U high / PWM A |             |          | Timer alternate function            |
| U low          |             |          | Complementary çıkış veya sürücü içi |
| V high / PWM B |             |          | Timer alternate function            |
| V low          |             |          | Complementary çıkış veya sürücü içi |
| W high / PWM C |             |          | Timer alternate function            |
| W low          |             |          | Complementary çıkış veya sürücü içi |
| Enable         |             |          | Reset anında güvenli seviye         |
| Fault          |             |          | BKIN veya MCPWM fault               |
| Akım ölçümü    |             |          | ADC kanal ve ölçek                  |
| BEMF U/V/W     |             |          | ADC/comparator, varsa               |
| Hall 1/2/3     |             |          | Input capture / EXTI, varsa         |

Tablo 8.2 — Yalnız sinyal bağlantıları. Sürücü girişleri 3.3 V uyumlu olmayabilir veya aktif-low olabilir; GND referansının nasıl bağlandığı, izolasyon olup olmadığı ve maksimum pin gerilimleri datasheet'ten doğrulanır.

### Aşama 2 · Güç katı kapalıyken PWM üretme

Amaç motoru döndürmek değil, doğru sinyalin üretildiğini kanıtlamaktır. Sürücü enable pasif tutulur; timer tek bir güvenli frekansta yapılandırılır; üç kanalın aynı periyotta olduğu ölçülür; complementary çıkışların polaritesi kontrol edilir; dead-time osiloskopa ölçülür; MCU reset ve debug duraklamasında çıkışların güvenli kaldığı doğrulanır.

**Geçiş kriteri:** High-side ve low-side hiçbir anda çakışmıyor; frekans ve duty hesaplanan değerde; enable kapalıyken güç anahtarları iletme geçmiyor; fault etkinleştirildiğinde PWM donanımsal kapanıyor.

### Aşama 3 · Komütasyon tablosunu sürücüsüz test etme

Her adım düşük hızda yapay olarak ilerletilir ve altı çıkış logic analyzer ile izlenir. Kontrol listesi: her adımda yalnız doğru faz çifti aktif mi? Boş fazın iki anahtarı da kapalı mı? Adım değişiminde kısa yanlış kombinasyon var mı? İleri ve geri sıra doğru mu? Geçersiz state olursa tüm çıkışlar kapanıyor mu?

**Geçiş kriteri:** Altı adımın tamamı beklenen desende, ara geçişlerde yanlış kombinasyon olmadan gözlenir.

### Aşama 4 · Geri beslemeyi tek başına doğrulama

**Hall varsa:** motor enerjisiz elle yavaşça çevrilir, Hall kodu ve timestamp UART'a kaydedilir. Altı geçerli kod periyodik tekrar etmeli, 000/111 sürekli görülmemeli, ileri çevirince kodlar bir sırada, geri çevirince ters sırada ilerlemeli.

**Sensorless ise:** BEMF algılama donanımı kartta doğrulanır. Faz gerilimleri MCU ADC seviyesine uygun direnç bölücülerden geçmelidir; faz hattı doğrudan ADC pinine bağlanmaz.

### Aşama 5 · Düşük enerjili ilk dönüş

Motor mekanik olarak sabitlenir, tehlikeli yük veya pervane çıkarılır. Laboratuvar kaynağı düşük gerilim ve düşük akım limitiyle ayarlanır. Başlangıç duty'si sınırlı seçilir. Hall tabanlıysa mevcut Hall durumundan doğru adım seçilir; sensorless ise hizalama ve yumuşak açık çevrim rampası uygulanır. Akım, bus gerilimi, fault ve RPM loglanır. Titreşim, ters yön, aşırı akım veya düzensiz ses varsa hemen durdurulur.

**İlk hedef hız değildir:** başarı kriteri motorun doğru yönde, sınırlı akımla, altı adımı kaybetmeden ve koruma mekanizmaları etkin halde dönmesidir.

### Aşama 6 · RPM ölçümü ve rampa

Geçiş sürelerinden RPM hesaplanır ve bağımsız takometre veya çalışan bir kartla karşılaştırılır. Start/stop sırasında duty bir anda sıçratılmaz; her döngüde sınırlı bir adımla artırılır:  $D_{k+1} = \text{sınır}(D_k + \Delta D, D_{\min}, D_{\max})$ . Aynı yaklaşım hedef RPM rampası için de kullanılır. Ani yön değişimi yüksek akım ve mekanik zorlanma oluşturur; önce hız güvenli seviyeye düşürülür.

### Aşama 7 · Kapalı çevrim hız kontrolü

Önce yalnız P ile küçük kazançtan başlanır; sistem kararlı olduğunda kalıcı hatayı azaltmak için I eklenir. Her değişiklikte şunlar kaydedilir: hedef RPM, ölçülen RPM, duty-cycle, faz veya DC-bus akımı, fault bayrakları, başlangıç ve yerleşme süresi.

## Aşama 8 · Hata testleri

| TEST                 | BEKLENEN DAVRANIŞ                                        | KANIT                      |
|----------------------|----------------------------------------------------------|----------------------------|
| Enable kapatılması   | Tüm güç çıkışları güvenli seviyeye geçer                 | Osiloskop görüntüsü        |
| Driver fault         | PWM donanımsal kapanır, fault kaydedilir                 | Fault pini ve PWM birlikte |
| Hall kablosu çıkarma | Geçersiz kod veya zaman aşımı ile durur                  | Log ve hata kodu           |
| Rotor kilitlenmesi   | Akım veya komütasyon zaman aşımı koruması çalışır        | Akım grafiği, fault        |
| Düşük bus gerilimi   | Kontrollü stop veya startup engeli                       | Gerilim ve state logu      |
| Aşırı hız            | Duty kesilir veya kontrollü duruş                        | RPM ve state logu          |
| Watchdog             | Yazılım kilitlenmesinde güvenli reset ve çıkış kapanması | Reset sebebi               |

Tablo 8.3 — Proje sonuna doğru uygulanacak örnek güvenlik testleri.

## 9 · Hata ayıklama ve ölçüm

Motor dönmediğinde tahminle parametre değiştirmek zaman kaybıdır. Belirti → olası neden → kontrol sırası izlenir.

| BELİRTİ                        | OLASI NEDENLER                                                                                               | KONTROLLÜ İNCELEME                                                                                        |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Motor yalnız titriyor          | Faz sırası yanlış; Hall tablosu yanlış; başlangıç torku yetersiz; bir faz çalışmıyor                         | Altı state'i logic analyzer ile incele; faz-Hall eşlemesini yeniden çıkar; akım limitini kontrol et       |
| Motor ters dönüyor             | Komütasyon sırası ters; iki faz yer değişmiş                                                                 | Yazılım sırasını kontrollü tersle; sensör eşlemesini doğrula                                              |
| Başlangıçta yüksek akım        | Shoot-through; yanlış state; çok yüksek duty; rotor kilitli                                                  | Dead-time ve gate dalga şekillerini ölç; güç vermeden tabloyu doğrula; duty'yi sınırla                    |
| Boşta dönüyor, yükte kalkmıyor | Startup rampası hızlı; akım limiti düşük; besleme zayıf; komütasyon gecikmesi yanlış                         | Bus gerilimi ve akımı logla; rampayı yumuşat; üretici sınırları içinde parametre ayarla                   |
| Sensorless senkron kaybediyor  | Zero crossing gürültülü; demanyetizasyon maskesi kısa; 30° gecikme yanlış; duty algılama penceresini bozuyor | Boş faz dalga şeklini ölç; ADC tetikleme anını ve filtreyi doğrula; hız aralığına göre gecikmeyi güncelle |
| RPM sıcıyor                    | Capture taşması; Hall bounce/gürültü; tek periyot ölçümü; yanlış kutup çifti                                 | Timestamp logla; geçersiz kısa aralıkları reddet; hareketli ortalama kullan; p değerini doğrula           |
| MOSFET ısınıyor                | Dead-time az veya fazla; yavaş gate sürme; yüksek anahtarlama kaybı; sürekli yüksek akım                     | Gate dalga şekli, sıcaklık ve akımı ölç; driver/MOSFET datasheet'ini incele                               |
| MCU reset oluyor               | Besleme çökmesi; EMI; ground/layout problemi; watchdog                                                       | Reset sebebini kaydet; logic ve güç beslemelerini ölç; decoupling ve bağlantıları incele                  |

Tablo 9.1 — Belirti, olası neden ve kontrol sırası.

### Hangi dört sinyali aynı anda görmelisin?

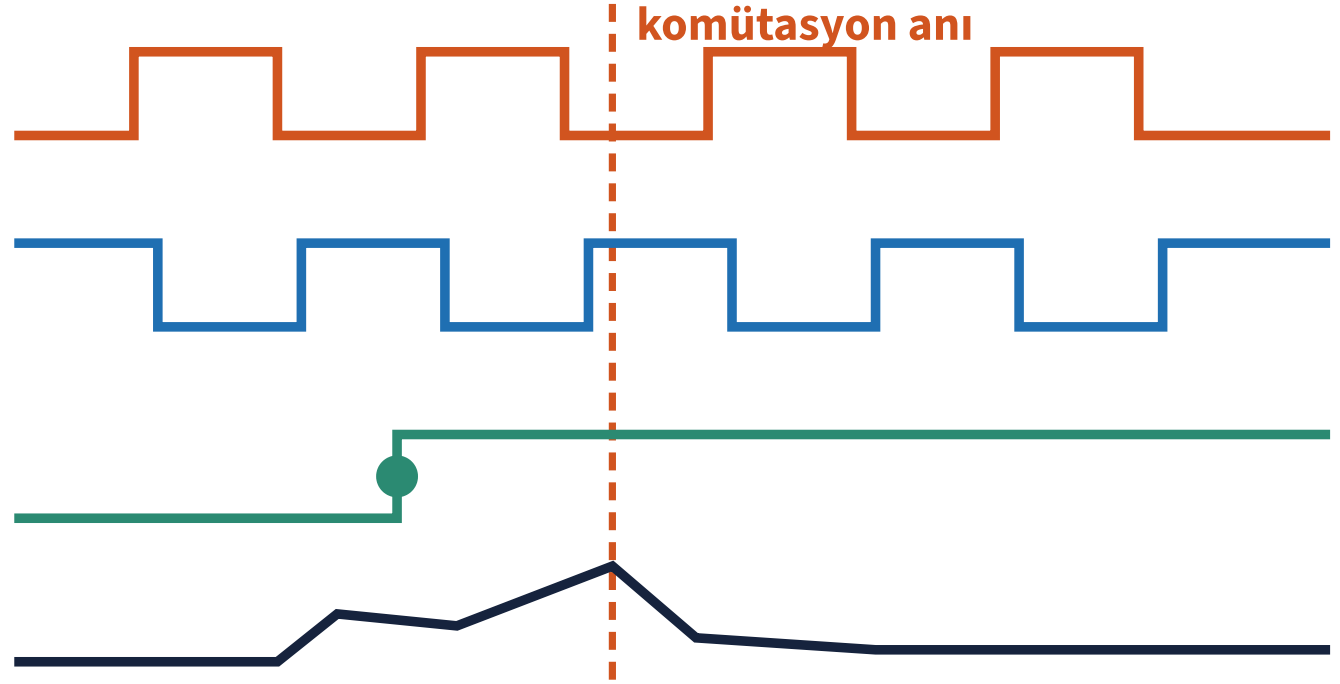
Sinyalleri ayrı ayrı görmek "önce hangisi oldu?" sorusunu cevaplamaz. Dördünü aynı zaman ekseninde kaydedin.

1 • high-side PWM

2 • aynı kol low-side

3 • Hall / BEMF olayı

4 • akım / fault



Şekil 9.1 — Dört kanal birlikte kaydedilirse "komütasyon geç mi kaldı, PWM çakıştı mı, fault önce mi geldi?" soruları zaman ekseninde cevaplanır.

#### OSİLOSKOP GROUND KLİPSİ UYARISI

Toprak referanslı standart probun ground klipsini inverter'ın U-V-W switching node'una bağlamak kısa devre ve ekipman hasarı oluşturur. Yüksek taraf veya faz-faz ölçümü için uygun diferansiyel prob, izolasyon yöntemi ve laboratuvar prosedürü kullanılır. Kondansatörlerin boşaldığı ölçülmeden karta dokunulmaz.

#### Log tasarımı

Gerçek zamanlı kontrolü bozmamak için ring buffer kullanılır: hızlı ISR yalnız veri örneğini belleğe yazar, UART aktarımı daha yavaş görevde yapılır. Her satırda en az şu alanlar bulunur:

time\_us

state

step

hall

bemf\_event

target\_rpm

rpm

duty

current

bus\_v

fault

## Motor dönmüyor: karar ağacı

Sürücü girişinde doğru PWM var mı?

hayır →

Timer, pin alternate function, enable, polarite ve dead-time kontrolü

evet ↓

Üç fazın tamamı anahtarlanıyor mu?

hayır →

Driver fault, gate sinyali, MOSFET ve besleme kontrolü

evet ↓

Hall / BEMF sırası komütasyonla uyumlu mu?

hayır →

Faz-sensör eşlemesini yeniden çıkar

evet ↓

Duty, akım limiti ve startup rampasını incele

Şekil 9.2 — Sırayı bozmadan ilerleyin: her adım bir sonrakinin ön koşuludur.

## 10 · Güvenlik ve koruma tasarımı

Koruma iki katmandır: donanım hızlı yolu olayı milisaniyeden önce keser, yazılım ise sebebi anlar ve kaydeder. İkisi birbirinin yerine geçmez.

| KORUMA                    | DONANIMSAL HIZLI YOL                      | YAZILIM GÖREVİ                                |
|---------------------------|-------------------------------------------|-----------------------------------------------|
| Aşırı akım                | Comparator + BKIN/fault ile PWM kesme     | Sebebi kaydetme, yeniden başlatmayı engelleme |
| Gate driver fault         | Driver çıkışı PWM brake girişine bağlanır | Fault register'larını okuma ve raporlama      |
| Yüksek/düşük bus gerilimi | Sürücüde UVLO/OVLO varsa hızlı koruma     | ADC ile eşik ve zaman filtresi                |
| Aşırı sıcaklık            | Termal shutdown veya comparator           | Sıcaklık trendi, derating ve kontrollü stop   |
| Stall                     | Akım limiti katkı sağlar                  | Hız yok + yüksek duty/akım koşulunu tanıma    |
| Watchdog                  | Bağımsız watchdog reseti                  | Periyodik besleme ve reset sebebi kaydı       |

Tablo 10.1 — Aynı olay için iki katman. Yazılım tek başına yeterli değildir: interrupt gecikmesi bile MOSFET için uzun olabilir.

### GÜVENLİ BAŞLANGIÇ VARSAYIMLARI

- MCU reset olurken PWM pinleri floating kalıp sürücüyü açmamalı.
- Enable pini harici pull-down/pull-up ile güvenli varsayılan seviyede tutulmalı.
- Yazılım başlamadan fault pininin geçerli olduğu doğrulanmalı.
- Akım sensörü offset kalibrasyonu motor enerjisizken yapılmalı.
- Geçersiz Hall kodu veya beklenmeyen timeout çıkışları kapatmalı.
- Debugger breakpoint'inde timer çıkışlarının ne yaptığı bilinmeli.

### MEKANİK GÜVENLİK

- Motor sağlam bir düzeneğe sabitlenmeli.
- İlk testte pervane, kesici veya ağır dengesiz yük bulunmamalı.
- Gevşek kablolar dönen parçadan uzak tutulmalı.
- Koruyucu gözlük ve laboratuvar prosedürü kullanılmalı.
- Motorun nominal hız ve akım sınırları aşılmamalı.

## Rejeneratif enerji: motor jeneratöre dönüşünce



Motor yavaşlarken jeneratör gibi davranır ve enerjiyi DC bus'a geri gönderir. Her laboratuvar güç kaynağı bu enerjiyi ememez; bus gerilimi yükselir. Ani stop veya aktif fren uygulamadan önce sürücünün frenleme yöntemi, bus kondansatörü, koruma eşiği ve gerekiyorsa fren direnci incelenir.

Şekil 10.1 — Enerji akışı yavaşlama sırasında ters yöne döner.

# 11 · Proje yapısı, teslimatlar ve çalışma planı

## ÖNERİLEN KLASÖR AYRIMI

### motor\_control

üst seviye API: start, stop, hedef RPM

### motor\_state\_machine

durum geçişleri ve fault kararları

### commutation

six-step tablosu · donanımdan bağımsız

### speed\_control

rampa ve PI

### motor\_fault

fault bayrakları ve kayıt

### motor\_hw\_stm32 / motor\_hw\_esp32

timer, ADC, GPIO · tek değişken katman

### Tests · Docs

tablo testleri, pin\_mapping, test\_plan, kaydedilmiş dalga şekilleri

Bu ayırım sayesinde six-step tablosu ve durum makinesi bilgisayarda test edilebilir; timer register ayrıntısı yalnız donanım katmanında kalır.

## GÖSTERİLEBİLİR PROJE ÇIKTILARI

- 01 Motor ve sürücü parametre tablosu
- 02 Sistem blok diyagramı ve pin eşleme tablosu
- 03 Altı durum komütasyon tablosu
- 04 Complementary PWM ve dead-time osiloskop görüntüsü
- 05 Hall veya Back-EMF geçiş görüntüsü
- 06 Başlangıç durum makinesi diyagramı
- 07 Hedef ve ölçülen RPM grafiği
- 08 Fault testi sonuç tablosu
- 09 Kod modülleri ve README
- 10 Bilinen sınırlamalar ve sonraki geliştirmeler

"Motor döndü" bir sonuç değil, bir gözlemdir. Yukarıdaki on kanıt, işin mühendislik tarafını gösterir.

## İki haftalık örnek çalışma planı

| GÜN | KONU                                          | SOMUT ÇIKTI                                        |
|-----|-----------------------------------------------|----------------------------------------------------|
| 1   | BLDC, stator, rotor, U-V-W ve six-step        | Kavram özeti ve altı durumu elle çizme             |
| 2   | Yarım köprü, inverter, gate driver, dead-time | Güç ve kontrol yollarını açıklayabilme             |
| 3   | Motor ve sürücü datasheet'i                   | Donanım kimlik kartı ve pin eşleme tablosu         |
| 4   | STM32 TIM1/TIM8 veya ESP MCPWM                | Güç kapalıyken üç senkron PWM                      |
| 5   | Complementary PWM, dead-time, fault           | Osiloskop kanıtı ve güvenli kapatma                |
| 6   | Komütasyon lookup table                       | Altı adımı düşük hızda logic analyzer ile gösterme |
| 7   | Hall veya Back-EMF geri beslemesi             | Geçiş sırası ve timestamp kaydı                    |
| 8   | RPM hesabı ve filtre                          | Elle döndürme veya düşük hız testinde RPM          |
| 9   | Düşük enerjili ilk dönüş                      | Sınırlı akımda kararlı dönüş                       |
| 10  | Sensorless ise startup rampası                | ALIGN → OPEN LOOP → CLOSED LOOP geçişi             |
| 11  | Hız rampası ve temel PI                       | Hedef-ölçülen RPM grafiği                          |
| 12  | Yön ve kontrollü duruş                        | Güvenli forward/reverse senaryosu                  |
| 13  | Fault senaryoları                             | Test sonuç tablosu                                 |
| 14  | Temizlik ve dokümantasyon                     | Demo, README ve teknik sunum                       |

Tablo 11.1 — Örnek plan; ekibin hızına göre uyarlanır.

### ÖNCELİK SIRASI

Önce güvenli PWM, sonra doğru altı durum, ardından geri besleme, en son hız kontrolü. FOC, Clarke-Park ve SVPWM bu proje gerçekten six-step istiyorsa ilk hedef değildir.

## Ek A · Hızlı referans

### Six-step hızlı tablo

| ADIM | KAYNAK | DÖNÜŞ | BOŞ / ÖLÇÜLEN |
|------|--------|-------|---------------|
| 1    | U      | V     | W             |
| 2    | U      | W     | V             |
| 3    | V      | W     | U             |
| 4    | V      | U     | W             |
| 5    | W      | U     | V             |
| 6    | W      | V     | U             |

Bu tablo faz akışını anlatır. Sürücü pinlerine uygulanacak gerçek high/low kombinasyonu, giriş polaritesi ve PWM modu datasheet truth table'ından çıkarılır.

### Temel formüller

$$V = I \cdot R \cdot \text{Ohm kanunu}$$

$$P_{DC} = V \cdot I \cdot \text{aktarılan güç}$$

$$P_{direnç} = I^2 \cdot R \cdot \text{ısı kaybı}$$

$$D = t_{on} / T \cdot \text{duty-cycle}$$

$$\theta_e = p \cdot \theta_m \cdot \text{elektriksel/mekanik açı}$$

$$RPM = 60 / (6 \cdot p \cdot \Delta t) \cdot \text{sektör süresinden hız}$$

$$f_{PWM,center} = f_{TIM} / [2(PSC+1)(ARR+1)]$$

$$V_{ADC} = ADC_{raw} / (2^N - 1) \cdot V_{ref}$$

$$I = (V_{ADC} - V_{offset}) / (G \cdot R_{shunt})$$

### Kart geldiğinde sorulacak 15 soru

- 01 Motorun tam modeli nedir?
- 02 Nominal gerilim ve sürekli akım kaçtır?
- 03 Kutup çifti sayısı kaçtır?
- 04 Motorda Hall veya başka konum sensörü var mı?
- 05 Yalnızca U-V-W mi çıkıyor?
- 06 Sürücü kartının tam modeli ve şeması nedir?
- 07 Kart hazır inverter mi, gate driver mı, ESC mi?
- 08 Sürücü 3-PWM mi, 6-PWM mi, başka komut mu bekliyor?
- 09 Logic girişleri 3.3 V uyumlu mu, aktif-high mı?
- 10 Dead-time sürücünde mi MCU'da mı oluşacak?
- 11 Fault pini hangi hatalarda aktif olur?
- 12 Akım ölçümü ve ölçeği nedir?
- 13 Back-EMF ölçüm devresi kartta var mı?
- 14 İlk test için izin verilen bus gerilimi ve akım limiti nedir?
- 15 Beklenen teslimat yalnız dönmesi mi; RPM/yön/fault kontrolü de var mı?

## Ek B · Terimler sözlüğü

**BLDC** · Fırçasız doğru akım motoru; elektronik komütasyon gerekir.

**Stator** · Motorun sabit, sargıları taşıyan kısmı.

**Rotor** · Dönen, çoğunlukla kalıcı mıknatıslı kısım.

**Faz** · U, V, W olarak adlandırılan sargı bağlantılarından biri.

**Komütasyon** · Hangi faz çiftinin enerjilendiğini doğru sırada değiştirme işlemi.

**Six-step** · Bir elektriksel turu altı 60° sektöre bölen trapezoidal kontrol.

**Elektriksel açı** · Manyetik alanın bir elektriksel çevrimdeki açısı; p ile mekanikten ayrılır.

**PWM** · Dijital anahtarlama ile ortalama gerilim/güç kontrol yöntemi.

**Duty-cycle** · PWM periyodunda sinyalin açık kaldığı oran.

**High-side** · Yarım köprünün DC+ tarafındaki güç anahtarı.

**Low-side** · Yarım köprünün GND tarafındaki güç anahtarı.

**Shunt** · Akım ölçmek için kullanılan düşük değerli hassas direnç.

**ADC** · Analog gerilimi sayısal değere dönüştüren çevre birimi.

**DC-link** · İnverterin DC besleme barası ve üzerindeki enerji depolayan kondansatörler.

**Dead-time** · Aynı koldaki iki anahtar arasında bırakılan çakışmasız bekleme.

**Shoot-through** · High ve low side'in birlikte açılmasıyla oluşan DC-bus kısa devresi.

**Gate driver** · MOSFET kapılarını uygun gerilim ve akımla süren entegre.

**Inverter** · DC enerjiyi anahtarlayarak üç fazlı değişken gerilim üreten güç katı.

**Hall sensörü** · Manyetik alanı algılayıp rotor sektörünü dijital olarak bildiren sensör.

**Back-EMF** · Dönen motorun sargılarında indüklenen, hız/konum bilgisi taşıyan gerilim.

**Zero crossing** · Boştaki fazın Back-EMF sinyalinin referans eşiğini geçtiği olay.

**Sensorless** · Hall veya encoder olmadan faz gerilim/akımından konum tahmini.

**Open loop** · Geri beslemeye dayanmadan önceden belirlenen zamanlamayla kontrol.

**Closed loop** · Ölçülen hız/konumu hedefle karşılaştırıp komutu düzeltten kontrol.

**PI controller** · Oransal ve integral terimlerle hatadan çıkış üreten denetleyici.

**Input capture** · Bir dijital kenarın olduğu timer sayım değerini donanımda kaydetme.

**BKIN / brake** · Fault'ta PWM çıkışlarını donanımsal olarak güvenli duruma getiren giriş.

**Anti-windup** · Çıkış sınırdarken integral teriminin birikmesini engelleyen önlem.

## Ek C · Kaynaklar

Bu belgedeki teknik yaklaşım üretici kaynaklarıyla uyumludur. Kesin pin, register, elektriksel sınır ve konfigürasyon için **kullanılan parça numarasına ait güncel datasheet, reference manual ve errata** esas alınır.

- 1 STMicroelectronics — STM32 Motor Control, 6-step Firmware Algorithm (wiki.st.com). Sensorless Back-EMF algılama, zero crossing, ~30° gecikme, Hall ve six-step bileşenleri.
- 2 STMicroelectronics — AN4220 uygulama notu ve ilgili six-step referans dokümanları.
- 3 Kullanılan gate driver / motor driver entegresinin datasheet'i: giriş doğruluk tablosu, dead-time, fault davranışı.
- 4 Seçilen MCU'nun reference manual'ı: timer, complementary PWM, break girişi, ADC tetikleme ve capture bölümleri.
- 5 Motor üreticisinin veri sayfası: kutup çifti sayısı, Hall yerleşimi, faz sırası, akım ve hız sınırları.

# Encoder ve Hall Sensörleri

Mekanik hareketin dijital sinyale dönüşü: pozisyon ve hız ölçümü, BLDC komütasyonu ve STM32 tarafındaki uygulama.

## Ne öğreneceksiniz

Encoder'ın darbe sayarak pozisyon ve hız vermesi; Hall'in rotoru altı sektöre bölmesi.

## Ön bilgi

Temel dijital giriş/çıkış, GPIO, timer kavramı ve C dilinde bit işlemleri.

## Uygulama

staj\_motor\_simule\_g491 projesi: PC0/PC1/PC2 Hall girişleri, 6-step komütasyon.

## İÇİNDEKİLER

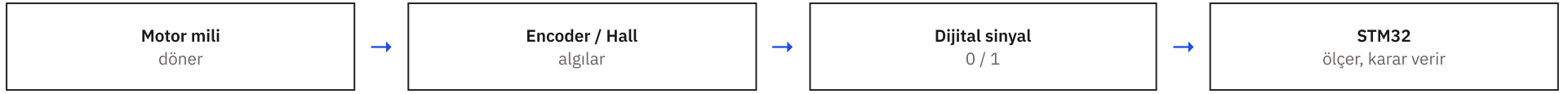
- |                                           |                                            |
|-------------------------------------------|--------------------------------------------|
| 01 Sensör neden gerekir?                  | 07 Hall ile hız hesabı                     |
| 02 Encoder: yapı, A/B kanalı, x4 sayım    | 08 Encoder ve Hall karşılaştırması         |
| 03 STM32 timer ile encoder okuma          | 09 Projedeki durum: staj_motor_simule_g491 |
| 04 Hall sensörleri ve rotor konumu        | 10 Kapalı çevrim kontrolün zihinsel modeli |
| 05 Altı adımlı BLDC komütasyonu           | 11 Sık karıştırılan noktalar               |
| 06 Hall girişleri: union, bit alanı, EXTI | 12 Özet ve terim sözlüğü                   |

## 01 Sensör neden gerekir?

Bir mikrodenetleyici motor milini göremez. Yazılım tarafında elinizde yalnızca pinlerdeki gerilim seviyeleri vardır. Bu yüzden dönme hareketinin önce elektriksel bir bilgiye çevrilmesi gerekir; bu çeviriyi yapan bileşen sensördür.

Motor sürüşünde iki ayrı soru sorulur ve bu iki soruyu genellikle iki ayrı sensör yanıtlar. Encoder ile Hall sensörünü karıştırmamanın en kısa yolu, hangi soruya cevap verdiklerini hatırlamaktır.

**Pozisyon** cismin referansa göre konumu. **Hız** pozisyonun zamana göre değişimi. **Komütasyon** tork üretmek için fazların doğru sırada enerjilenmesi.



ŞEKİL 1 — Mekanik harekettten yazılım kararına giden zincir.

#### ENCODER'IN SORUSU

“Mil ne kadar döndü ve ne hızla dönüyor?”

#### HALL'IN SORUSU

“Rotor şu anda hangi elektriksel sektörde; sıradaki komütasyon ne olmalı?”

## 02 Encoder: yapı, A/B kanalı ve x4 sayım

### 2.1 Darbe nasıl üretilir?

Artımlı (incremental) encoder, mil döndükçe darbe üretir. Optik modelde yarıklı bir disk, LED ile fotodedektör arasından geçer: yarık ışığı geçirdiğinde çıkış bir seviyede, diski kapattığında diğer seviyede olur. Dönüş devam ettikçe çıkış sürekli kenar üretir.

Çıkışın HIGH mi LOW mu olduğu devreye bağlıdır ve tek başına bir anlam taşımaz. Ölçüm için önemli olan, dönüş boyunca *kararlı ve sayılabilir kenarların oluşmasıdır*.

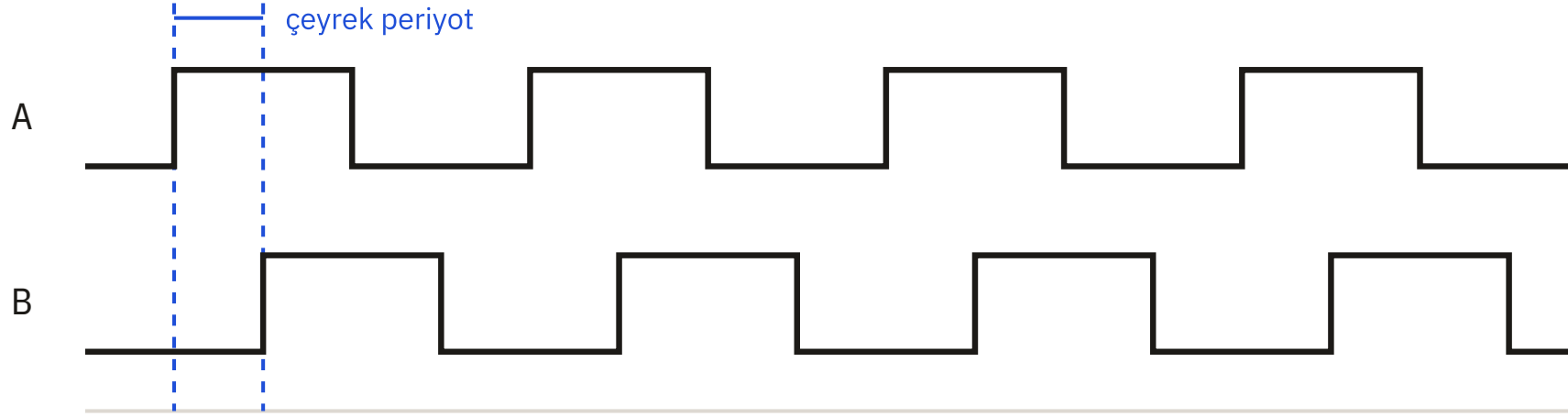
Artımlı encoder yalnızca **değişimi** bilir. Güç kesilirse sayaç sıfırlanır; mutlak konum için referans alma (homing) gerekir.

| Tür       | İlke                                   | Güçlü yanı                    | Dikkat                            |
|-----------|----------------------------------------|-------------------------------|-----------------------------------|
| Optik     | Yarık/desen ve ışık algılama           | Yüksek çözünürlük             | Kir, hizalama, optik yol          |
| Manyetik  | Mıknatıs ve manyetik algılayıcı        | Sağlam, kapalı gövdeye uygun  | Manyetik parazit, mıknatıs hizası |
| Kapasitif | Dönen desenin kapasitansı değiştirmesi | Kirli ortama görece dayanıklı | Arayüzü ürüne özgü                |

## 2.2 A ve B kanalı yönü nasıl söyler?

A ve B aynı frekanstadır, fakat aralarında yaklaşık 90° elektriksel faz farkı vardır; bu düzene *quadrature* denir. Yön bilgisi tek bir kanaldan çıkmaz: hangi kanalın kenarının önce geldiğine bakılır. A'nın kenarı B'den önce geliyorsa mil bir yöne, tersi olursa diğer yöne dönüyordur.

“A > B” bir büyüklük karşılaştırması **değildir**. Doğru ifade: A faz olarak B'den öndedir.



ŞEKİL 2 — Bu örnekte A'nın yükselen kenarı B'den önce gelir. A ile B'nin kabloları yer değiştirirse sayım yönü tersine döner.

## 2.3 x1, x2 ve x4 sayım

Tek bir A periyodunda dört kenar bulunur: A yükselir, B yükselir, A düşer, B düşer. Timer bu kenarlardan yalnızca birini sayarsa x1, ikisini sayarsa x2, dördünü de sayarsa x4 çözünürlük elde edilir. Yani aynı encoder, yalnızca sayım kipini değiştirerek dört kat çözünürlük verebilir.

Z (index) kanalı varsa turda bir darbe üretir. Başlangıç referansı içindir; A/B'nin yerine geçmez.

#### ÖRNEK – ÇÖZÜNÜRLÜK

Üretici “600 PPR” ile A kanalında tur başına 600 periyot kastediyorsa, quadrature x4 sayımda tur başına sayım ve bir sayımın aç karşılığı şöyle olur:

$$N_{tur} = 4 \times 600 = 2400 \text{ count/tur} \cdot \theta_{count} = 360^\circ / 2400 = 0,15^\circ$$

PPR ve CPR tanımları üreticiden üreticiye değişir. Datasheet’te verilen değerin periyot mu kenar mı olduğu mutlaka kontrol edilmelidir; aksi halde ölçülen açı dört kat yanlış olur.

#### 2.4 Mekanik encoder ve aktif encoder

“Ölü encoder / canlı encoder” standart teknik terimler değildir. Doğru ayrım şu şekildedir:

- **Mekanik artımlı encoder.** Kontakları açıp kapatır. Kontak sıçraması (bounce) oluşabileceği için yazılımda veya donanımda debounce gerekir.
- **Aktif optik/manyetik encoder.** Besleme ister ve elektronik çıkış üretir. Çıkış push-pull, open-collector veya diferansiyel olabilir.

Open-collector çıkışta harici pull-up gerekir. 5 V çıkışın STM32 pinine doğrudan uygun olduğu varsayılmamalı; pin toleransı kontrol edilmeli.

#### HIZLI TEKRAR · BÖLÜM 02

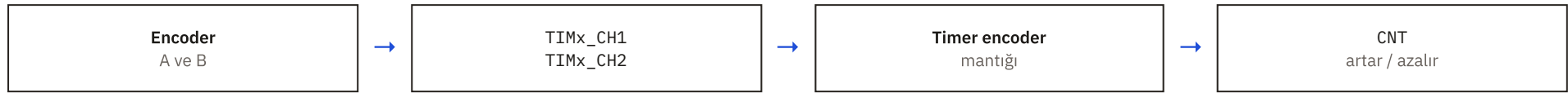
1. Tek kanallı bir encoder ile yön neden bulunamaz?
2. 1024 PPR bir encoder x4 sayımda turda kaç count üretir, bir count kaç dereceye karşılık gelir?
3. A ve B kabloları yer değiştirdiğinde ölçümde ne değişir, ne değişmez?

### 03 STM32 timer ile encoder okuma

STM32 timer'larının encoder arayüzü, A ve B kanallarını donanımda izler ve sayacı yönüne göre artırıp azaltır. CPU her kenarda kesmeye girip yön hesabı yapmak zorunda kalmaz; yazılımın işi yalnızca sayacı belirli aralıklarla okumaktır.

CubeMX'te uygun timer için Encoder Mode seçilir, iki pin Alternate Function olarak atanır, sayaç periyodu belirlenir ve gürültülü ortamda dijital giriş filtresi ayarlanır.

Donanım sayımının değeri  
şurada: 10 000 count/s hızda  
yazılım kesmesi CPU'yu boğar,  
timer ise hiç yük bindirmez.



ŞEKİL 3 – Encoder arayüzünde yön ve sayım donanımda çözülür.

```
HAL_TIM_Encoder_Start(&htim2, TIM_CHANNEL_ALL);  
  
uint16_t simdiki = (uint16_t)__HAL_TIM_GET_COUNTER(&htim2);  
int16_t delta = (int16_t)(simdiki - onceki); // wrap-around ile uyumlu  
onceki = simdiki;
```

#### HIZ FORMÜLÜ

$$RPM = (\Delta count / N_{tur}) \times (60 / \Delta t)$$

Hız daima *sabit* bir örnekleme aralığında hesaplanır;  $\Delta t$  kayarsa hız da kayar.

#### SAYISAL ÖRNEK

2400 count/tur encoder,  $\Delta t = 0,01$  s ve bu süre içinde  $\Delta count = 40$ :

$$RPM = (40 / 2400) \times (60 / 0,01) = 100$$

#### KODDA SIK YAPILAN HATA

Sayacı iki kez okumak yerine bir kez okuyup deęiřkende saklamak gerekir. İki okuma arasında yeni bir kenar gelirse hesaplanan fark tutarsız olur.

Sayaç tařmasında doęru sonucu almak için sayaç geniřlięine uygun *iřaretli* fark kullanılmalıdır. TIM2 bazı STM32 ailelerinde 32 bittir; kullanılan MCU'nun reference manual'ı kontrol edilmelidir.

## 04 Hall sensörleri ve BLDC rotor konumu

### 4.1 Hall etkisi ve motor içindeki yerleřim

Hall elemanı, içinden akım geçerken manyetik alana baęlı küçük bir gerilim üretir. Dijital Hall sensöründe bu analog bilgi bir eřik devresiyle 0/1 seviyesine çevrilir; yazılım tarafında elinizde temiz bir dijital giriř olur. BLDC motorlarda çoęunlukla üç dijital Hall çıkıřı bulunur.

Sensörler motor tasarımına göre mekanik olarak yaklaşık 120°, ya da elektriksel olarak 120° aralıklı yerleřtirilir. Amaç her an rotorun hangi sektörde olduęunu üç bitle kodlamaktır.

Hall çıkıřı rotorun **kaba** konumunu verir: sektör bilgisi, açı deęil. Açı çözünürlüęü encoder'ın iři.

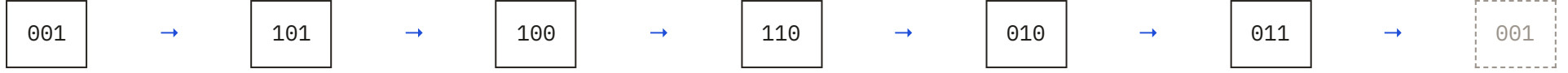
#### ÖNEMLİ DÜZELTME – HIGH/LOW

“Kuzey = HIGH, Güney = LOW” evrensel bir kural deęildir. Sensör tipi (unipolar, bipolar/latching), mıknatıs yönü, ara devre ve çıkıř terslemesi sonucu deęiřtirebilir. Kontrol yazılımı için güvenilir tek bilgi, *ölçülen üç bitlik kodun rotor sektörüyle eřleřmesidir*.

### 4.2 Üç bitten neden altı durum çıkar?

Üç dijital giriř teorik olarak  $2^3 = 8$  kombinasyon üretir. Normal 120° Hall düzeninde 000 ve 111 fiziksel olarak oluşmaz; bu ikisi geçersiz kabul edilir. Kalan altı kod, dönüş yönüne göre daima aynı halka üzerinde sırayla görünür.

Geçersiz kod görölmesi arıza iřaretidir: kopuk kablo, eksik pull-up, besleme sorunu veya gürültü.



ŞEKİL 4 – Bir elektriksel tur: örnek bir ileri sıra. Ters yönde aynı halka ters sırada ilerler. Sensör adlandırması, bit sırası ve motor kablolarına göre sıra farklı görünebilir.

#### ELEKTRİKSEL TUR ≠ MEKANİK TUR

Rotorun p kutup çifti varsa elektriksel açı, mekanik açının p katıdır. Mekanik bir turda görülen Hall geçişi sayısı da buna bağlıdır:

$$\theta_e = p \cdot \theta_m \cdot N_{Hall, mekanik\ tur} = 6p$$

Örneğin 4 kutup çiftli bir motorda mekanik turda 24 Hall geçişi görülür. Bu nedenle “Hall çözünürlüğü her zaman 6 adım/tur” ifadesi doğru değildir.

#### HIZLI TEKRAR · BÖLÜM 04

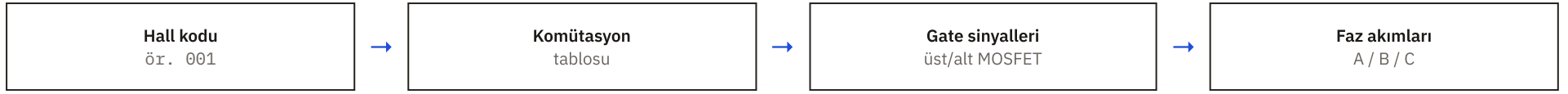
1. Üç Hall girişinden neden 8 değil 6 geçerli durum çıkar?
2. 3 kutup çiftli bir motorda mekanik turda kaç Hall geçişi beklenir?
3. Hall kodu 000 okunduğunda yazılım ne yapmalı?

## 05 Altı adımlı BLDC komütasyonu

Altı adımlı sürüşte her an üç fazdan biri DC bara artısına bağlanır (HIGH), biri eksiye bağlanır (LOW), biri enerjisiz bırakılır (FLOAT). Akım iki faz üzerinden akar, üçüncü faz boşta kalır. Hall kodu değiştiğinde bir sonraki adıma geçilir.

Tork ve hız, aktif fazlardan birine PWM uygulanarak ayarlanır. Yani Hall olayı *hangi* sektörde olduğunuzu, PWM ise o sektörde *ne kadar* gerilim uyguladığınızı belirler.

Hangi Hall kodunun hangi satırı seçtiği motor ve bağlantıya göre doğrulanmalıdır. Yanlış eşleme düşük tork, yüksek akım, titreşim veya ters yön üretir.



ŞEKİL 5 — Hall kodundan faz akımına giden karar zinciri.

| Adım | Faz A | Faz B | Faz C | Akım yolu |
|------|-------|-------|-------|-----------|
| 0    | HIGH  | LOW   | LOAT  | A → B     |
| 1    | HIGH  | LOAT  | LOW   | A → C     |
| 2    | LOAT  | HIGH  | LOW   | B → C     |
| 3    | LOW   | HIGH  | LOAT  | B → A     |
| 4    | LOW   | LOAT  | HIGH  | C → A     |
| 5    | LOAT  | LOW   | HIGH  | C → B     |

#### GÜVENLİK — HIGH/LOW NE DEMEK DEĞİLDİR

Buradaki HIGH ve LOW, mikrodenetleyici pininin basitçe 1/0 yapılması demek değildir. Bunlar üç fazlı inverterde ilgili üst veya alt MOSFET'in kontrollü sürülmesini temsil eder.

Aynı bacağıın üst ve alt MOSFET'i aynı anda açılırsa *shoot-through* oluşur ve güç katı anında zarar görür. Gate driver ve dead-time kullanılmadan güç katı sürülmemelidir.

## 06 STM32'de Hall girişleri: union, bit alanı ve EXTI

Projede Hall verisi HALL\_RAW türüyle temsil edilir. Bu yapı aynı 8 bitlik veriye iki farklı görünümünden erişmeyi sağlar: bit bit ve bütün byte olarak.

```
typedef union
{
    struct {
        uint8_t hall_c : 1;    // bit 0
        uint8_t hall_b : 1;    // bit 1
        uint8_t hall_a : 1;    // bit 2
        uint8_t reserved : 5;   // bit 3..7
    } bits;
    uint8_t val;               // 8 bitin tamamı
} HALL_RAW;
```

### 6.1 Union neden kullanılıyor?

Union, üyeleri için ayrı ayrı bellek ayırmaz; bütün üyeler aynı adresi paylaşır. Bu yüzden `bits` ve `val` iki farklı değişken değil, aynı 8 bitlik verinin iki farklı yorumudur. Kod, kimi yerde tek sensöre bakmak, kimi yerde bütün kodu tablo girişi olarak kullanmak ister; union bu iki ihtiyacı tek veriyle karşılar.

`val` tablo/switch için, `bits` okunabilirlik için.



ŞEKİL 6 – `val`: bütün byte. R: reserved, kullanılmayan bit.

`bits.hall_a / hall_b / hall_c`

`raw.val = 0x05` yazılırsa ikilik karşılığı `0000 0101` olur. Aynı bellek bit alanlarından okunduğunda `hall_a = 1`, `hall_b = 0`, `hall_c = 1` görülür; yani Hall kodu `ABC = 101` olarak yorumlanır.

Bit sırası, bit alanının bildirim sırasıyla aynıdır: `hall_c` en düşük bit.

```

HALL_RAW raw = {0};

raw.val = 0x05U;          // 0000 0101
if (raw.bits.hall_a) {    // bit 2 okunur -> 1
    // Hall A aktif
}
raw.bits.hall_b = 1U;      // bit 1 set edilir
// raw.val artık 0x07: 0000 0111

```

## 6.2 PC0, PC1 ve PC2 tek okumayla nasıl alınır?

Projede bağlantı PC0 = Hall C, PC1 = Hall B, PC2 = Hall A ise GPIOC giriş veri register'ının alt üç biti union düzeniyle birebir uyuşur. Bu sayede üç pini ayrı ayrı okumak yerine register bir defa örneklenir; üç sensör *aynı anda* yakalanmış olur.

$$GPIOC \rightarrow IDR[2:0] = HALL\_RAW.val[2:0]$$

A B C · hall\_a hall\_b hall\_c

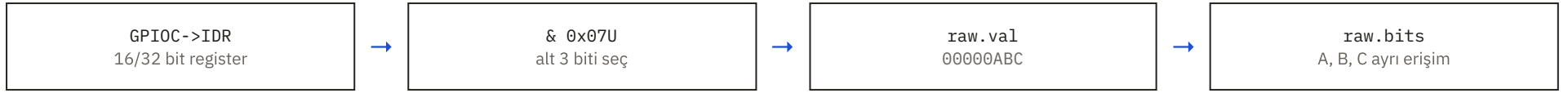
```

static HALL_RAW hall_oku(void)
{
    HALL_RAW raw = {0};
    // IDR'nin yalnızca bit 0, 1 ve 2'sini al.
    raw.val = (uint8_t)(GPIOC->IDR & 0x07U);
    return raw;
}

```

0x07U maskesi ikilik olarak 0000 0111'dir. AND işlemi PC0–PC2 bitlerini korur, diğer pinlere ait bitleri sıfırlar. Böylece reserved alanı daima 0 kalır ve val doğrudan tablo indeksi olarak kullanılabilir.

Maskleme olmadan val diğer pinlerin durumunu da taşır ve tablo indeksi bozulur.



ŞEKİL 7 — Tek register okumasından iki farklı veri görünümüne.

### 6.3 EXTI kesmesinde kullanım

Gerçek Hall sinyalleri PC0, PC1 ve PC2 üzerindeyse pinler dijital giriş olarak yapılandırılır. Her Hall geçişinde hızlı tepki vermek için EXTI kesmesi kullanılır: kesme geldiğinde kod okunur, geçerliliği sınanır ve komütasyon uygulanır.

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0 || GPIO_Pin == GPIO_PIN_1 ||
        GPIO_Pin == GPIO_PIN_2) {

        HALL_RAW hall = hall_oku();

        if (hall.val == 0x00U || hall.val == 0x07U) {
            mosfetleri_kapat(); // 000 ve 111 gecersiz
            return;
        }
        komutasyon_uygula(hall.val);
    }
}
```

hall.val lookup tablosu veya switch-case için uygundur. hall.bits.hall\_a gibi alanlar ise tek sensörü kontrol ederken kodun okunmasını kolaylaştırır. Tek register okuması, üç ayrı HAL\_GPIO\_ReadPin() çağrısına göre A/B/C'nin aynı anda örneklenmesini garanti eder.

Üç ayrı okuma arasında rotor dönmeye devam eder; sektör sınırında karışık bir kod okunabilir.

### BIT-FIELD TAŞINABİLİRLİĞİ

C standardı, bit-field üyelerinin byte içindeki yerleşim sırasını tamamen sabitlemez; düzen derleyici ve hedef mimariye bağlı olabilir. STM32 projesinde kullanılan ARM GCC/Clang derleyicisiyle düzen doğrulanmalı ve boyut kontrolü eklenmelidir. Donanım register'larında en taşınabilir yöntem maske ve kaydırmadır: `(raw.val >> 2) & 1U`. Union görünümü okunabilirlik sağlar, maskeler ise bit konumunu açıkça garanti eder.

```
_Static_assert(sizeof(HALL_RAW) == 1U,
               "HALL_RAW tam olarak 1 byte olmalı");

#define HALL_C_MASK (1U << 0)
#define HALL_B_MASK (1U << 1)
#define HALL_A_MASK (1U << 2)
#define HALL_MASK (HALL_A_MASK | HALL_B_MASK | HALL_C_MASK)
```

### GEÇERSİZ DURUM GÜVENLİĞİ

000, 111 veya beklenmeyen bir geçiş görülürse MOSFET'leri güvenli duruma almak, hata sayacını artırmak ve sistemi durdurmak iyi bir tasarımdır. Geçersiz kod; kopuk kablo, yanlış pull-up, besleme sorunu veya gürültü göstergesi olabilir.

### HIZLI TEKRAR · BÖLÜM 06

1. `raw.val = 0x06` yazılırsa `hall_a`, `hall_b` ve `hall_c` ne olur?
2. Neden üç ayrı `ReadPin` çağrısı yerine tek register okuması tercih edilir?
3. `& 0x07U` maskesi kaldırılırsa hangi hata ortaya çıkar?

## 07 Hall ile hız hesabı yapılabilir mi?

Evet. Hall temel olarak komütasyon konumu sağlasa da, geçişler arasındaki süre ölçülerek hız tahmin edilir. Rotorun p kutup çifti varsa,  $\Delta t$  süresinde sayılan n geçişten hız şöyle bulunur:

$$RPM = (n / 6p) \times (60 / \Delta t)$$

Hall çözünürlüğü encoder'a göre düşüktür. Özellikle düşük hızda geçişler seyrekleşir; iki geçiş arasında yazılım hız hakkında yeni bilgi almaz ve tahmin basamaklı görünür. Yüksek hızda ise fark küçülür.

Düşük hızda daha iyi sonuç için sabit pencerede geçiş saymak yerine **iki geçiş arasındaki süreyi** ölçmek tercih edilir.

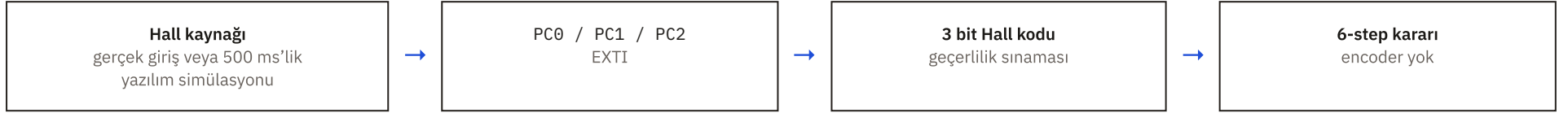
## 08 Encoder ve Hall karşılaştırması

| Özellik           | Artımlı encoder                           | BLDC içindeki 3 dijital Hall          |
|-------------------|-------------------------------------------|---------------------------------------|
| Ana amaç          | Hassas görelî pozisyon ve hız             | Elektriksel sektör / komütasyon       |
| Çıkış             | A, B; bazen Z                             | H1, H2, H3                            |
| Çözünürlük        | Genellikle yüksek; modele bağlı           | Elektriksel turda 6 sektör            |
| Yön bilgisi       | A/B faz sırasından                        | Hall kod sırasından                   |
| Mutlak başlangıç  | A/B tek başına vermez; home/Z gerekebilir | Açılışta sektör bilgisi verir         |
| STM32 çevrebirimi | Timer encoder mode                        | GPIO + EXTI veya timer Hall interface |
| Hız ölçümü        | Çok uygun                                 | Yapılır; düşük hızda daha kaba        |
| Tipik kullanım    | Servo, konum/hız döngüsü                  | Sensörlü 6-step BLDC sürüş            |

#### DÜZELTME – OPEN-LOOP / CLOSED-LOOP

Encoder veya Hall'ın varlığı tek başına sistemi closed-loop yapmaz. Closed-loop, ölçülen değerin hedefle karşılaştırılıp hataya göre komutun düzeltilmesidir. Hall ile yapılan komütasyon geri beslemeli bir *anahtarlama* işlemidir; hız denetimi ise ancak ölçülen hız hedef hızla karşılaştırılıp PWM buna göre düzeltilirse kapalı çevrim olur.

## 09 Bu projedeki durum: staj\_motor\_simule\_g491



ŞEKİL 8 — Projenin sinyal zinciri. Bu projede encoder bulunmaz.

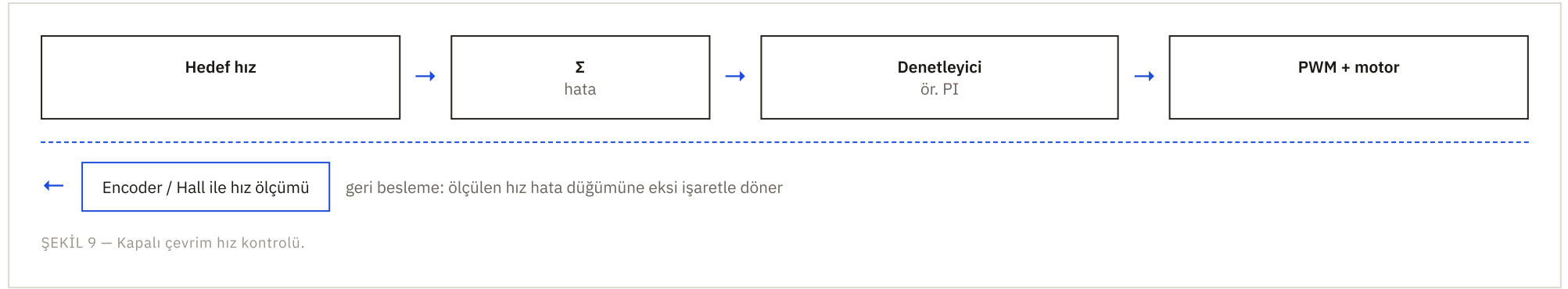
500 ms'lik yazılım simülasyonu, Hall kodlarının ve komütasyon tablosunun mantığını güç katı olmadan gözlemlemek için uygundur. Bu süre gerçek motor hızını temsil etmez; her adım yarım saniye sürdüğü için yapılan şey yalnızca yavaş bir durum makinesi testidir.

Simülasyonda motor dönmez.  
Yalnız sensör durumları ve  
yazılım akışı taklit edilir.

#### DEVREYE ALMA SIRASI

1. LED veya UART ile 001, 101, 100, 110, 010, 011 sırasının doğru üretildiği doğrulanır.
2. Her kodun doğru komütasyon satırını seçtiği, güç katı bağlı *değilken* incelenir.
3. Gerçek Hall girişlerinde pull-up/pull-down, lojik seviye ve EXTİ kenarları osiloskop veya logic analyzer ile doğrulanır.
4. Motorun gerçek Hall sırası mil elle yavaşça çevrilerek kaydedilir; tablo buna göre eşlenir.
5. Gate driver, dead-time, akım sınırlama ve acil kapatma doğrulanmadan motora güç verilmez.

## 10 Kapalı çevrim kontrolün zihinsel modeli



Hedef ile gerçek hız arasındaki fark hatadır. Denetleyici bu hataya göre PWM’i artırır veya azaltır. Sensör ölçüm sağlar; kararı veren, ölçümü kullanan kontrol algoritmasıdır. Sensör eklemek tek başına performans getirmez — ölçümün karara dönüşmesi gerekir.

## 11 Sık karıştırılan noktalar

### Hall sensörü encoder mıdır?

Geniş anlamda konum bilgisi üreten bir sensördür; ancak BLDC’deki üç Hall çıkışı, yüksek çözünürlüklü quadrature encoder ile aynı kabiliyette değildir.

### Encoder motoru döndürür mü?

Hayır. Encoder ölçer. Motoru inverter/güç sürücüsü döndürür; kontrolcü ölçümü kullanarak sürücüyü yönetir.

### Hall varsa encoder gereksiz midir?

Yalnız komütasyon ve kaba hız için çoğu zaman yeterlidir. Hassas mil konumu, düşük hız kararlılığı veya servo kontrol isteniyorsa encoder gerekir.

### Hall kodu doğrudan MOSFET pini midir?

Hayır. Hall kodu lookup tablosuna girer ve orada güvenli gate sinyallerine dönüştürülür.

## 12 Özet ve terim sözlüğü

### AKILDA KALACAK DÖRT CÜMLE

1. Encoder darbeleri sayılarak görelî pozisyon, zamana göre fark alınarak hız bulunur.
2. A/B faz sırası yönü verir; bütün kenarların sayılması x4 çözünürlük sağlar.
3. Üç Hall sensörü rotoru altı elektriksel sektöre ayırır ve 6-step komütasyonu seçtirir.
4. Bu projede encoder yoktur; PC0/PC1/PC2 gerçek Hall girişleri veya 500 ms’lik yazılım simülasyonu kullanılır.

### ANAHTAR TERİMLER

|                        |                                                                                                               |
|------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Quadrature</b>      | Aynı frekanslı, aralarında yaklaşık 90° faz farkı bulunan iki kanal (A ve B) düzeni.                          |
| <b>PPR / CPR</b>       | Tur başına darbe / sayım. Tanımı üreticiye göre değişir; periyot mu kenar mı olduğu datasheet’ten doğrulanır. |
| <b>Z (index)</b>       | Turda bir darbe üreten referans kanalı; başlangıç noktası işaretlemek için kullanılır.                        |
| <b>Kutup çifti (p)</b> | Rotordaki N-S mıknatıs çifti sayısı. Elektriksel tur sayısını ve Hall geçiş sayısını belirler.                |
| <b>Komütasyon</b>      | Tork üretmek için motor fazlarının doğru sırada enerjilenmesi.                                                |
| <b>FLOAT</b>           | Bir fazın hem üst hem alt MOSFET’i kapalı, yani enerjisiz bırakılmış durumu.                                  |
| <b>Shoot-through</b>   | Aynı inverter bacağına üst ve alt MOSFET’inin aynı anda ilettime geçmesi; kısa devre.                         |
| <b>Dead-time</b>       | Üst ve alt MOSFET arasında bilinçli olarak bırakılan, ikisinin de kapalı olduğu kısa süre.                    |
| <b>EXTI</b>            | STM32’de bir GPIO pinindeki kenar değişiminde kesme üreten harici kesme birimi.                               |
| <b>Union</b>           | Üyeleri aynı belleği paylaşan C türü; aynı veriye farklı görünümlerden erişim sağlar.                         |

---

Not: Pin alternatif fonksiyonları, timer genişliđi, giriş toleransları ve Hall/komütasyon sırası; kullanılan STM32 modeli, motor ve sürücü kartının datasheet ve reference manual belgelerinden doğrulanmalıdır.

# 4

KISIM DÖRT

## Simülasyon ve Uygulama

Bilgi artık karta iniyor. Önce STM32F401 ile hiç motor bağlamadan, GPIO pinlerinde komütasyon sırasını canlandıran donanımsız simülasyon; sonra NUCLEO-G491RE ile UART telemetrisi ve staj sırasında yazılan kendi izleme arayüzü. Her belgenin ardından o aşamanın ekran görüntüleri yer alır.

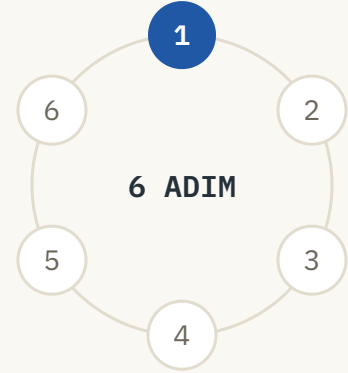
---

### BU KISIMDA

- 06** Sensörsüz BLDC Komütasyon Simülasyonu — STM32F401  
+ F401 uygulama görüntüleri (3)
- 07** NUCLEO-G491RE Üzerinde BLDC Komütasyonu  
+ Nucleo uygulama görüntüleri (9)

# Sensörsüz 3 Fazlı BLDC Motor Komütasyon Simülasyonu

Half-bridge kavramından 6 adımlı komütasyona; state machine simülasyonundan UART telemetrisine ve PC izleme arayüzüne.



# Üç Aşamalı Yol Haritası

## FAZ 1

### State machine + half-bridge takibi

6 adımı yazılımda doğru  
üret, GPIO ile ölçülebilir kıl



## FAZ 2

### UART ile durum telemetrisi

Her adım değişimini PC'ye  
metin olarak raporla



## FAZ 3

### PC tarafında izleme arayüzü

Terminal → Python → canlı  
görsel gösterim



## İLERİ SEVİYE

### Gerçek motor + sensörsüz BEMF

Donanım geldiğinde;  
haritada yerini bil

## Neden önce donanımsız?

Gerçek bir 3 fazlı inverter kartında yanlış sinyal MOSFET veya sürücü IC yakar. Mantığı önce yazılımda kurup GPIO pinlerinden multimetre ile doğrulamak, en ucuz hata ayıklama aşamasıdır.

# BLDC Motorun Çalışma Mantığı

## Fırçalı DC motor

Komütasyonu fırçalar mekanik temasla, kendiliğinden yapar. Kolay ama fırçalar aşınır, kıvılcım ve gürültü üretir.

## BLDC (fırçasız)

Rotor kalıcı mıknatıs, stator 3 sargı (A, B, C). Fırça yok — hangi sargının ne zaman enerjileneceğine yazılım karar verir. Komütasyon artık bizim sorumluluğumuzda.

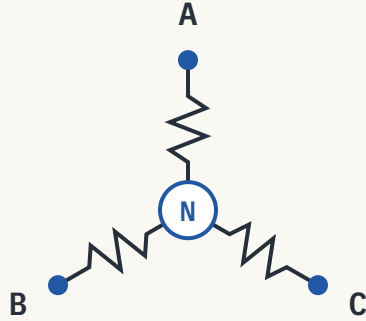
Rotor = dönen mıknatıs · Stator = sabit A, B, C sargıları

## SOMUT DÜŞÜN: SALINCAK

Rotor mıknatısı salıncaktaki çocuk, sargılar iten eller.

Doğru anda, doğru elle itersen salıncak düzenli döner; erken ya da geç itersen yavaşlar, hatta durur. 6 adımlı komütasyon, bu "doğru anda itme" sırasının ta kendisidir.

# Y Baęlantılı Stator ve Temel Kavramlar



Üç sargının bir ucu ortak nötr (N) noktasında buluşur; A, B, C uçları birer half-bridge çıkışına gider.

## Sargı (faz) A/B/C

Statordeki 3 elektrik hattı; her biri bir half-bridge'e bağlanır

## Nötr noktası (N)

3 sargının ortak iç noktası; gerçek veya sanal olarak ölçülebilir

## Elektriksel derece

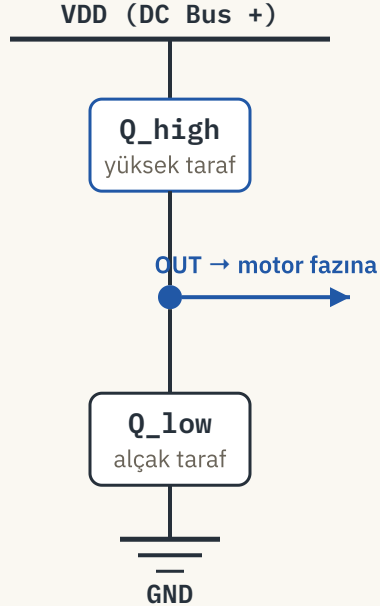
Bir komütasyon döngüsü =  $360^\circ$  elektriksel; kutup sayısına göre mekanik dönüşten farklı olabilir

## Trapezoidal komütasyon

Her an yalnız 2 faz sürülür (biri yüksek, biri düşük), 3. faz boşta — "6 adım" yöntemi

FOC (vektör kontrolü) çok daha pürüzsüzdür ama Clarke/Park dönüşümleri ister — önce bu temeller oturmalı.

# Half-Bridge: En Küçük Yapı Taşı



**YÜKSEK**

Q\_high ON, Q\_low OFF → OUT = VDD (kaynak akım)

**DÜŞÜK**

Q\_low ON, Q\_high OFF → OUT = GND (akım yutar)

**FLOAT**

İkisi de OFF → OUT yüksek empedans (Hi-Z) — BEMF burada ölçülür

**Somut düşün:** Üç konumlu bir anahtar — yukarı = artı uca, aşağı = eksiye, orta = hiçbir yere bağlı değil (boşta).

Hız/tork kontrolü Q\_high'ı PWM ile anahtarlayarak yapılır: duty ne kadar yüksekse faza uygulanan ortalama gerilim o kadar büyük — PC13 LED parlaklık ayarıyla aynı prensip.

# Shoot-Through Tehlikesi ve Dead-Time

## Shoot-through nedir?

Q\_high ve Q\_low aynı anda ON olursa, VDD'den GND'ye — motor sargısına hiç uğramayan — doğrudan bir kısa devre yolu açılır. Akımı sınırlayan hiçbir şey yoktur; MOSFET'ler saniyeler içinde aşırı akımla yanar.

**Somut düşün:** pilin + ve – ucunu kalın bir telle birleştirmek gibi — enerji işe değil, ısıya gider.

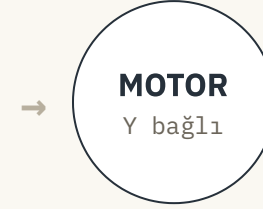
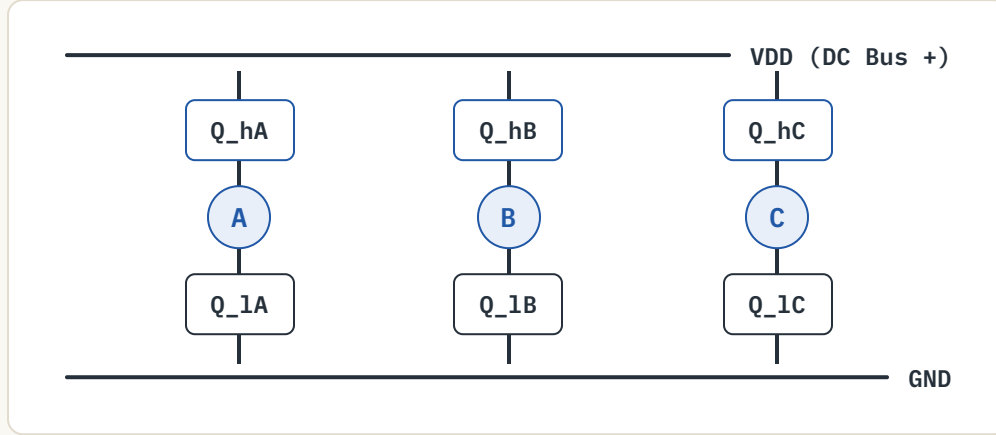
Q\_high

Q\_low

↑ dead-time – ikisi de OFF, güvenli geçiş

**STM32'de:** TIM1 (advanced-control) complementary PWM + donanımsal dead-time (BDTR / DTG) üretir. TIM2 bunu yapamaz — gerçek motor sürüşünde TIM1 şart.

# Üç Fazlı İnverter: 3 Half-Bridge, 6 MOSFET



Her sütun, bir önceki slayttaki half-bridge'in aynısı.  
Üçü de aynı DC bus'ı (VDD / GND) paylaşır:  $3 \times 2 = 6$  MOSFET.

Gerçek donanıma geçince: 6 MOSFET veya entegre sürücü (örn. DRV8305/DRV8353), gate sürme, koruma, TIM1 dead-time. **Şimdilik hiçbirisi gerekmiyor — FAZ 1 tamamen yazılımda.**

## 6 ADIM

# 6 Adımlı Komütasyon Tablosu

| ADIM   | FAZ A  | FAZ B  | FAZ C  |
|--------|--------|--------|--------|
| STEP 1 | YÜKSEK | DÜŞÜK  | FLOAT  |
| STEP 2 | YÜKSEK | FLOAT  | DÜŞÜK  |
| STEP 3 | FLOAT  | YÜKSEK | DÜŞÜK  |
| STEP 4 | DÜŞÜK  | YÜKSEK | FLOAT  |
| STEP 5 | DÜŞÜK  | FLOAT  | YÜKSEK |
| STEP 6 | FLOAT  | DÜŞÜK  | YÜKSEK |

Her adımda **tam 1 faz FLOAT** kalır — float faz sırayla  $C \rightarrow B \rightarrow A \rightarrow C \rightarrow B \rightarrow A$  döner.

Ardışık adımlar arasında **yalnız 2 fazın durumu değişir**; üçüncüsü sabit kalır.

STEP 6'dan sonra STEP 1'e dönülür: **6 adım = 360° elektriksel** = bir tam döngü.

Yön (CW/CCW) sargı bağlantısına göre değişebilir; buradaki atamalar örnek kabuldür — önemli olan desen.

# Komütasyon Döngüsü: 360° Elektriksel



A

FLOAT

B

YÜKSEK

C

DÜŞÜK

Tabloyu ezberleme, deseni gör: aktif çift saat yönünde kayar, boşta kalan faz her adımda bir sonrakine devreder.

# GPIO ile Üç Durumun Taklidi

## YÜKSEK

`GPIO_MODE_OUTPUT_PP`  
`WritePin(..., SET)`

Pin  $\approx 3.3V$ , kaynak akım.  
Q\_high ON durumunu temsil eder.

## DÜŞÜK

`GPIO_MODE_OUTPUT_PP`  
`WritePin(..., RESET)`

Pin  $\approx 0V$ , akım yutar.  
Q\_low ON durumunu temsil eder.

## FLOAT

`GPIO_MODE_INPUT`  
`Pu11 = GPIO_NOPULL`

Pin yüksek empedans (Hi-Z).  
İki MOSFET'in de OFF halini temsil eder.

**Somut düşün:** FLOAT, fişi çekilmiş kablo gibidir — ne artıya ne eksiye bağlı, sadece boşta.

Böylece "kâğıt üzerinde doğru" değil, **multimetreyle pinden ölçülebilir** bir sinyal üretirsin — ileride aynı 3 sinyal gerçek gate driver IC'sine gidecek.

# Enum ve Arama Tablosu

```
typedef enum {  
    PHASE_FLOAT = 0,  
    PHASE_LOW,  
    PHASE_HIGH  
} PhaseState_t;  
  
typedef enum {  
    MOTOR_STEP_1 = 0,  
    /* ... MOTOR_STEP_6'ya kadar */  
} MotorStep_t;  
  
const PhaseState_t commutation_table[6][3] = {  
    { PHASE_HIGH,  PHASE_LOW,  PHASE_FLOAT }, // STEP 1  
    { PHASE_HIGH,  PHASE_FLOAT, PHASE_LOW  }, // STEP 2  
    { PHASE_FLOAT, PHASE_HIGH,  PHASE_LOW  }, // STEP 3  
    { PHASE_LOW,   PHASE_HIGH,  PHASE_FLOAT }, // STEP 4  
    { PHASE_LOW,   PHASE_FLOAT, PHASE_HIGH  }, // STEP 5  
    { PHASE_FLOAT, PHASE_LOW,   PHASE_HIGH  }, // STEP 6  
};
```

## Nasıl okunur?

Satır = adım, sütun = faz (0: A, 1: B, 2: C). Bu tablo, bir önceki slayttaki tablonun birebir C karşılığıdır.

## Tanıdık desen

LED projesindeki LedState\_t state machine'inin aynısı — sadece 4 yerine 6 durum ve her durumda 3 fazlık hedef var.

## Henüz yazma

Kod tamamen kavramsal. main.c'ye eklemeyen önce GPIO pin seçimi, CubeMX ayarları ve geçiş timer'ı birlikte planlanmalı.

# Zamanlama ve Donanımsız Doğrulama

**TIM kesmesi** — örn. her 10 ms'de bir tetiklenir ("elektriksel hız")



```
current_step = (current_step + 1) % 6;
```



**commutation\_table[current\_step]** satırını oku → 3 GPIO'yu YÜKSEK / DÜŞÜK / FLOAT yap

Adım süresi kısaldıkça "motor" hızlanır — donanım yokken bu yalnızca simülasyon hızıdır.

## DOĞRULAMA YÖNTEMLERİ

### Debugger watch

current\_step ve tablo satırı adım adım izlenir

### PC13 blink-code

LED her adımda adım numarası kadar kısa yanıp söner

### Multimetre / osiloskop

Pinde gerçekten 3.3V / 0V / Hi-Z görüldüğü doğrulanır

### UART

En rahat ve kalıcı yöntem — sıradaki bölüm: FAZ 2

# UART ile Durum Telemetrisi

## CUBEMX AYARLARI

|                                        |        |
|----------------------------------------|--------|
| Periferik                              | USART2 |
| TX pini                                | PA2    |
| RX pini (yalnız gönderimde şart değil) | PA3    |
| Baud rate                              | 115200 |
| Çerçeve                                | 8N1    |

Black Pill'de dahili USB-seri köprüsü yok — harici USB-TTL adaptör gerekir (CP2102, CH340, FT232). Bonus: `_write()` fonksiyonunu `HAL_UART_Transmit`'e yönlendirirsen `printf` doğrudan UART'a yazar.

### SERİ PORT ÇIKTISI — HER ADIM DEĞİŞİMİNDE BİR SATIR

STEP:1 A:HIGH B:LOW C:FLOAT

STEP:2 A:HIGH B:FLOAT C:LOW

STEP:3 A:FLOAT B:HIGH C:LOW

...

Protokol karmaşık olmak zorunda değil: insan okuyabilsin, yeter. Faz 3'te ayrıştırmayı kolaylaştırmak istersen CSV alternatifi: **1,H,L,F**

# Kesme Kuralı: Bayrak Deseni

## TIM ISR – KISA TUT

### Adımı ilerlet + bayrağı set et

UART gönderimi YOK

`main while(1)`

### Bayrağı kontrol eder

Set ise gönderime geçer

`HAL_UART_Transmit`

### Durum satırını gönderir

Bloklasa da kesmeleri engellemez

**Neden?** HAL\_UART\_Transmit varsayılan olarak blocking'dir — kesme içinde çağrılırsa ISR'ı milisaniyelerce kilitler, diğer kesmeler gecikir.

**Somut düşün:** kapı zili çaldığında kapıda iş yapılmaz — sadece "geldi" notu düşülür, paket sonra alınır.

**İleri seviye:** HAL\_UART\_Transmit\_IT veya \_DMA — CPU hiç bloklanmaz. İlk sürüm için bayrak deseni yeterli.

# PC Tarafında İzleme Arayüzü

1

## Terminal programı — PuTTY, Tera Term, VS Code Serial Monitor

Ham metni gör, protokolün doğru çalıştığını onayla

ÇOK KOLAY

2

## Python + pyserial — konsola yazdırma

Satırları oku, ayrıştır (parse), ayrıştırılmış hâlde konsola yaz

KOLAY

3

## Python GUI — tkinter / PyQt

6 dairesel canlı gösterim: bu sunumdaki döngü diyagramının gerçek zamanlı hâli

ORTA

4

## Web tabanlı — yerel sunucu + WebSocket + HTML, ya da STM32CubeMonitor

Tarayıcıda görselleştirme; hazır alternatif olarak CubeMonitor

ORTA-İLERİ

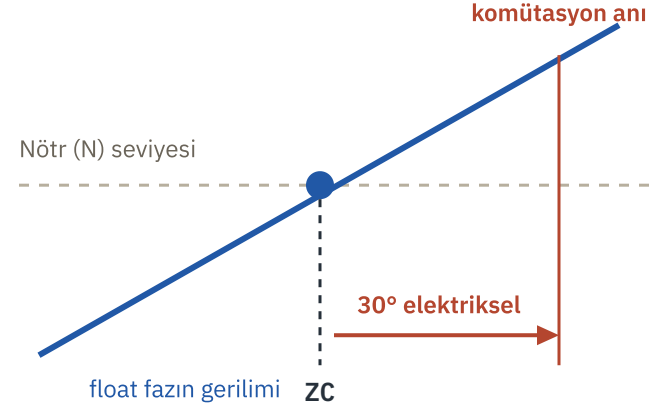
Önerilen sıra yukarıdan aşağı: her katman bir öncekinin doğru çalıştığını kanıtlar.

## BEMF Sıfır Geçişi

Float bırakılan faz pasif değildir: dönen rotor mıknatısları o sargıda bir gerilim indükler — **Back-EMF**. Bu gerilim, adımın ortasına yakın nötr (N) seviyesini keser: **sıfır geçişi (ZC)**.

ZC'yi yakaladığın an, bir sonraki komütasyonun tam **30° elektriksel** sonra gelmesi gerektiğini bilersin — motor kendi konumunu kendisi söyler.

**Somut düşün:** dönen motor aynı zamanda bir dinamodur. Boştaki sargı, rotorun nerede olduğunu fısıldayan bedava bir sensöre dönüşür — ek donanım yok, sinyal motorun kendisinden.



# Sensörsüz Sürüşün Gereksinimleri

## TIM1

Advanced-control timer: complementary PWM kanalları + donanımsal dead-time (BDTR/DTG) — TIM2'nin yapamadığı iş.

## Senkronize ADC

Float fazın gerilimini anahtarlama gürültüsünden uzak, TIM1 tetiklemesiyle tam doğru anda örneklemek.

## Blanking (maskeleme)

Her komütasyondan hemen sonra ZC tespitini kısa süre yok saymak — anahtarlama çınlaması (ringing) yanlış tetiklemesin.

## Startup ramp

Duran motorda BEMF = 0. Önce açık çevrim, artan hızlı komütasyonla döndür; yeterli BEMF oluşunca sensörsüz moda geç.

**Bu aşama şimdi değil:** Faz 1-2-3'ü tamamlamadan buraya atlama — 6 adım ve half-bridge mantığını sağlam oturtmak, BEMF algılamayı anlamanın ön koşulu.

## ÖZET

# Hızlı Tekrar

- ✓ Half-bridge = 2 MOSFET; 3 çıkış durumu: YÜKSEK / DÜŞÜK / FLOAT.
- ✓ 3 fazlı inverter = 3 half-bridge = 6 MOSFET, ortak DC bus.
- ✓ GPIO OUTPUT\_PP ↔ INPUT (NOPULL) geçişiyle 3 durum donanımsız taklit edilir.
- ✓ FAZ 2 = USART2 (PA2/PA3), 115200-8N1; ISR bayrak set eder, gönderim main'de yapılır.
- ✓ Shoot-through riski → dead-time şart; TIM1 (BDTR/DTG) donanımda üretir, TIM2 üretmez.
- ✓ 6 adım: her adımda 1 faz YÜKSEK, 1 DÜŞÜK, 1 FLOAT; float faz C→B→A sırasıyla döner.
- ✓ FAZ 1 = enum + commutation\_table[6][3] + timer; doğrulama: watch, blink-code, multimetre.
- ✓ FAZ 3 = terminal → pyserial → GUI; ileri seviye = BEMF ZC + 30° gecikme.

# Quiz: Kavram Soruları I

## SORU 1

Q\_high ve Q\_low aynı anda ON olursa ne olur?

- A Motor daha hızlı döner
- B VDD'den GND'ye kısa devre oluşur; MOSFET'ler yanabilir
- C Faz çıkışı FLOAT durumuna geçer

## SORU 2

6 adımlı komütasyonda her adımda kaç faz aktif sürülür?

- A 2 — biri YÜKSEK, biri DÜŞÜK; üçüncüsü boşta
- B 3 — üç faz da her an enerjili
- C 1 — her seferinde tek sargı

## Quiz: Kavram Soruları II

SORU 3

**FLOAT durumunu GPIO ile nasıl taklit edersin?**

- A** GPIO\_MODE\_INPUT + Pull = NOPULL
- B** OUTPUT\_PP + WritePin(..., RESET)
- C** INPUT + Pull-up direnci aktif

SORU 4

**Gerçek motor sürüşünde neden TIM2 değil TIM1 gerekir?**

- A** TIM1'in kesme önceliği daha yüksektir
- B** TIM1 daha hızlı sayabilir
- C** Complementary PWM + donanımsal dead-time yalnız TIM1'de var

## Quiz: Kod Okuma I

### KOD 1

```
current_step = (current_step + 1) % 6;
```

- A Adımı 1 ilerletir; 5'ten sonra 0'a sarar (STEP 6 → STEP 1)
- B current\_step'i 6 ile çarpar, kalanı atar
- C Adımı rastgele bir değere ayarlar

### KOD 2

```
PhaseState_t s =  
    commutation_table[current_step][2];
```

- A Faz B'nin hedef durumunu okur
- B Bu adımda Faz C'nin hedef durumunu okur
- C STEP 2 satırının tamamını okur

## Quiz: Kod Okuma II

### KOD 3 – BİR TIMER ISR'ININ İÇİNDE

```
HAL_UART_Transmit(&huart2, msg,  
                  len, HAL_MAX_DELAY);
```

Buradaki sorun ne?

- A** UART kesme içinde donanımsal olarak çalışamaz
- B** Blocking'dir: gönderim bitene dek ISR kilitli kalır, diğer kesmeler gecikir
- C** Baud rate otomatik olarak düşer

### KOD 4 – PİN OUTPUT\_PP MODUNDA

```
HAL_GPIO_WritePin(GPIOA, FAZ_A_Pin,  
                  GPIO_PIN_SET);
```

Bu çağrı ne yapar?

- A** Pini  $\approx 3.3V$ 'a çeker — half-bridge'de Q\_high ON (YÜKSEK) taklidi
- B** Pini Hi-Z (FLOAT) durumuna alır
- C** Pini GND'ye çeker (DÜŞÜK)

KAPANIŞ

## Sonraki Adım

1 CubeMX planı: hangi 3 GPIO pini Faz A / B / C olacak

2 Adım timer'ının seçimi ve yapılandırması — LED fade projesindeki TIM2 deseni

3 main.c USER CODE bölümlerine birlikte ekleme — LED/buton projesindeki akışın aynısı

Önce mantık → sonra telemetri → sonra görselleştirme. Güç elektroniği en sona.

# F401 — Uygulama Görüntüleri

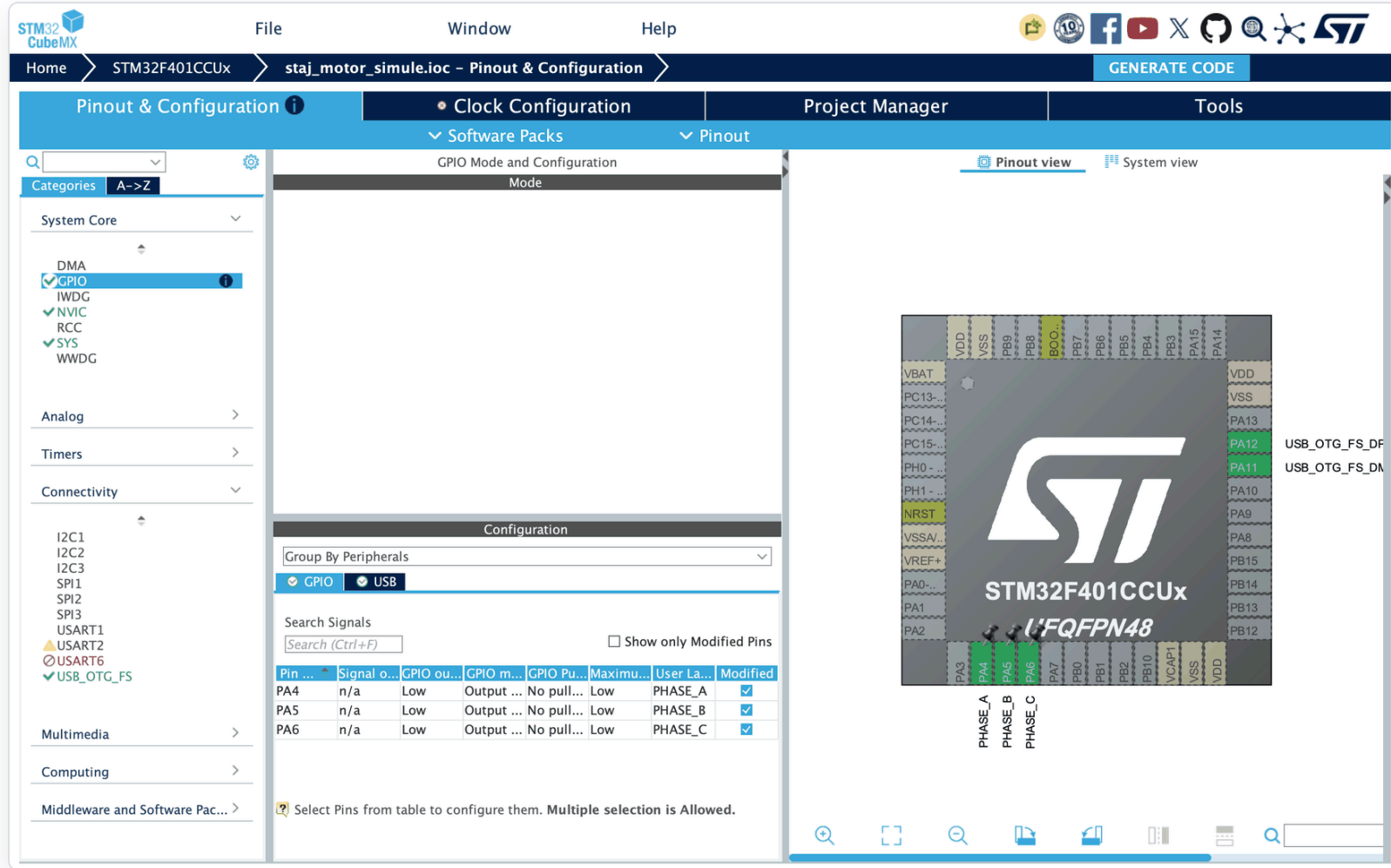
STM32F401CCU6 üzerinde donanımsız komütasyon simülasyonuna ait ekran görüntüleri: pin ve saat yapılandırması, kodun karta yazılması ve seri porttan komütasyon çıktısının izlenmesi. Bu aşamada gerçek motor yoktur; altı adımlı sıra üç GPIO pininde ve UART'ta izlenir.

---

## BU BÖLÜMDEKİ GÖRÜNTÜLER

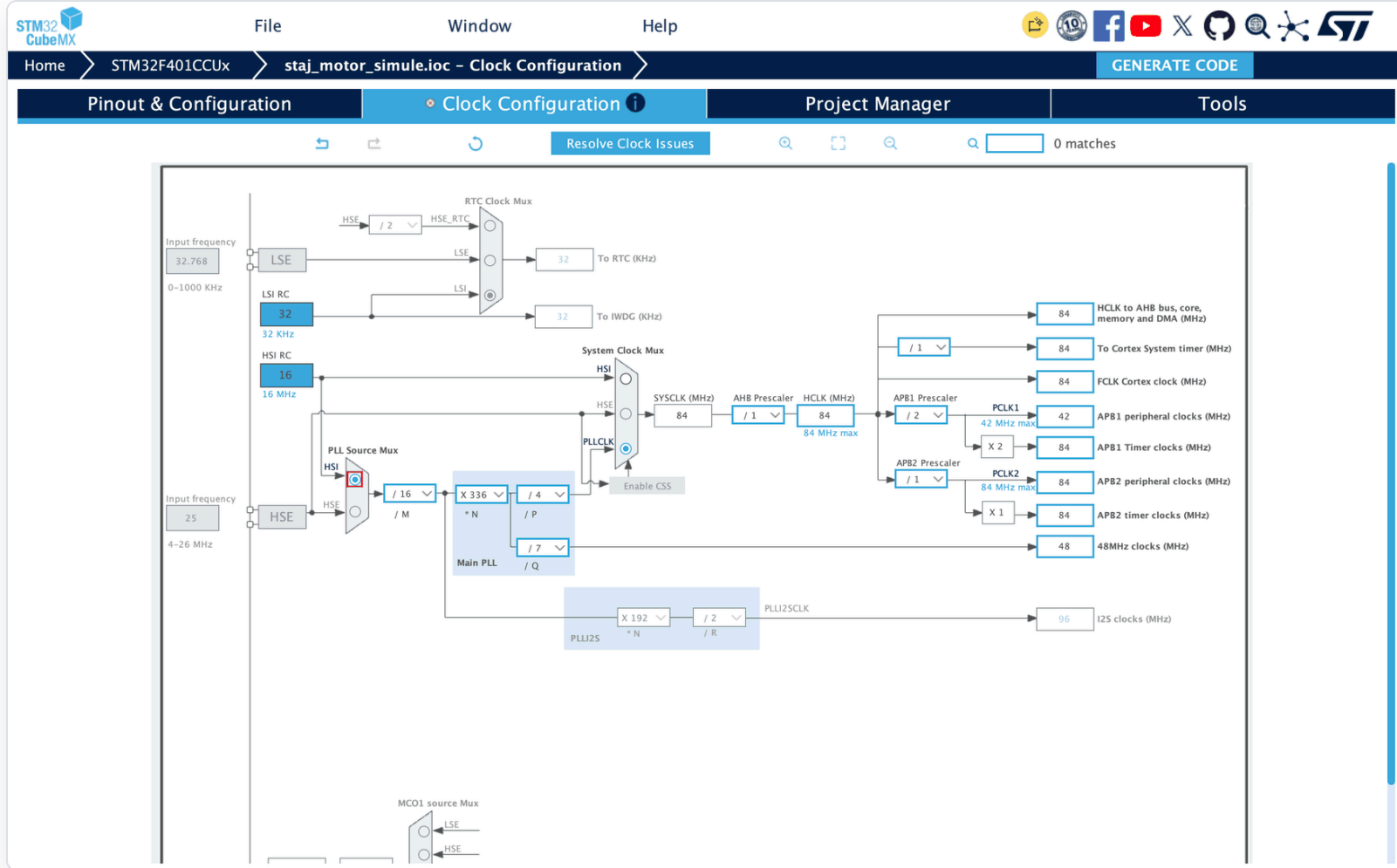
- 1 CubeMX — Pinout: PA4/PA5/PA6 = PHASE\_A/B/C
- 2 CubeMX — Clock: HSE 25 MHz → 84 MHz
- 3 Kod, karta yazma ve CoolTerm STEP çıktısı

# CubeMX • Pinout & Configuration



**ŞEKİL 1** STM32F401CCUx pin ataması. PA4, PA5, PA6 pinleri GPIO çıkışı olarak **PHASE\_A / PHASE\_B / PHASE\_C** kullanıcı etiketleriyle yapılandırıldı — üç fazın altı adımlı komütasyonu bu üç pinden sürülür.

# CubeMX • Clock Configuration



**ŞEKİL 2** Saat ağacı: dış **HSE 25 MHz** kaynağı PLL üzerinden **84 MHz** sistem saatine (SYSCLK) çıkarıldı. Komütasyon adım süresinin doğru olması için saat frekansının bilinmesi gerekir.

## Kod • Karta Yazma • Seri Çıktı

The screenshot displays a development environment with three main panels:

- Left Panel (Code Editor):** Shows the `main.c` file with a 6-step `commutation_table` defined in the `Set_Phase` function. The code includes comments in Turkish and C for setting motor phases.
- Middle Panel (Terminal):** Shows the output of the `STM32CubeProgrammer` v2.23.0. It reports successful erasing and programming of sectors 1 and 2, including SHA-256 verification and device information (STM32F411xC/E, 512 KB NVM).
- Right Panel (Serial Monitor):** Shows the `CoolTermSettings` window with a live stream of the motor's step sequence (STEP...H...L...F...L...H...) and status indicators (TX, RX, RTS, CTS).

**ŞEKİL 3** Solda `main.c` içindeki 6 adımlık `commutation_table`; ortada `STM32CubeProgrammer` ile sektör silme + doğrulama (SHA-256) ve uygulamanın `0x08000000`'dan başlatılması; sağda `CoolTerm`'de her adımda değişen faz durumlarının (STEP) canlı metin akışı.

# NUCLEO-G491RE üzerinde 3 fazlı BLDC komutasyon simülasyonu

Neyi, neden, hangi sırayla yaptığımı adım adım anlatan tekrar notu.

CubeMX → CubeIDE → ST-LINK → CoolTerm zinciri.

| PROJE                  | MCU           | SERİ HAT      | HIZ          |
|------------------------|---------------|---------------|--------------|
| staj_motor_simule_g491 | STM32G491RET6 | LPUART1 / VCP | 115200 8-N-1 |

## İÇİNDEKİLER

1. Amaç ve güvenlik sınırı
2. Neden Nucleo?
3. CubeMX: proje kurulumu
4. CubeIDE: kodun mantığı
5. Derleme ve karta yazma
6. CoolTerm: metin çıktısı
7. CoolTerm Chart: grafik
8. Kontrol listesi
9. Sonraki adımlar
10. Terimler sözlüğü
11. Kendi izleme uygulamamız: CoolTerm'ün ötesi
12. Ek - main.c satır satır

## 01 Amaç ve güvenlik sınırı

Bu uygulama gerçek bir BLDC motoru döndürmez. Üç GPIO pini üzerinde BLDC'nin altı adımlı (six-step) komutasyon sırasını canlandırır. Her 200 ms sonunda A, B ve C fazının durumu değişir; aynı bilgi LPUART1 ile bilgisayara gönderilir ve CoolTerm'de metin veya grafik olarak izlenir.

? **Neden gerçek motor değil?** PA4, PB4 ve PB5 yalnızca 3.3 V mantık çıkışıdır. Motor fazı veya MOSFET kapısı bu pinlere doğrudan bağlanmaz. Gerçek sistemde 3-faz inverter, gate driver, akım ölçümü ve PWM gerekir. Bu çalışma yalnızca komutasyon mantığını güvenli biçimde öğretir.

```
Komutasyon tablosu → GPIO A/B/C → LPUART1 TX  
→ ST-LINK Virtual COM Port → CoolTerm
```

## 02 Neden Nucleo kartı işi kolaylaştırır?

Kart üzerindeki ST-LINK V3E iki ayrı iş yapar: **SWD** ile firmware yazma ve hata ayıklama, **Virtual COM Port (VCP)** ile UART verisini USB üzerinden Mac'e taşıma. Bu yüzden ek USB-UART dönüştürücü ve DFU modu gerekmez.

Mac + CoolTerm

| micro-USB veri kablosu



NUCLEO üzerindeki ST-LINK V3E

| SWD ◀▶ PA13 (SWDIO), PA14 (SWCLK) : programlama / debug

| VCP ◀▶ PA2 (LPUART1\_TX), PA3 (LPUART1\_RX) : UART



STM32G491RET6

? **Neden Board Selector ile kart seçtik, sadece MCU değil?** CubeMX kart şablonunda ST-LINK ve VCP bağlantılarını (PA2/PA3) tanır. Yalnız MCU seçilirse bu fiziksel bağlantıları kendin doğrulaman gerekir.

## 03 STM32CubeMX ile sıfırdan proje

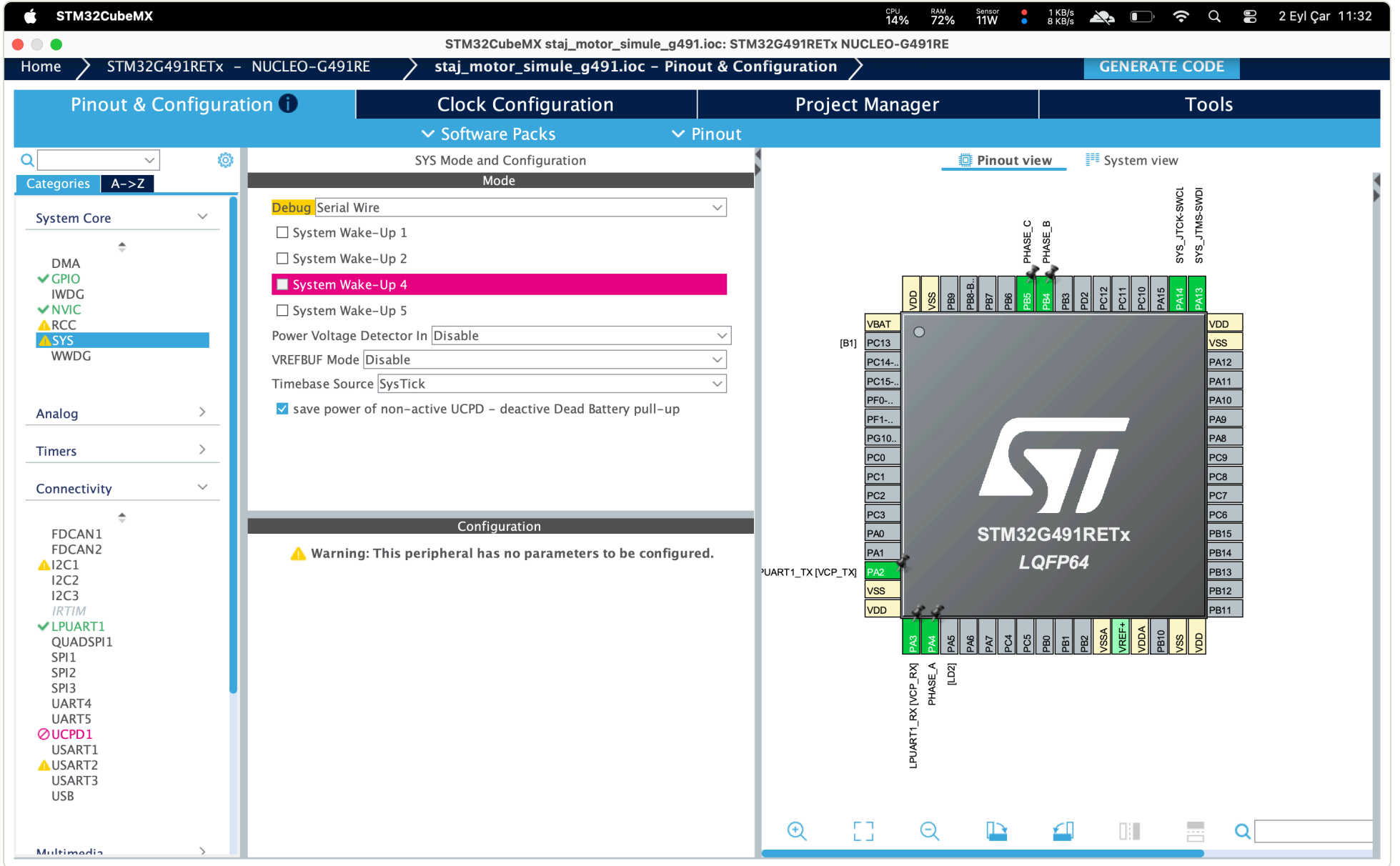
### 3.1 Kart seçimi

1. STM32CubeMX'i aç, **Access to Board Selector** seç.
2. Arama alanına NUCLEO-G491RE yaz.
3. Kartı seç, **Start Project**.

? **LPUART alanı gri veya kilitli görünürse?** Kart şablonu bazı pinleri önceden ayırmış olabilir. Pinout menüsünden Clear Pinout / Reset Configuration ile temizleyip ayarları yeniden yap. Mevcut projede rastgele yapma; önceki pin ayarlarını siler.

### 3.2 SYS: Serial Wire debug

**System Core** > **SYS** ekranında Debug = Serial Wire seçilir. Timebase Source SysTick olarak kalır.



Şekil 1 · SYS > Debug = Serial Wire. Sağdaki pinout'ta PA13/PA14 SWD için ayrılmış (yeşil).

? **Neden Serial Wire?** SWDIO (PA13) ve SWCLK (PA14) pinlerini ST-LINK programlayıcısına ayırır. "No Debug" seçilirse tekrar programlama gereksiz derecede zorlaşır. Bu iki pini başka GPIO için kullanma.

### 3.3 LPUART1: bilgisayara seri veri

**Connectivity** > **LPUART1** ekranında:

| Alan                        | Değer                              |
|-----------------------------|------------------------------------|
| Mode                        | Asynchronous                       |
| Baud Rate                   | 115200                             |
| Word Length / Parity / Stop | 8 Bits / None / 1                  |
| Hardware Flow Control       | Disable                            |
| Pinler                      | PA2 = LPUART1_TX, PA3 = LPUART1_RX |

STM32CubeMX

FileWindowHelp

Home

STM32G491RETx - NUCLEO-G491RE

staj\_motor\_simule\_g491.ioc - Pinout & Configuration

GENERATE CODE

Pinout & Configuration

Clock Configuration

Project Manager

Tools

Software Packs

Pinout

Categories

A->Z

System Core

DMA

GPIO

IWDG

NVIC

RCC

SYS

WWDG

Analog

Timers

Connectivity

FDCAN1

FDCAN2

I2C1

I2C2

I2C3

IRTIM

LPUART1

QUADSPI1

SPI1

SPI2

SPI3

UART4

UART5

UCPD1

USART1

USART2

USART3

USB

Multimedia

LPUART1 Mode and Configuration

Mode

Asynchronous

Hardware Flow Control (RS232)

Disable

Hardware Flow Control (RS485)

LPBAM Advanced Settings

Configuration

Reset Configuration

NVIC Settings

DMA Settings

GPIO Settings

Parameter Settings

User Constants

NVIC Interrupt Table

Add

Preemption Priority

Sub Priority

LPUART1 global interrupt

0

0

Pinout view

System view

VBAT

PC13

PC14...

PC15...

PF0...

PF1...

PG10...

PC0

PC1

PC2

PC3

PA0

PA1

PA2

VDD

PA3

PA4

PA5

PA6

PA7

PC4

PC5

PB0

PB1

PB2

VSSA

VREF+

VDDA

PB10

VSS

VDD

PHASE\_C

PHASE\_B

PHASE\_A

LD2

UART1\_TX [VCP\_TX]

UART1\_RX [VCP\_RX]

PA12

PA11

PA10

PA9

PA8

PC9

PC8

PC7

PC6

PB15

PB14

PB13

PB12

PB11

Şekil 2 · LPUART1 Asynchronous, flow control kapalı. PA2 [VCP\_TX] ve PA3 [VCP\_RX] ST-LINK'e gidiyor.

? **Neden 115200 8-N-1?** Terminal uygulamalarında yaygın ve bu proje için yeterince hızlı. 8 veri biti, parity yok, 1 stop biti. CoolTerm tarafında da aynısı seçilmelidir. CTS/RTS kullanmadığımız için flow control kapalı.

? **Henüz komut almıyorsak RX'i neden ayarladık?** TX, STM32'nin gönderdiği hat; STEP satırları PA2'den çıkar. PA3 (RX) şimdilik boş, ama ileride bilgisayardan başlat/durdur/hız komutu göndermek için altyapı hazır olur.

### 3.4 Üç faz GPIO çıkışı

| Pin | User Label | GPIO ayarı                                |
|-----|------------|-------------------------------------------|
| PA4 | PHASE_A    | Output Push Pull, Low, No pull, Low speed |
| PB4 | PHASE_B    | Output Push Pull, Low, No pull, Low speed |
| PB5 | PHASE_C    | Output Push Pull, Low, No pull, Low speed |

FileWindowHelp

Home

STM32G491RETx - NUCLEO-G491RE

staj\_motor\_simule\_g491.ioc - Pinout & Configuration

GENERATE CODE

Pinout & Configuration

Clock Configuration

Project Manager

Tools

Software Packs

Pinout

Categories

A-Z

System Core

DMA

GPIO

IWDG

NVIC

RCC

SYS

WWDG

Analog

Timers

Connectivity

FDCAN1

FDCAN2

I2C1

I2C2

I2C3

IRTIM

LPUART1

QUADSPI1

SPI1

SPI2

SPI3

UART4

UART5

UCPD1

USART1

USART2

USART3

USB

GPIO Mode and Configuration

Mode

Configuration

Group By Peripherals

GPIO LPUART SYS

Search Signals

Search (Ctrl+F)

Show only Modified Pins

| Pin ... | Signal ... | GPIO o... | GPIO ... | GPIO P... | Maxim... | Fast M... | User L... | Modified                            |
|---------|------------|-----------|----------|-----------|----------|-----------|-----------|-------------------------------------|
| PA4     | n/a        | Low       | Outpu... | No pul... | Low      | n/a       | PHASE...  | <input checked="" type="checkbox"/> |
| PB4     | n/a        | Low       | Outpu... | No pul... | Low      | n/a       | PHASE_B   | <input checked="" type="checkbox"/> |
| PB5     | n/a        | Low       | Outpu... | No pul... | Low      | n/a       | PHASE...  | <input checked="" type="checkbox"/> |

Select Pins from table to configure them. Multiple selection is Allowed.

Pinout view

System view

Şekil 3 · GPIO tablosunda PA4/PB4/PB5 → PHASE\_A/B/C. Pinout'ta üstte PB4/PB5, altta PA4 yeşil.

? **Neden bu üç pin?** Bu projede boş olan üç bağımsız GPIO. PA5 karttaki LD2 kullanıcı LED'i, o yüzden faz pini olarak seçilmedi.

? **Neden Push Pull ve başlangıç Low?** Push Pull, pin HIGH iken aktif 3.3 V, LOW iken aktif GND sürer. Başlangıcın Low olması reset sonrası güvenli durum verir. Low speed bu yavaş simülasyon için yeterli.

### 3.5 Clock ve kod üretimi

CubeMX'in ürettiği yapı HSI ve PLL ile 170 MHz sistem saatini kullanır. Clock Configuration ekranında elle değişiklik gerekmez.

```
HSI          = 16 MHz
PLL girişi   = 16 / 4 = 4 MHz      (PLLM = DIV4)
VCO          = 4 × 85 = 340 MHz   (PLLN = 85)
SYSCLK       = 340 / 2 = 170 MHz  (PLLR = DIV2)
```

**Project Manager:** Project Name `staj_motor_simule_g491`, Toolchain STM32CubeIDE, Firmware STM32Cube FW\_G4, **Keep User Code** etkin. Sonra **GENERATE CODE**.

? **Keep User Code neden açık?** .ioc değişip tekrar Generate Code yapıldığında USER CODE BEGIN/END bloklarındaki uygulama kodu korunur. Kapalıysa yazdığın kod silinir.

## 04 STM32CubeIDE: kodun çalışma mantığı

Kod Core/Src/main.c içindedir. Eklenen kod yalnızca USER CODE bloklarına konur; CubeMX'in oluşturduğu MX\_GPIO\_Init() ve MX\_LPUART1\_UART\_Init() silinmez.



## 4.1 Faz durumları

```
typedef uint8_t PhaseState_t;
enum { PHASE_FLOAT = 0U, PHASE_LOW = 1U, PHASE_HIGH = 2U };
```

- **PHASE\_FLOAT**: pin sürülmez, input / yüksek empedans.
- **PHASE\_LOW**: pin aktif 0 V (lojik 0).
- **PHASE\_HIGH**: pin aktif 3.3 V (lojik 1).

? **FLOAT ile LOW aynı şey değil mi?** Değil. LOW aktif olarak GND'ye çeker; FLOAT hiç sürmez. Gerçek motor sürücüsünde FLOAT, ilgili faz kolunun tamamen kapatılmasına (her iki MOSFET off) karşılık gelir.

## 4.2 Altı adımlı komutasyon tablosu

| Adım | PHASE_A   | PHASE_B   | PHASE_C   |
|------|-----------|-----------|-----------|
| 1    | HIGH (+1) | LOW (-1)  | FLOAT (0) |
| 2    | HIGH (+1) | FLOAT (0) | LOW (-1)  |
| 3    | FLOAT (0) | HIGH (+1) | LOW (-1)  |
| 4    | LOW (-1)  | HIGH (+1) | FLOAT (0) |
| 5    | LOW (-1)  | FLOAT (0) | HIGH (+1) |
| 6    | FLOAT (0) | LOW (-1)  | HIGH (+1) |

```
#define MOTOR_STEP_COUNT      6U
#define MOTOR_STEP_PERIOD_MS  200U

static const uint8_t commutation_table[MOTOR_STEP_COUNT] = {
    PACK_PHASES(PHASE_HIGH,  PHASE_LOW,  PHASE_FLOAT),
    PACK_PHASES(PHASE_HIGH,  PHASE_FLOAT, PHASE_LOW),
    PACK_PHASES(PHASE_FLOAT, PHASE_HIGH,  PHASE_LOW),
    PACK_PHASES(PHASE_LOW,   PHASE_HIGH,  PHASE_FLOAT),
    PACK_PHASES(PHASE_LOW,   PHASE_FLOAT, PHASE_HIGH),
    PACK_PHASES(PHASE_FLOAT, PHASE_LOW,   PHASE_HIGH)
};
```

? **Neden her adımda bir HIGH, bir LOW, bir FLOAT?** Six-step komutasyonda akım her an yalnızca iki sargıdan geçer; üçüncü sargı serbest bırakılır (gerçek motorda back-EMF ölçümü buradan yapılır). Altıncı adımdan sonra tekrar birinciye dönülür.

? **PACK\_PHASES ne işe yarıyor?** Üç faz durumunu ikişer bit halinde tek byte'ta saklar (3 durum için 2 bit yeter). UNPACK\_PHASE seçilen adımdan A/B/C bilgisini geri ayırır. Tablo küçük ve okunur kalır.

### 4.3 Set\_Phase: HIGH, LOW ve FLOAT uygulanması

```
static void Set_Phase(GPIO_TypeDef *port, uint16_t pin, PhaseState_t state)
{
    GPIO_InitTypeDef gpio = {0};
    gpio.Pin    = pin;
    gpio.Pull   = GPIO_NOPULL;
    gpio.Speed  = GPIO_SPEED_FREQ_LOW;

    if (state == PHASE_FLOAT) {
        gpio.Mode = GPIO_MODE_INPUT;           // sürme yok
        HAL_GPIO_Init(port, &gpio);
    } else {
        gpio.Mode = GPIO_MODE_OUTPUT_PP;
        HAL_GPIO_Init(port, &gpio);
        HAL_GPIO_WritePin(port, pin,
            (state == PHASE_HIGH) ? GPIO_PIN_SET : GPIO_PIN_RESET);
    }
}
```

? **Neden her çağrıda HAL\_GPIO\_Init tekrar çağrılıyor?** FLOAT için pinin modunu Input'a, HIGH/LOW için Output'a çevirmek gerekiyor. Sadece WritePin ile FLOAT üretilemez; mod değişimi şart.

### 4.4 Motor\_Advance, SysTick ve UART

Motor\_Advance() güncel adımı seçer, A/B/C pinlerini günceller ve aynı bilgiyi LPUART1 üzerinden yollar. Ana döngüde zamanlama:

```
const uint32_t now_ms = HAL_GetTick();
if ((uint32_t)(now_ms - last_step_ms) >= MOTOR_STEP_PERIOD_MS)
{
    last_step_ms += MOTOR_STEP_PERIOD_MS;
    Motor_Advance();
}
```

|        |  |       |  |       |  |       |
|--------|--|-------|--|-------|--|-------|
| STEP 1 |  | A: 1  |  | B: -1 |  | C: 0  |
| STEP 2 |  | A: 1  |  | B: 0  |  | C: -1 |
| STEP 3 |  | A: 0  |  | B: 1  |  | C: -1 |
| STEP 4 |  | A: -1 |  | B: 1  |  | C: 0  |
| STEP 5 |  | A: -1 |  | B: 0  |  | C: 1  |
| STEP 6 |  | A: 0  |  | B: -1 |  | C: 1  |

? **Neden HAL\_Delay(200) değil?** HAL\_Delay işlemciyi bloklar. HAL\_GetTick ile fark almak ana döngüyü serbest bırakır; ileride buton okuma veya UART RX eklemek kolaylaşır. last\_step\_ms += yazımı da birikimli kaymayı önler.

? **Tam bir tur ne kadar sürer?**  $6 \times 200 \text{ ms} = 1200 \text{ ms}$ , yani 1.2 s.

## 05 Derleme, karta yazma ve debug

CubeIDE'de proje adına tıkla ve çekiç simgesine bas. Beklenen çıktı:

```
11:12:12 Build Finished. 0 errors, 0 warnings. (took 934ms)
→ Debug/staj_motor_simule_g491.elf
```

Kart USB ile bağlıyken Run/Debug ile yükleme yapılır. ST-LINK firmware'i SWD üzerinden yazar ve MCU'yu resetler. DFU moduna gerek yok.

? **"ST-Link Server is required to launch the debug session" uyarısı?** Kod hatası değil; macOS'ta ST-LINK Server bileşeni eksik. Kurup IDE'yi yeniden aç, ya da STM32CubeProgrammer'da ST-LINK/SWD ile .elf dosyasını programla.

## 06 CoolTerm ile UART verisini görme

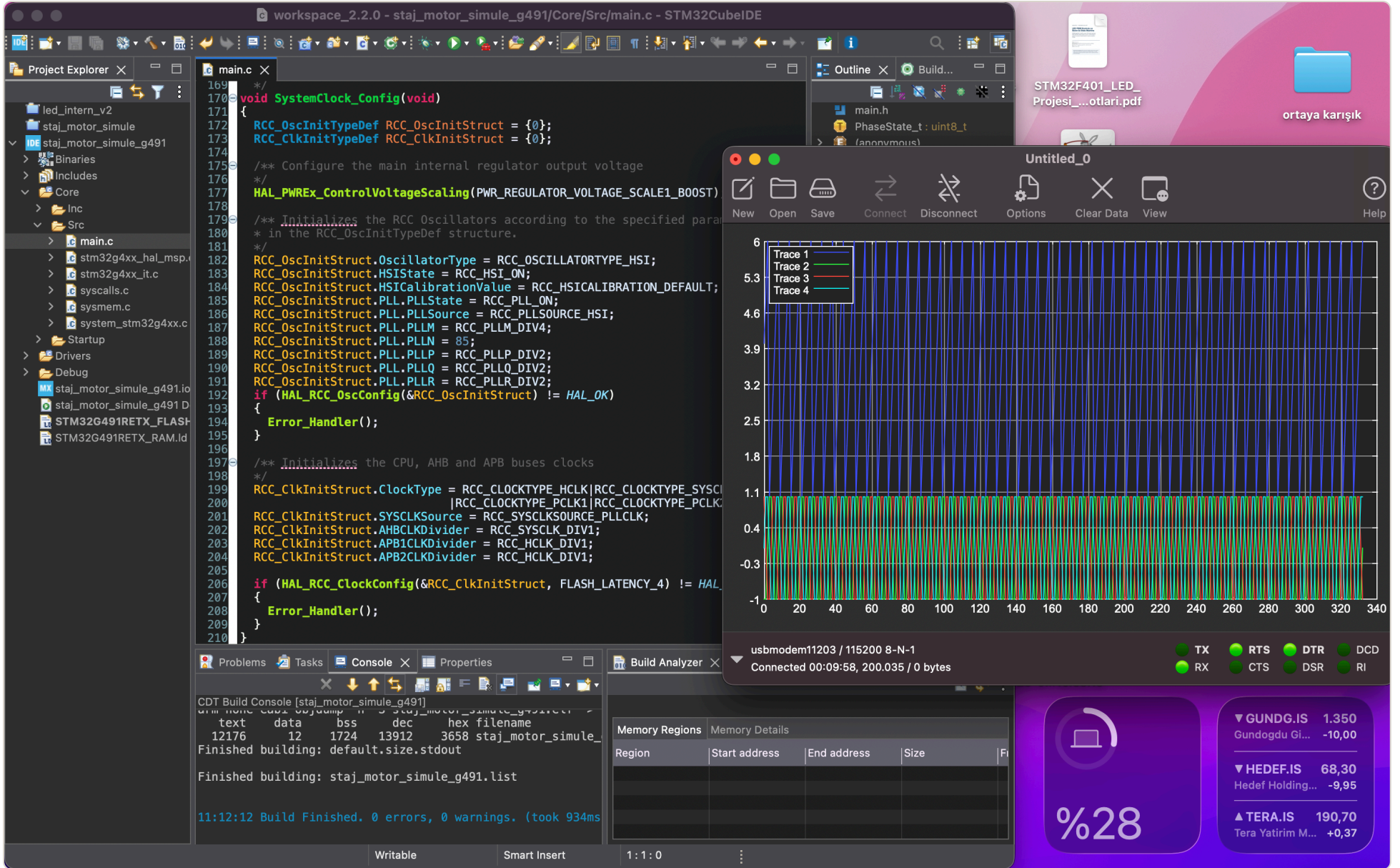
| Alan                 | Değer               |
|----------------------|---------------------|
| Serial Port          | /dev/cu.usbmodem... |
| Baudrate             | 115200              |
| Data / Parity / Stop | 8 / None / 1        |
| Flow Control         | None                |

Connect'e basınca STEP 1–6 satırları yaklaşık her 200 ms'de bir görünür (Şekil 4, sağ panel).

? **Port adı neden değişiyor?** Sondaki numara her bağlantıda farklı olabilir; önemli olan NUCLEO/ST-LINK'e ait `/dev/cu.usbmodem...` girdisini seçmek.

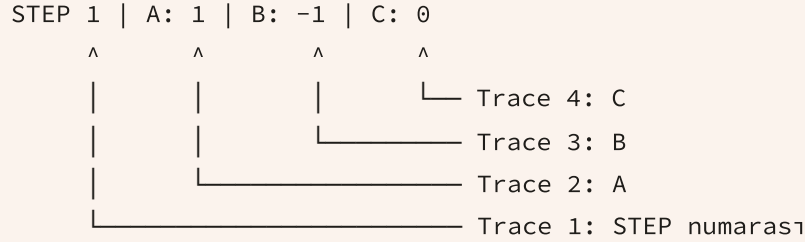
? **Metin bozuk geliyorsa?** İlk kontrol baud: hem CubeMX hem CoolTerm 115200. Sonra 8-N-1 ve Flow Control=None. Port görünmüyorsa veri taşıyan USB kablosu kullan; başka bir terminalin portu açık tutmadığından emin ol.

## 07 CoolTerm Chart: ayar ve yorum



Şekil 5 · Ham chart: Trace 1 (mavi) 1–6 arasında yükselen STEP numarası; Trace 2–4 ise –1/0/+1 arasında basamaklanan A/B/C fazları.

## 7.1 Neden dört trace oluşur?



CoolTerm satırdaki sayıları sırayla ayıklar. 0–6 arasında yükselen ilk eğri faz gerilimi değil, adım numarasıdır.

## 7.2 Temiz grafik için ayar

1. **View > Chart** görünümüne geç.
2. Grafik alanında sağ tık → **Chart Properties**.
3. *Only plot lines with exactly the specified number of data points* aç, değeri **4** yap.
4. Sağ tık → **Trace Properties**; Trace 1 için Visible kapat.
5. Trace 2: Phase A, Trace 3: Phase B, Trace 4: Phase C olarak adlandır.
6. **Clear Data**; 1–2 s yeni veri biriktir. Zoom: View > Zoom Chart veya Control + scroll.

## 7.3 Grafiği yorumlama

| Y değeri | Anlamı                                               |
|----------|------------------------------------------------------|
| +1       | Faz GPIO'su aktif 3.3 V (HIGH) sürüyor.              |
| 0        | FLOAT; pin input / yüksek empedans, aktif sürme yok. |
| -1       | Faz GPIO'su aktif 0 V / GND (LOW) sürüyor.           |

A: +1, +1, 0, -1, -1, 0, ...

B: -1, 0, +1, +1, 0, -1, ...

C: 0, -1, -1, 0, +1, +1, ...

? **Eğriler neden sinüs değil?** Six-step komutasyon basamaklı/trapezoidal sinyal üretir. Her adımda bir faz HIGH, bir faz LOW, bir faz FLOAT görülmesi doğru sonuçtur. Fazlar birbirine 120° (2 adım) kaymıştır.

? **Hem okunaklı metin hem temiz chart istersem?** İki satır türü gönder: STEP 1 | A: 1 | B: -1 | C: 0 terminal için, 1,-1,0 chart için. Chart Properties'te "yalnızca 3 sayı içeren satırları çiz" filtresi seçilir.

## 08 Başarı kontrol listesi

- ☐ **CubeIDE** · 0 errors, 0 warnings
- ☐ **ST-LINK** · Firmware karta yazılabiliyor
- ☐ **CoolTerm** · /dev/cu.usbmodem..., 115200 8-N-1, Flow Control=None
- ☐ **Terminal** · STEP 1'den 6'ya sıra sürekli tekrar ediyor
- ☐ **Chart** · A/B/C yalnızca -1, 0, +1 seviyelerinde

## 09 Sonraki adımlar

1. PA5/LD2 LED'ini her komutasyon döngüsünde değiştir.
2. PC13/B1 butonuyla başlat/durdur ekle.
3. UART RX ile s, p, +, - komutlarını al.
4. HAL\_GetTick() yerine timer interrupt kullan.
5. GPIO yerine timer PWM kanallarını öğren.
6. Hall sensörü / back-EMF geri beslemesi ve gerçek gate driver ekle.

Özet • CubeMX donanımı tanımlar; CubeIDE HAL kodunu derler; ST-LINK firmware'i SWD ile karta yazar; LPUART1 veriyi ST-LINK VCP üzerinden Mac'e taşır; CoolTerm veriyi terminal ve grafik olarak görünür yapar.

## 10 Terimler sözlüğü

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| BLDC            | Fırçasız DC motor. Komütasyon mekanik fırça yerine elektronik olarak yapılır.  |
| Six-step        | Altı adımlı trapezoidal komutasyon; her adımda iki faz sürülür, biri serbest.  |
| HAL             | Hardware Abstraction Layer. ST'nin çevre birimlerini soyutlayan C kütüphanesi. |
| GPIO            | Genel amaçlı giriş/çıkış pini.                                                 |
| Push Pull       | Pinin hem 3.3 V'a hem GND'ye aktif sürebildiği çıkış modu.                     |
| Yüksek empedans | Pinin sürülmediği, "boşta" durum (FLOAT / input modu).                         |
| LPUART          | Low-Power UART. Asenkron seri haberleşme birimi.                               |
| 8-N-1           | 8 veri biti, parity yok, 1 stop biti.                                          |
| SWD             | Serial Wire Debug. İki telli (SWDIO, SWCLK) programlama/debug arayüzü.         |
| VCP             | Virtual COM Port. ST-LINK'in UART'ı USB üzerinden seri port gibi göstermesi.   |
| HSI / PLL       | Dahili 16 MHz osilatör ve onu 170 MHz'e çarpan faz kilitli döngü.              |
| SysTick         | Cortex-M çekirdek zamanlayıcısı; HAL_GetTick() ms sayacını besler.             |
| .ioc            | CubeMX proje dosyası; pin ve çevre birimi ayarlarını tutar.                    |
| .elf            | Derlenmiş firmware dosyası; ST-LINK bunu karta yazar.                          |

## 11 Kendi izleme uygulamamız: CoolTerm'ün ötesi

CoolTerm genel amaçlı bir terminal; satırdaki sayıları çizer ama STEP'in ne olduğunu bilmez. Motor kontrolüne özel bir uygulama, gelen veriyi *anlayan* ve karta komut da gönderebilen bir araç olur. Bu bölüm, şu anki step takibinden ileride gerçek motor kontrolüne kadar neyin eklenebileceğini planlar.

? **CoolTerm varken neden kendi uygulamamızı yazalım?** Üç eksik var: (1) Trace 1'in STEP olduğunu bilmiyor, hep elle filtre gerekiyor; (2) tek yönlü, karta hız/başlat/durdur gönderemiyor; (3) veri anlamsız sayı; RPM, adım süresi, hata sayısı gibi türetilmiş bilgi üretmiyor.

### 11.1 Mimari: veri zinciri değişmiyor, sadece son halka

KART TARAFI

MAC · IZLEME UYGULAMASI



Şekil 6 · Uygulama mimarisi. Sol sütun mevcut zincirin aynısı; sağdaki üç katman yeni yazılacak kısım. Katmanlar arasında yalnızca ok yönündeki bağımlılık var.

Uygulama üç katmandan oluşur: **seri katman** (port aç/kapat, byte oku/yaz), **veri katmanı** (satırı ayrıştır, tampona yaz, istatistik üret), **görünüm katmanı** (terminal, grafik, kontrol paneli). Katmanlar ayrı tutulursa ileride protokol değişince yalnızca parser değişir.

## 11.2 Teknoloji seçenekleri

| Yol                                | Seri erişim              | Grafik                   | Ne zaman uygun                                                               |
|------------------------------------|--------------------------|--------------------------|------------------------------------------------------------------------------|
| <b>Python</b>                      | pyserial                 | pyqtgraph / matplotlib   | En hızlı başlangıç; staj süresinde bitirilebilir. PyQt ile masaüstü pencere. |
| <b>Web (tarayıcı)</b>              | Web Serial API           | Canvas / uPlot           | Kurulum yok, Chrome'da çalışır. macOS'ta doğrudan usbmodem'e bağlanır.       |
| <b>Masaüstü (Electron / Tauri)</b> | serialport (Node) / Rust | Web grafik kütüphaneleri | Dağıtılabılır .app istenirse. Web sürümünün üstüne inşa edilir.              |
| <b>C++ / Qt</b>                    | QSerialPort              | QtCharts                 | Endüstriyel araç hedefleniyorsa; öğrenme eğrisi en dik.                      |

? **Hangisiyle başlamalı?** Python + pyserial + pyqtgraph. Seri okuma 10 satır, canlı grafik 20 satır kod. Protokol oturduktan sonra web veya masaüstüne taşınır; parser mantığı aynı kalır.

## 11.3 Aşama 1 · Şimdiki firmware ile yapılabilenler

Firmware değişmeden, mevcut STEP n | A: x | B: y | C: z satırıyla:

### Terminal

Ham satırlar, zaman damgalı. Renk: HIGH turuncu, LOW mavi, FLOAT gri.

### Faz grafiği

A/B/C ayrı eksenlerde, -1/0/+1 basamaklı. STEP otomatik olarak grafikten ayrılır.

### Komutasyon göstergesi

6 dilimli daire; aktif adım vurgulu. Fazların motor etrafında dönüşü görünür olur.

### Türetilmiş değerler

Adım süresi (ms), elektriksel RPM =  $60000 / (6 \times \text{periyot})$ , toplam tur, kaçırılan adım sayısı.

### Doğrulama

Her satırda tam bir HIGH, bir LOW, bir FLOAT var mı? Adım sırası 1→6 mı? Hata sayacı.

### Kayıt

CSV/JSON'a zaman damgalı log; sonra offline analiz veya tekrar oynatma.

? **Adım süresi neden ölçülmeli?** Firmware 200 ms diyor; bilgisayar tarafında ölçülen süre farklıysa ya USB tamponlaması ya da ana döngüde bloklama var. Bu ölçüm ileride timer interrupt'a geçişin etkisini de gösterir.

## 11.4 Aşama 2 · İki yönlü: karta komut göndermek

Firmware'de UART RX açılır (PA3 zaten ayarlı). Basit tek karakterlik komut seti:

| Komut | Anlamı                             | Uygulamada          |
|-------|------------------------------------|---------------------|
| s     | Start                              | Başlat düğmesi      |
| p     | Pause / stop                       | Durdur düğmesi      |
| + / - | Periyodu $\pm 50$ ms değiştir      | Hız kaydırıcısı     |
| d     | Yön değiştir (tabloyu tersten gez) | CW / CCW anahtarı   |
| n     | Tek adım ilerle (manuel mod)       | Adım düğmesi        |
| ?     | Durum raporu iste                  | Bağlanınca otomatik |

? **Firmware tarafında RX nasıl alınır?** HAL\_UART\_Receive\_IT(&hlpuart1, &rx\_byte, 1) ile interrupt bazlı tek byte; CubeMX'te LPUART1 global interrupt kutusu işaretlenir (Şekil 2'de NVIC Settings). Ana döngü bloklanmaz, komut anında işlenir.

## 11.5 Protokolü büyütme: metinden yapılandırılmış veriye

Şu anki satır insan için okunaklı ama makine için gevşek. Uygulama büyüdükçe seçenekler:

```
# A • CSV satırı (basit, CoolTerm ile de uyumlu)
```

```
t_ms,step,a,b,c,period_ms
```

```
12400,3,0,1,-1,200
```

```
# B • JSON satırı (alan eklemek kolay, kendi uygulaman için)
```

```
{"t":12400,"step":3,"ph":[0,1,-1],"per":200,"dir":1}
```

```
# C • Binary çerçeve (yüksek hız, gerçek motor için)
```

```
[0xAA][len][type][payload...][crc8]
```

? **Neden zaman damgasını kart göndersin?** Bilgisayarın okuma zamanı USB tamponu yüzünden  $\pm 10$  ms oynar. HAL\_GetTick() değeri satıra eklenirse grafik gerçek zamanı gösterir; periyot analizi doğru olur.

? **Binary'ye ne zaman geçilir?** PWM ve akım ölçümü eklendiğinde ms başına onlarca örnek gelir; metin 115200 baud'u doldurur. Binary çerçeve + CRC hem hız hem hata tespiti sağlar. O noktada baud da 921600'e çıkarılabilir.

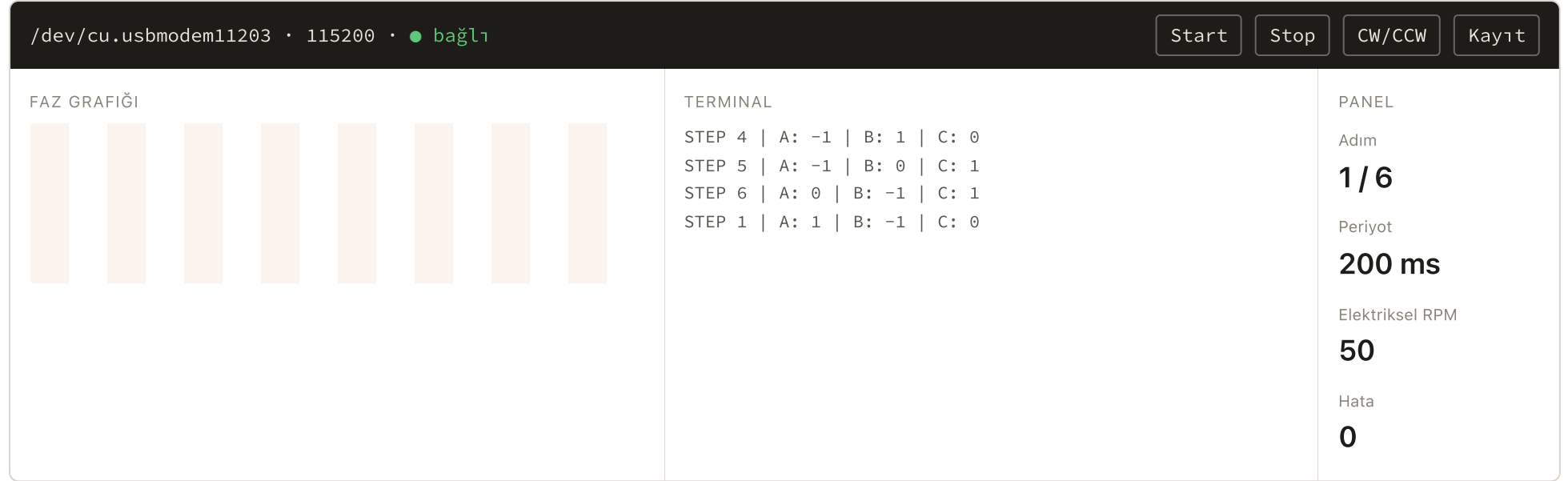
## 11.6 Aşama 3 • Gerçek motor kontrolüne doğru izlenecekler

Firmware geliştikçe (bkz. Bölüm 9) uygulamaya eklenecek paneller:

| Firmware adımı    | Karttan gelen yeni veri                   | Uygulamada görünüm                                             |
|-------------------|-------------------------------------------|----------------------------------------------------------------|
| Timer interrupt   | Kesin periyot, jitter                     | Periyot histogramı; hedef vs ölçülen                           |
| Timer PWM         | Duty cycle (%), PWM frekansı              | Duty kaydırıcısı; faz grafiği -1/0/+1 yerine -duty...+duty     |
| Hall sensörü      | Hall durumu (3 bit), gerçek rotor konumu  | Komutasyon dairesinde beklenen vs ölçülen adım; senkron hatası |
| Back-EMF          | FLOAT fazın ADC gerilimi, sıfır geçiş anı | Osiloskop görünümü; sıfır geçiş işaretleri                     |
| Akım ölçümü       | Faz akımları, bus akımı                   | Akım grafiği; aşırı akım alarmı ve eşik çizgisi                |
| Kapalı çevrim hız | Hedef RPM, ölçülen RPM, PID çıkışı        | Hedef/ölçülen üst üste; PID kazançlarını canlı ayarlama        |
| Koruma            | Hata kodları, sıcaklık, bus gerilimi      | Durum ışıkları; hata günlüğü; acil durdur düğmesi              |

? **Gerçek motorda acil durdur nasıl güvenli olur?** Yazılım komutuna tek başına güvenilmez; USB kopabilir. Uygulama p gönderir ama firmware'de ayrıca watchdog olmalı: belirli süre komut gelmezse fazları FLOAT'a al. Donanım koruması (sigorta, akım limiti) her zaman yazılımın üstündedir.

## 11.7 Ekran taslağı



Şekil 7 · Üstte bağlantı ve komut çubuğu; solda faz grafiği, ortada terminal, sağda türetilmiş değerler paneli. Komutasyon dairesi ikinci sekmede yer alabilir.

## 11.8 Geliştirme sırası

1. Port listele, bağlan, ham satırları göster. CoolTerm ile birebir aynı davranış: referans noktası.
2. Parser: `STEP n | A: x | B: y | C: z` → `{step, a, b, c, t}`. Hatalı satırları say, atma.
3. Faz grafiği + komutasyon dairesi. Aşama 1 türetilmiş değerler.
4. CSV kayıt ve tekrar oynatma. Kartsız geliştirme mümkün olur.
5. Firmware'e RX ekle; `s / p / + / -` komutları. Hız kaydırıcısı.
6. Protokolü CSV veya JSON'a taşı; zaman damgasını karta al.
7. Firmware ilerledikçe 11.6'daki panelleri tek tek ekle.

?

**Kayıt/tekrar oynatma neden erken?** Bir kez kaydedilen 30 saniyelik log ile grafik ve panel kartsız geliştirilir; her denemede kartı bağlamak gerekmez. Hata ayıklamada "o an ne oldu" sorusunu da cevaplar.

## 12 Ek · main.c satır satır

Bölüm 4 mantığı anlattı; burada dosyanın tamamı yukarıdan aşağıya, her blok yanında ne yaptığı ile. CubeMX'in ürettiği kısımlar gri, bizim eklediğimiz USER CODE blokları koyu arka planda.

### 12.1 Dosyanın haritası

|                         |                                                                                                                            |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------|
| #include "main.h"       | HAL, GPIO ve UART tanımları. CubeMX üretir.                                                                                |
| USER CODE PTD / PD / PM | Bizim: PhaseState_t, sabitler, PACK/UNPACK makroları.                                                                      |
| hlpuart1                | UART tanıtıcısı (handle). CubeMX üretir.                                                                                   |
| USER CODE PV            | Bizim: current_step, commutation_table, chart metinleri.                                                                   |
| Prototipler             | SystemClock_Config, MX_GPIO_Init, MX_LPUART1_UART_Init. CubeMX üretir.                                                     |
| USER CODE PFP + 0       | Bizim: Set_Phase ve Motor_Advance prototipleri ve gövdeleri.                                                               |
| main()                  | HAL_Init → SystemClock_Config → MX_GPIO_Init → MX_LPUART1_UART_Init → <b>USER CODE 2</b> → while(1) { <b>USER CODE 3</b> } |
| SystemClock_Config      | HSI + PLL → 170 MHz. CubeMX üretir (Şekil 4'te görünen kod).                                                               |
| MX_LPUART1_UART_Init    | 115200 8-N-1, TX/RX. CubeMX üretir.                                                                                        |
| MX_GPIO_Init            | Port saatlerini açar, PA4/PB4/PB5'i Output PP Low yapar. CubeMX üretir.                                                    |
| Error_Handler           | Kesmeleri kapatır, sonsuz döngü. CubeMX üretir.                                                                            |

? **USER CODE bloklarının adları ne anlama geliyor?** PTD = Private TypeDef, PD = Private Define, PM = Private Macro, PV = Private Variables, PFP = Private Function Prototypes, 0 = fonksiyon gövdeleri, 2 = main içinde init sonrası, 3 = while döngüsü içi. Bu sıra CubeMX'in kendi dosya düzenidir.

## 12.2 Tipler, sabitler, makrolar

```
/* USER CODE BEGIN PTD */
typedef uint8_t PhaseState_t;
enum { PHASE_FLOAT = 0U, PHASE_LOW = 1U, PHASE_HIGH = 2U };
/* USER CODE END PTD */

/* USER CODE BEGIN PD */
#define MOTOR_STEP_COUNT      6U
#define MOTOR_STEP_PERIOD_MS 200U
/* USER CODE END PD */

/* USER CODE BEGIN PM */
#define PACK_PHASES(a, b, c)  (uint8_t)(((a) << 4) | ((b) << 2) | (c))
#define UNPACK_PHASE(v, shift) (PhaseState_t)(((v) >> (shift)) & 0x03U)
/* USER CODE END PM */
```

| Satır                                            | Ne yapar                                                                                          |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>typedef uint8_t<br/>PhaseState_t</code>    | Faz durumunu 1 byte'lık isimli tip yapar; fonksiyon imzaları okunur olur.                         |
| <code>enum { FLOAT=0, LOW=1,<br/>HIGH=2 }</code> | Üç durum, 2 bite sığar (0–3). FLOAT'ın 0 olması: sıfırlanmış bir değer güvenli durumdur.          |
| <code>MOTOR_STEP_PERIOD_MS 200U</code>           | Adım süresi. Tek yerden değişir; ileride + / – komutu bunu değiştirecek bir değişkene dönüşür.    |
| <code>PACK_PHASES(a,b,c)</code>                  | A'yı 4–5. bite, B'yi 2–3. bite, C'yi 0–1. bite koyar: 00AA BBCC. Tablo 6 byte tutar.              |
| <code>UNPACK_PHASE(v, shift)</code>              | Sağa kaydır, & 0x03 ile 2 biti al. shift: A için 4, B için 2, C için 0.                           |
| <code>U_soneki</code>                            | Sabit unsigned yapar; işaretli/işaretsiz karşılaştırma uyarılarını önler. "0 warnings" bu yüzden. |

## 12.3 Değişkenler

```
/* USER CODE BEGIN PV */
static uint8_t current_step = 0U;

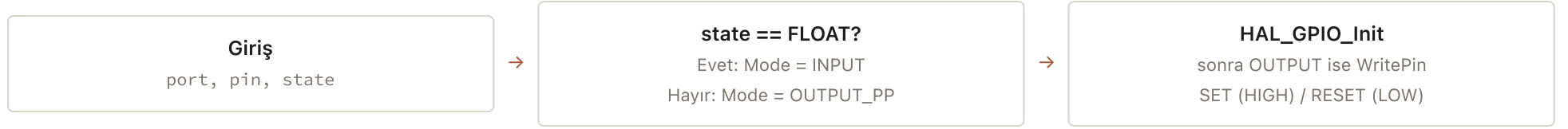
static const uint8_t commutation_table[MOTOR_STEP_COUNT] = {
    PACK_PHASES(PHASE_HIGH, PHASE_LOW, PHASE_FLOAT), /* 1 */
    PACK_PHASES(PHASE_HIGH, PHASE_FLOAT, PHASE_LOW), /* 2 */
    PACK_PHASES(PHASE_FLOAT, PHASE_HIGH, PHASE_LOW), /* 3 */
    PACK_PHASES(PHASE_LOW, PHASE_HIGH, PHASE_FLOAT), /* 4 */
    PACK_PHASES(PHASE_LOW, PHASE_FLOAT, PHASE_HIGH), /* 5 */
    PACK_PHASES(PHASE_FLOAT, PHASE_LOW, PHASE_HIGH) /* 6 */
};

static const char chart_lines[][40] = { ... }; /* CoolTerm başlığı */
static const char chart_legend[] = "...";
/* USER CODE END PV */
```

| Satır                                         | Ne yapar                                                                                                              |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>static uint8_t current_step</code>      | 0–5 arası indeks. static: yalnızca bu dosyadan görünür, değer çağrılar arasında korunur.                              |
| <code>static const<br/>uint8_t table[]</code> | const ile Flash'a yerleşir, RAM harcamaz. Build çıktısındaki <code>data = 12 byte</code> 'in küçük kalmasının nedeni. |
| <code>chart_lines / chart_legend</code>       | Bağlantı anında bir kez gönderilen açıklama satırları; CoolTerm'de veri başlamadan önce ne izlendiğini gösterir.      |

## 12.4 Set\_Phase

Kod Bölüm 4.3'te. Burada yalnızca akış:



? **GPIO\_InitTypeDef gpio = {0} neden?** Yapının tüm alanlarını sıfırlar; Alternate gibi kullanmadığımız alanlar rastgele değer taşımaz. Yığında (stack) tanımlanan yapılar sıfırlanmadan gelir.

## 12.5 Motor\_Advance

```
static void Motor_Advance(void)
{
    const uint8_t packed = commutation_table[current_step];
    const PhaseState_t a = UNPACK_PHASE(packed, 4);
    const PhaseState_t b = UNPACK_PHASE(packed, 2);
    const PhaseState_t c = UNPACK_PHASE(packed, 0);

    Set_Phase(PHASE_A_GPIO_Port, PHASE_A_Pin, a);
    Set_Phase(PHASE_B_GPIO_Port, PHASE_B_Pin, b);
    Set_Phase(PHASE_C_GPIO_Port, PHASE_C_Pin, c);

    char line[48];
    const int n = snprintf(line, sizeof line, "STEP %u | A: %d | B: %d | C: %d\r\n",
                           (unsigned)(current_step + 1U),
                           Phase_To_Int(a), Phase_To_Int(b), Phase_To_Int(c));
    HAL_UART_Transmit(&hlpuart1, (uint8_t *)line, (uint16_t)n, 100U);

    current_step = (uint8_t)((current_step + 1U) % MOTOR_STEP_COUNT);
}
```

| Satır                                         | Ne yapar                                                                                                            |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>packed =<br/>table[current_step]</code> | Bu adımın 1 byte'ını çeker.                                                                                         |
| <code>UNPACK_PHASE(packed, 4/2/0)</code>      | A, B, C'yi ayırır.                                                                                                  |
| <code>PHASE_A_GPIO_Port / _Pin</code>         | CubeMX'teki User Label'dan otomatik üretilen makrolar (main.h). PA4 yerine isim kullanmak pin değişirse kodu korur. |
| <code>Phase_To_Int</code>                     | HIGH→+1, LOW→-1, FLOAT→0. CoolTerm chart'ın -1/0/+1 görmesinin kaynağı.                                             |
| <code>snprintf(... "\r\n")</code>             | Satırı tamponda kurar. \r\n terminalin satır atlaması için; CoolTerm satır sonunu buradan tanır.                    |
| <code>HAL_UART_Transmit(...,<br/>100U)</code> | Bloklayan gönderim, 100 ms zaman aşımı. ~30 byte 115200'de 2.6 ms sürer; 200 ms periyoda göre ihmal edilir.         |
| <code>(current_step + 1) % 6</code>           | 5'ten sonra 0'a döner; tablo döngüsel gezilir.                                                                      |

? **Neden önce pinler, sonra UART?** Pin güncellemesi mikrosaniyeler sürer, UART milisaniyeler. Fazlar önce değişince satır "şu an olan"ı bildirir; tersi olsaydı satır 2–3 ms ileriye anlatırdı.

## 12.6 main(): başlangıç ve ana döngü

```
int main(void)
{
    HAL_Init();                /* SysTick 1 ms, NVIC, Flash cache */
    SystemClock_Config();      /* 170 MHz */
    MX_GPIO_Init();            /* PA4/PB4/PB5 Output Low */
    MX_LPUART1_UART_Init();    /* 115200 8-N-1 */

    /* USER CODE BEGIN 2 */
    Send_Chart_Header();        /* chart_lines + legend → CoolTerm */
    uint32_t last_step_ms = HAL_GetTick();
    Motor_Advance();            /* beklemeden STEP 1'i uygula */
    /* USER CODE END 2 */

    while (1)
    {
        /* USER CODE BEGIN 3 */
        const uint32_t now_ms = HAL_GetTick();
        if ((uint32_t)(now_ms - last_step_ms) >= MOTOR_STEP_PERIOD_MS)
        {
            last_step_ms += MOTOR_STEP_PERIOD_MS;
            Motor_Advance();
        }
        /* USER CODE END 3 */
    }
}
```

| Satır                  | Ne yapar                                                                                                                |
|------------------------|-------------------------------------------------------------------------------------------------------------------------|
| HAL_Init()             | SysTick'i 1 ms'ye kurar; HAL_GetTick ve HAL_Delay buna dayanır. Her zaman ilk çağrı.                                    |
| Init sırası            | Saat → GPIO → UART. UART, kendi pinlerinin (PA2/PA3) GPIO saatinin açık olmasına ihtiyaç duyar; CubeMX bu sırayı korur. |
| İlk Motor_Advance()    | Döngüye girmeden STEP 1 uygulanır; CoolTerm'e bağlanınca 200 ms boş beklenmez.                                          |
| (uint32_t)(now - last) | Fark unsigned alınır: HAL_GetTick 49.7 günde taşsa bile fark doğru çıkar.                                               |
| last_step_ms += PERIOD | = now_ms yazılsaydı döngünün gecikmesi her adımda birikirdi. Toplama ile ortalama periyot tam 200 ms kalır.             |

? **SystemClock\_Config'e neden dokunmadık?** Şekil 4'teki kod CubeMX'in ürettiği hâli. 170 MHz bu iş için fazlasıyla yeterli; PLL değerlerini elle değiştirmek yanlış saat ve çalışmayan UART riski taşır. Baud hesabı bu saate göre CubeMX tarafından yapılır.

? **Build çıktısı ne diyor?** text 12176: kod + const tablolar (Flash). data 12: ilk değerli değişkenler. bss 1724: sıfırlanan RAM (hlpuart1 yapısı, yığın). Toplam 13.9 KB; G491'in 512 KB Flash'ının %3'ü.

# NUCLEO-G491RE — Uygulama Görüntüleri

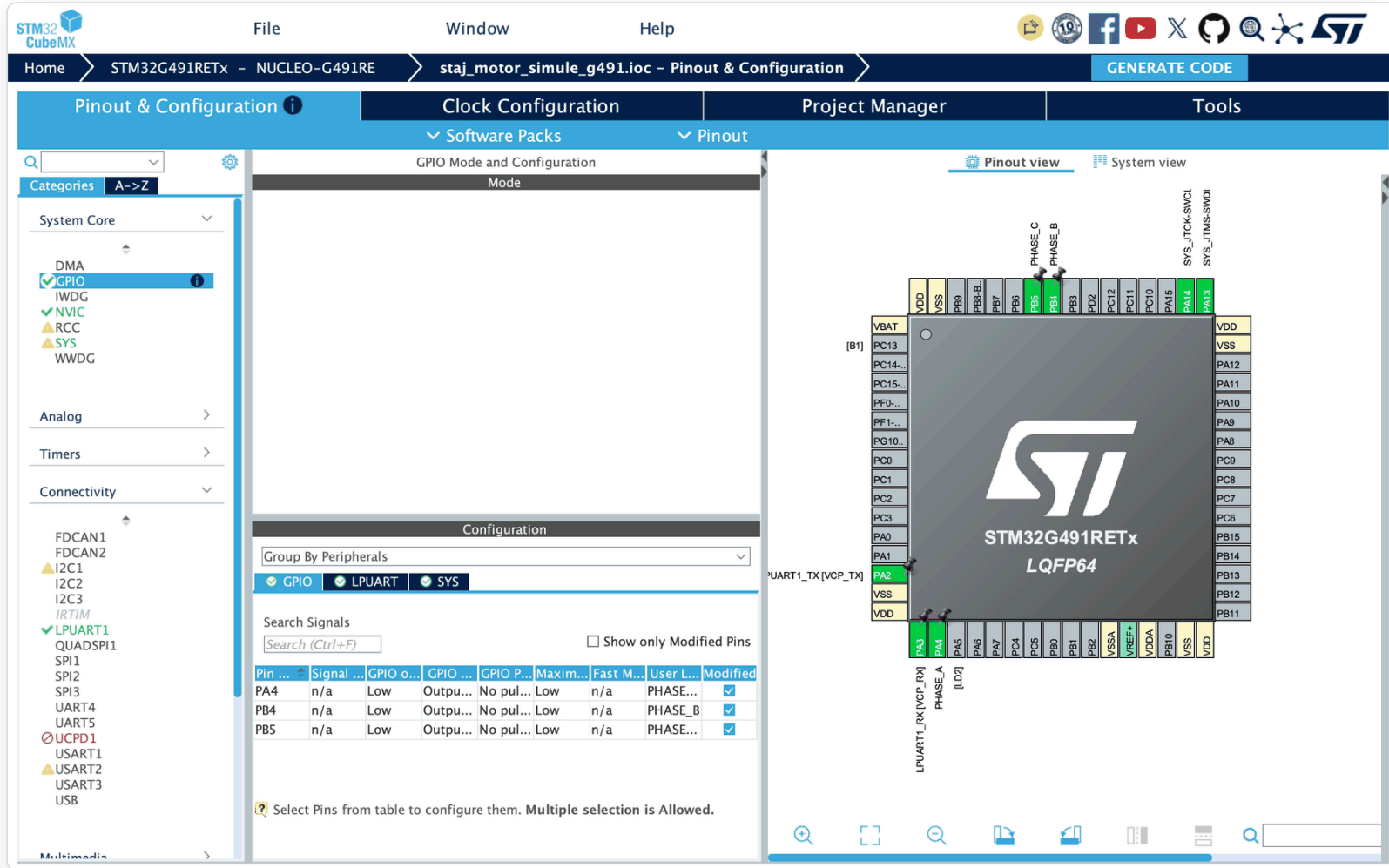
NUCLEO-G491RE ile yapılan çalışmanın ekran görüntüleri: CubeMX yapılandırması, CubeIDE derlemesi ve ST-LINK ile karta yazma, CoolTerm ile seri telemetri ve nihayet staj sırasında yazılan kendi izleme arayüzü. Görüntüler CoolTerm'den kendi uygulamamıza doğru ilerler.

---

## BU BÖLÜMDEKİ GÖRÜNTÜLER

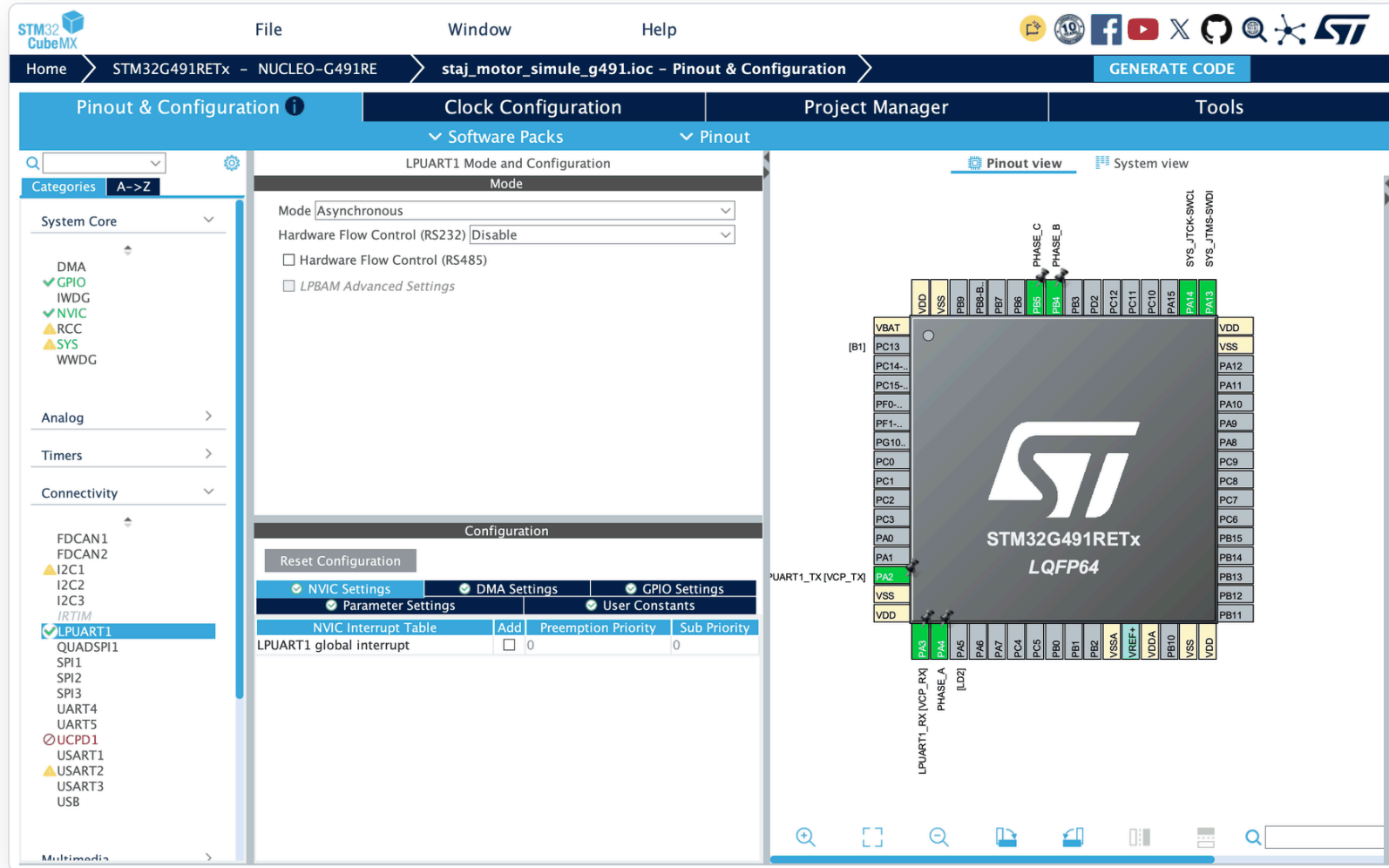
- 1–3** CubeMX — pinout, LPUART1 ve SYS yapılandırması
- 4–5** CubeIDE — saat kurulumu, derleme ve CoolTerm
- 6–9** Kendi izleme uygulamamız — animasyon, canlı telemetri, encoder

## CubeMX • Pinout (NUCLEO-G491RE)

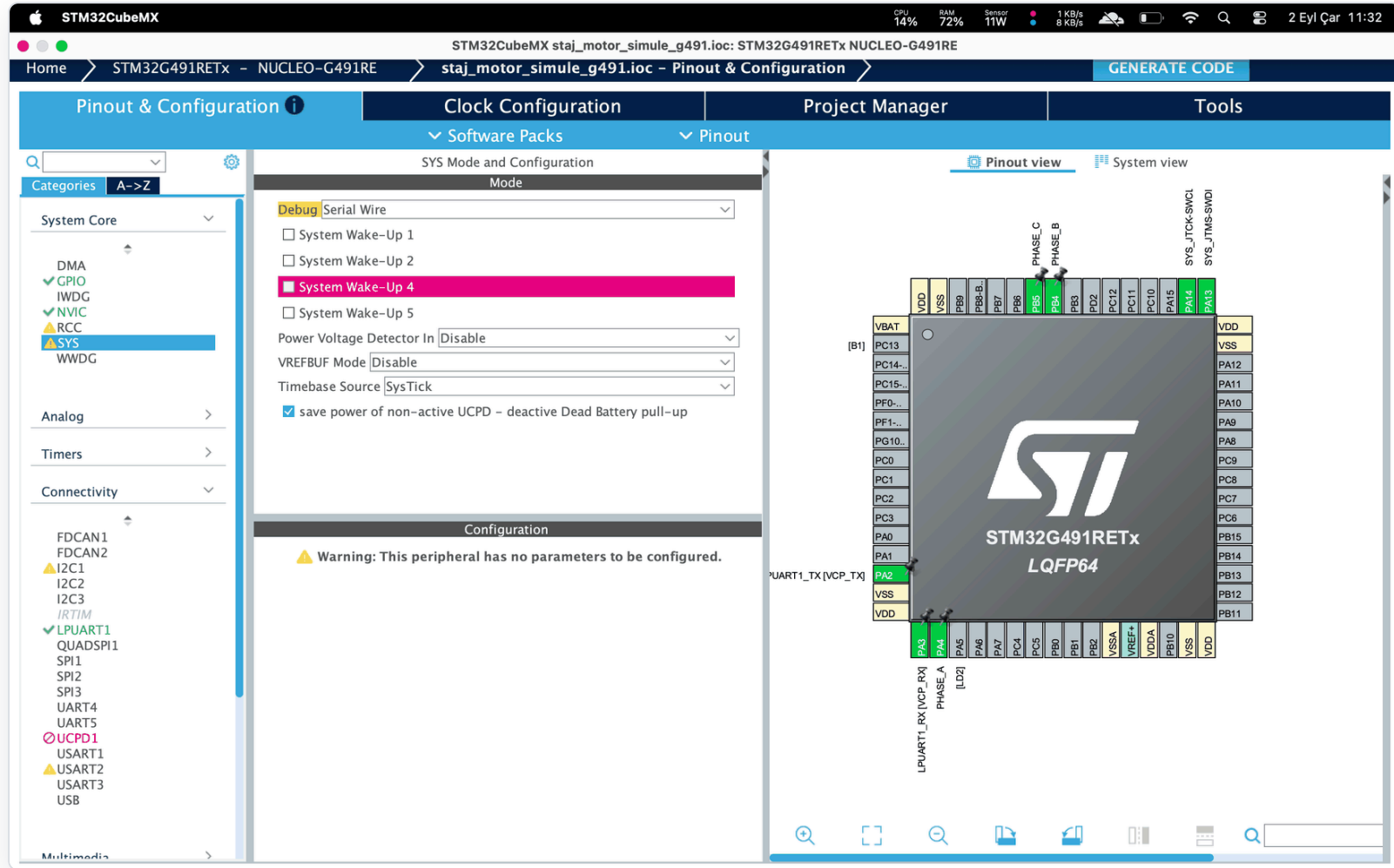


ŞEKİL 1 STM32G491RETx pinout'u. PA4 / PB4 / PB5 faz çıkışları (PHASE\_A/B/C); PA2 / PA3 ise LPUART1'in sanal COM portu (VCP\_TX / VCP\_RX) — telemetri ST-LINK üzerinden PC'ye gider.

## CubeMX · LPUART1

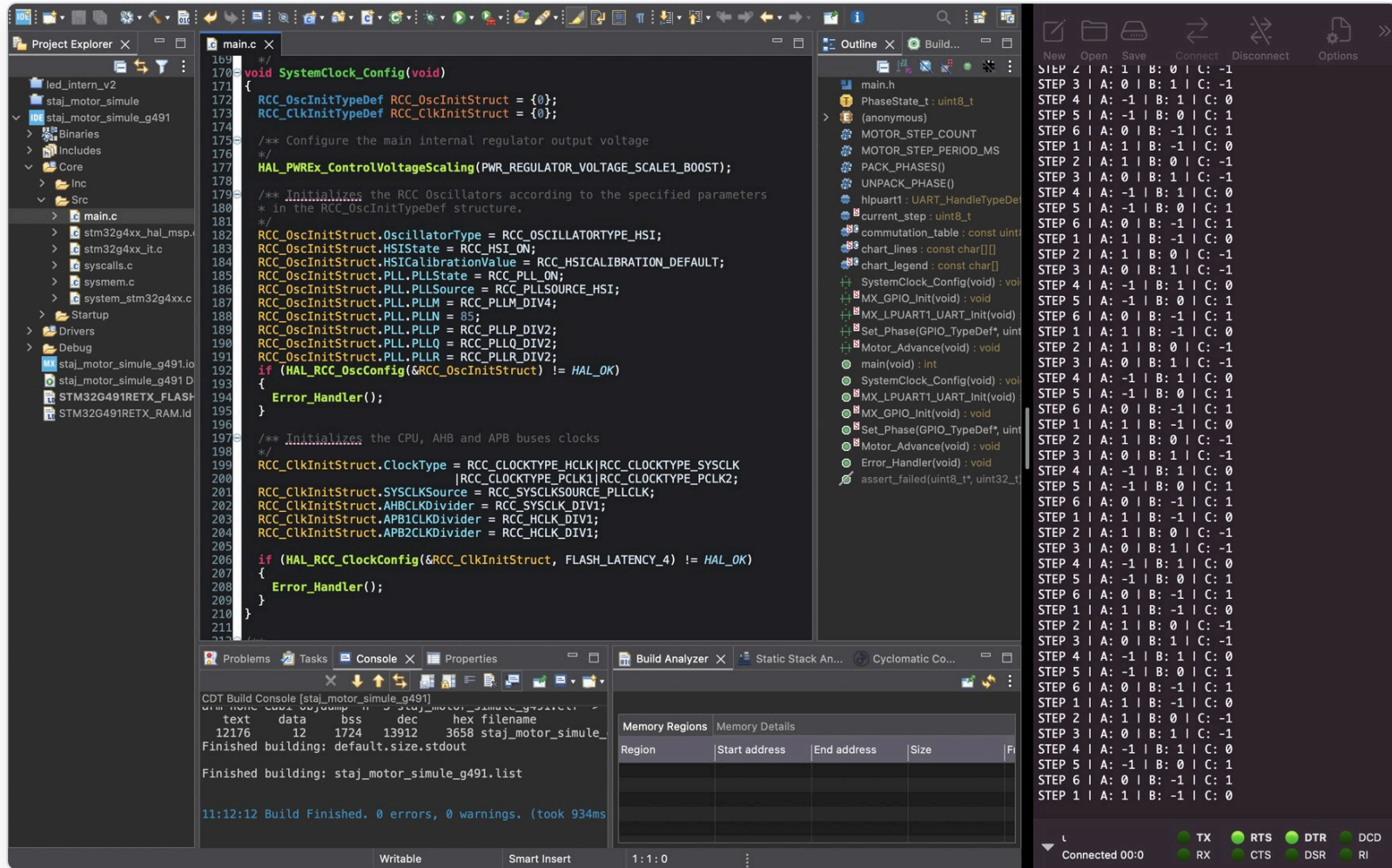


**ŞEKİL 2** LPUART1 **asenkron mod**, donanım akış kontrolü kapalı. Faz durumları bu hat üzerinden **115200 8-N-1** ile bilgisayara aktarılır.



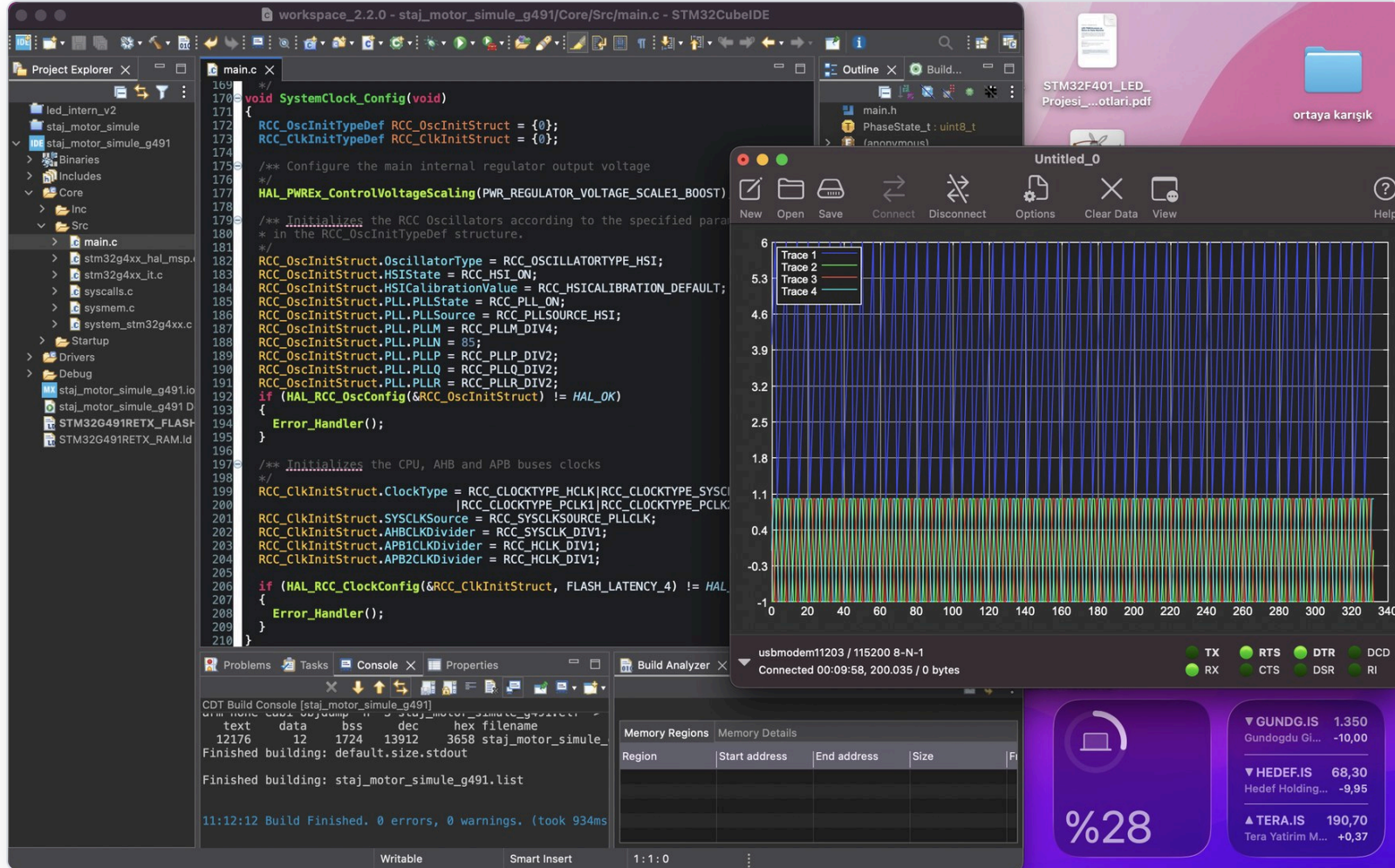
**ŞEKİL 3** Sistem ayarları: hata ayıklama için **Serial Wire** (SWD) ve zaman tabanı olarak **SysTick**. ST-LINK ile karta yazma ve adımlama bu arayüz üzerinden yapılır.

## CubeIDE · SystemClock\_Config + CoolTerm



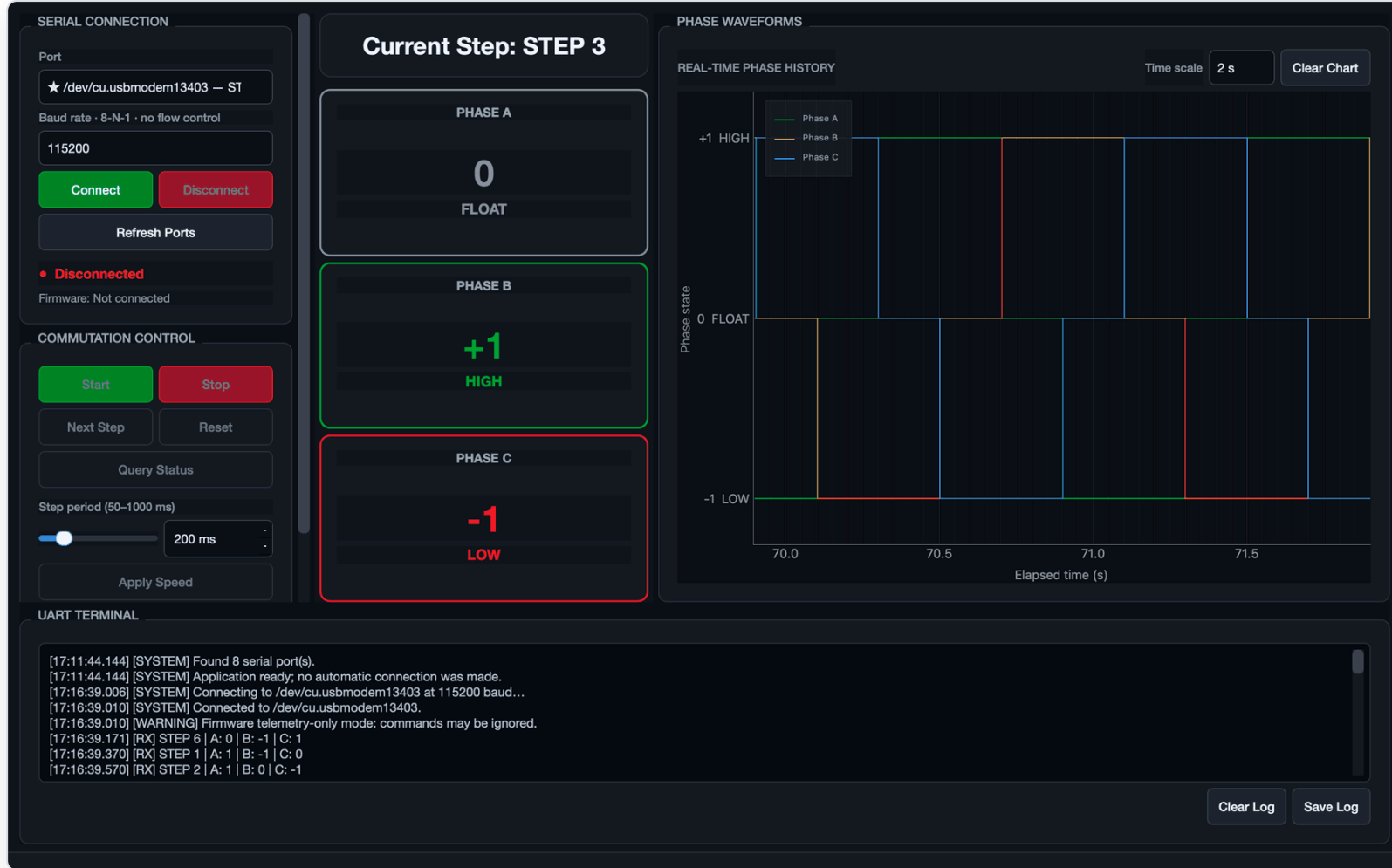
**ŞEKİL 4** CubelIDE'de **SystemClock\_Config**: HSI kaynağı PLL ile **170 MHz**'e çıkarılıyor. Sağda CoolTerm'de **STEP** satırları — her adımda A/B/C fazlarının +1 / 0 / -1 (HIGH / FLOAT / LOW) durumu.

## CubeIDE • Derleme + CoolTerm Chart



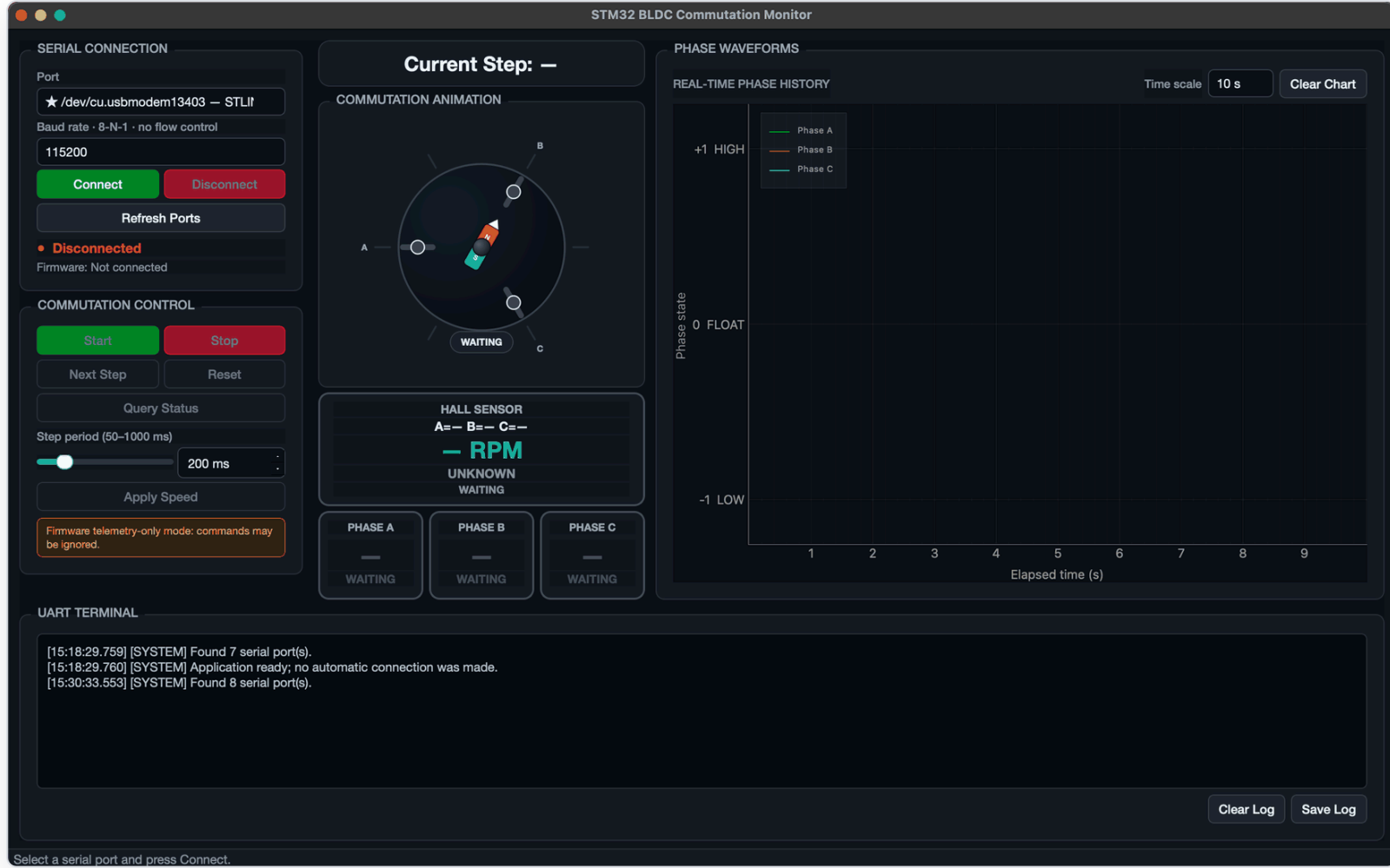
**ŞEKİL 5** Derleme **0 hata, 0 uyarı** ile tamamlandı. CoolTerm'ün grafik (Chart) görünümünde faz durumları kare dalga olarak çizilir; altı adımlı komütasyonun periyodik yapısı burada görülür.

## Kendi İzleme Uygulamamız — Başlangıç



**ŞEKİL 6** CoolTerm'ün ötesi: staj sırasında yazılan **STM32 BLDC Commutation Monitor**. Sol panelde seri bağlantı, ortada anlık adım ve faz kartları, sağda gerçek zamanlı faz geçişi grafiği (bağlantı bekleniyor).

## İzleme Uygulaması — Komütasyon Animasyonu



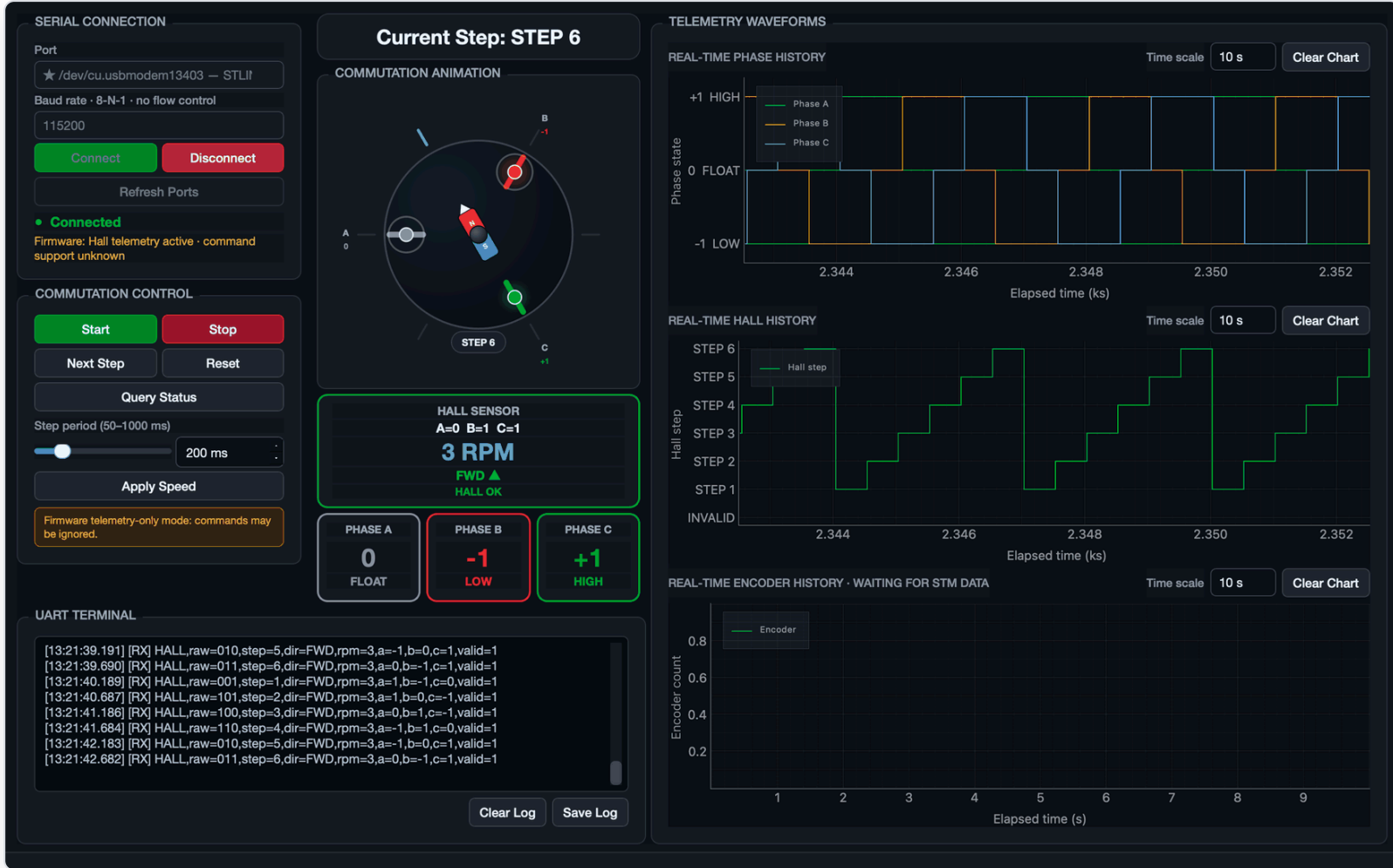
ŞEKİL 7 Uygulamanın komütasyon animasyonu: rotor mıknatısı ve üç faz konumu görsel olarak canlandırılır; altında **Hall sensör** durumu ve **RPM** göstergesi.

## İzleme Uygulaması — Canlı Telemetri



**ŞEKİL 8** Bağlı durumda **STEP 2**: Hall telemetrisi aktif, **8 RPM ileri (FWD)**, HALL OK. Sağdaki faz dalga formları ve alttaki UART terminali (raw=..., step=..., dir=FWD, rpm=...) canlı akıyor.

## İzleme Uygulaması — Genişletilmiş Telemetri



ŞEKİL 9 Son aşama: aynı arayüzde **faz geçmişi**, **Hall adım geçmişi** (STEP 1–6 merdiveni) ve **encoder telemetri** paneli bir arada. Hall + encoder verisi rotor konumu ve hız takibini birlikte gösterir.